EKNATHALABS · INTERVIEW PREP

Kubernetes Production

Platform, SRE & DevOps interview guide

Questions	**170 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/kubernetes_interview_guide.html**
Printed	**2026-10-21**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

certs.eknathalabs.com

Q1 **What happens when a pod is stuck in Pending state — how do you debug it?** Pods & workloads

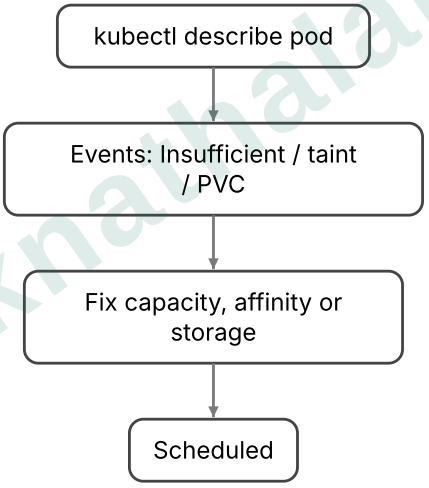
🎙 Spoken answer · ~68 s

Pending means the scheduler hasn't placed the pod on a node, so I start with kubectl describe pod and read the Events section, because the scheduler almost always says why. The usual suspects are: CPU or memory requests larger than any node's allocatable capacity, a nodeSelector or affinity rule nothing satisfies, a taint with no matching toleration, or a PersistentVolumeClaim that isn't bound. I check kubectl describe nodes for allocated resources, and kubectl get pvc to see whether the claim is bound; a wrong storage class or a zone mismatch is very common. If it's pure capacity, I check whether the cluster autoscaler is enabled and why it hasn't scaled up, for example a node group at its maximum size. The fix follows the cause: right-size requests, add capacity, fix the selector or toleration, or fix the storage class. In production I'd also alert on pods Pending for more than five minutes, and set requests from real usage data so scheduling stays predictable.

Likely follow-ups

- The pod fits on paper but still stays Pending. What else could it be?
- How does the cluster autoscaler decide to add a node?

🗺 Pending triage



Pending means the scheduler cannot place the pod on any node. The root cause is almost always resource constraints, node selectors, taints, or missing PVCs.

DEBUG COMMANDS

```
kubectl describe pod <pod> -n <ns>  # check Events section — scheduler message is here
```

```
kubectl get nodes -o wide           # confirm nodes are Ready
```

```
kubectl describe nodes | grep -A5 "Allocated resources"  # check capacity
```

```
kubectl get events --sort-by='.lastTimestamp' -n <ns>
```

certs.eknathalabs.com

Common causes: insufficient CPU/memory on all nodes (scale node pool or reduce requests), node selector / affinity rules not matching any node, taint with no matching toleration, PVC stuck in Pending (storage class issue), or image pull waiting on volume mount.

Interview tip: Mention the scheduler's filter + score pipeline — Pending means the filter phase eliminated all nodes. The Events section tells you which predicate failed.

📚 **Official docs**

- Kubernetes: Debug Pods

Q2 **Pod is in CrashLoopBackOff — how do you diagnose and fix it?** Pods & workloads

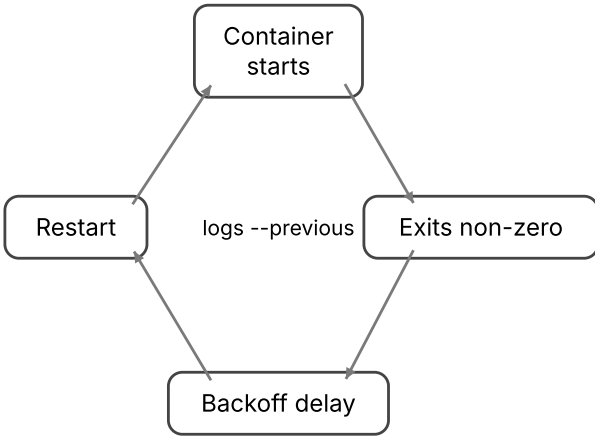
🎙 **Spoken answer · ~69 s**

CrashLoopBackOff means the container starts, exits, and the kubelet keeps restarting it with an exponential backoff that caps at five minutes. First I find out why it exited: kubectl describe pod shows the last state, exit code, and reason, and kubectl logs with the previous flag shows the crashed container's output, since the current one may be empty. Exit code 1 usually means an application error or bad config, 137 means it was killed, often OOMKilled if the reason says so, and 139 is a segfault. Common causes are missing environment variables or secrets, a wrong command or entrypoint, an unreachable dependency at startup, and a liveness probe killing a slow-starting app. If it dies too fast to inspect, I override the command with sleep, or use kubectl debug with an ephemeral container. The fix is usually config, a resource limit, or a startup probe so liveness doesn't fire early. I'd also check recent deploys, because rolling back is the fastest fix while I investigate.

Likely follow-ups

- The logs are empty. What now?
- How do you tell an OOM kill from an application crash?

🗺 **CrashLoopBackOff**



CrashLoopBackOff means the container starts, crashes, Kubernetes restarts it, and it crashes again — with exponential backoff delay (10s → 20s → 40s → max 5min).

certs.eknathalabs.com

```
kubectl logs <pod> --previous          # logs from the crashed container
```

```
kubectl logs <pod> -c <container> -f   # follow live logs
```

```
kubectl describe pod <pod>             # check Exit Code and Last State
```

Exit code meanings: 0 = success (CMD exited cleanly — check your ENTRYPOINT), 1 = app error, 137 = OOMKilled (increase memory limit), 139 = segfault, 143 = SIGTERM (graceful shutdown issue). Fix the application error, check liveness probe config, verify env vars and secrets are mounted correctly.

OOMKilled pattern: If Exit Code is 137 consistently, increase `resources.limits.memory` or find the memory leak. Kubernetes never lies about OOM.

📚 **Official docs**

- Kubernetes: Pod lifecycle & termination
- Kubernetes: Debug Pods

Q3 **Pod is in ImagePullBackOff — how do you resolve it?** Pods & workloads

🎙 **Spoken answer · ~70 s**

ImagePullBackOff means the kubelet tried to pull the image, failed, and is now backing off between retries. I start with kubectl describe pod and read the Events section, because it gives the exact error: manifest not found, unauthorized, or a timeout. Manifest not found usually means a typo in the image name or a tag that doesn't exist, so I verify it by pulling the image from a node or a workstation. Unauthorized points at credentials. For a private registry I check that the imagePullSecret exists in the pod's namespace, that it's referenced in the pod spec or attached to the service account, and that the credentials haven't expired. I can recreate it with kubectl create secret docker-registry. If the error is a timeout, I look at network egress, NAT, proxies, or a network policy blocking the registry. On GKE I'd prefer Workload Identity or the node service account with Artifact Registry reader rights rather than static secrets. Finally I check registry rate limits, especially on Docker Hub.

Likely follow-ups

- Why might the same image pull fine on one node and fail on another?
- How do you avoid Docker Hub rate limits in a cluster?

```
kubectl describe pod <pod>  # check Events for exact pull error
```

```
kubectl get secret <pull-secret> -o jsonpath='{.data..dockerconfigjson}' | base64 -d
```

```
kubectl create secret docker-registry regcred --docker-server=<registry> --docker-username=
<u> --docker-password=<p>
```

Causes: image tag does not exist, wrong registry URL, missing or expired `imagePullSecret`, registry authentication failure, or network policy blocking egress to the registry. Verify the image exists with `docker pull` from the node. Check the service account has the pull secret attached.

certs.eknathalabs.com

GKE/GCP context: For GCR/Artifact Registry, use Workload Identity — no secrets needed. The node SA must have `roles/artifactregistry.reader`.

📚 **Official docs**

- Kubernetes: Debug Pods
- Kubernetes: Images (pull policy, registries)
- Kubernetes: Pull an image from a private registry

Q4

A Deployment is not rolling out — pods are stuck in old version. How do you debug?

Pods & workloads

🎙 **Spoken answer · ~70 s**

When a Deployment is stuck on the old version, I first run kubectl rollout status and kubectl describe deployment to see the conditions, then look at the new ReplicaSet's pods. The usual blockers are new pods in CrashLoopBackOff, a readiness probe that never passes, a resource quota or lack of capacity that keeps pods Pending, or a PodDisruptionBudget preventing old pods from being removed. The rollout is governed by maxUnavailable and maxSurge. If maxUnavailable is zero, a new pod must become Ready before an old one is terminated, so any failing new pod freezes the rollout, which is actually the safe behaviour. I read the pod events and logs, fix the cause, and if production is affected I run kubectl rollout undo to go back to the previous revision first and investigate after. I also check progressDeadlineSeconds, since the Deployment will report ProgressDeadlineExceeded after that time. In CI I'd make the pipeline wait on rollout status so a stuck rollout fails the deploy instead of passing silently.

Likely follow-ups

- What do maxSurge and maxUnavailable control?
- How would your pipeline detect and roll back a failed rollout automatically?

```
kubectl rollout status deployment/<name>     # check rollout progress
```

```
kubectl rollout history deployment/<name>    # list revisions
```

```
kubectl describe deployment <name>          # check conditions
```

```
kubectl rollout undo deployment/<name>       # rollback to previous
```

```
kubectl rollout undo deployment/<name> --to-revision=2  # rollback to specific
```

Common blockers: new pods are CrashLoopBackOff so the rolling update stalls (maxUnavailable / maxSurge config), readiness probe failing so new pods never become Ready, resource quota exceeded so new pods can't schedule, or PodDisruptionBudget blocking old pod termination.

Principal-level answer: Explain the `maxUnavailable` and `maxSurge` parameters and how they control rollout speed vs availability tradeoff. A stuck rollout with `maxUnavailable: 0` means the new pod must become Ready before the old one is removed.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Deployments

Q5 **A StatefulSet pod is not coming up — how do you troubleshoot?** Pods & workloads

🎙 **Spoken answer · ~71 s**

For a StatefulSet pod that won't start, the first thing to remember is that pods are created in order. Pod one will not start until pod zero is Running and Ready, so I look at the lowest-numbered pod that isn't healthy rather than the one that's reported. Then kubectl describe pod and the events. The most common cause is storage: each pod gets its own PersistentVolumeClaim, so I check kubectl get pvc and describe the claim. Is the StorageClass present, is the provisioner running, does the access mode match what the storage supports, and is there quota? With WaitForFirstConsumer, a zone mismatch between the scheduled node and the existing volume can leave a pod stuck, especially after a node moves. If the pod starts but never gets Ready, I check readiness probes and application logs, because the next replica waits on it. One important difference from Deployments: deleting a StatefulSet pod doesn't delete its PVC, which protects data, so if I need a clean slate I delete the PVC deliberately.

Likely follow-ups

- Why can't a StatefulSet pod move to another zone with its volume?
- What does podManagementPolicy Parallel change?

```
kubectl describe pod <sts-pod-0>   # StatefulSet pods start sequentially
```

```
kubectl get pvc -n <ns>            # check PVC binding status
```

```
kubectl describe pvc <pvc-name>    # storage provisioner events
```

StatefulSets have ordered, sequential pod creation — pod-1 won't start until pod-0 is Running and Ready. PVC binding failure is the most common blocker. Check: storage class exists and has a provisioner, the provisioner pod itself is running, correct access modes (ReadWriteOnce vs ReadWriteMany), and sufficient storage quota.

Critical difference from Deployments: Deleting a StatefulSet pod does NOT delete its PVC. If you need to wipe data, delete the PVC explicitly. This is intentional — protect stateful data.

📚 **Official docs**

- Kubernetes: StatefulSets

certs.eknathalabs.com

Q6

How do you handle a pod that is consuming too much CPU or memory in production?

Pods & workloads

🎙 Spoken answer · ~68 s

I start by confirming what's actually happening with kubectl top pod, sorted by CPU or memory, and compare it with the requests and limits in the spec. The two failure modes behave differently. Memory over the limit gets the container OOMKilled and restarted, which is loud. CPU over the limit gets throttled, which is silent: the pod just gets slow, so I check the container CPU throttled seconds metric in Prometheus. Immediate mitigation is to raise the limit or scale out replicas, depending on whether the load is legitimate. Then I find the cause: a leak shows as memory growing steadily, a traffic spike shows as load correlating with request rate, and a bad deploy shows as a step change at release time. Long term I'd set requests based on observed usage, run the Vertical Pod Autoscaler in recommendation mode to right-size them, and set an HPA for load-driven scaling. Requests drive scheduling, limits drive throttling and OOM, so both need to be realistic.

Likely follow-ups

- How do you detect CPU throttling when usage looks low?
- Should you set CPU limits at all?

```
kubectl top pod <pod> -n <ns>      # current resource usage
```

```
kubectl top pod --sort-by=memory   # find top consumers
```

```
kubectl describe pod <pod> | grep -A4 Limits  # check current limits
```

Immediate: if OOMKilling, increase memory limit with `kubectl set resources deployment`. Long term: add Vertical Pod Autoscaler (VPA) in recommendation mode to get right-sized request/limit suggestions. Set resource requests accurately — they affect scheduling; limits affect throttling/OOM.

CPU vs memory: CPU throttling is silent — the pod runs slowly. Memory OOM kills the container. Both cause degraded performance. Use Prometheus metrics (`container_cpu_throttled_seconds_total`) to detect CPU throttling in production.

📚 Official docs

- Kubernetes: Resource management for Pods & containers

certs.eknathalabs.com

Q7 **A Service is not routing traffic to pods — how do you debug?** Networking

🎙 Spoken answer · ~71 s

I walk the path from the Service to the pod. First I look at the endpoints or EndpointSlices for the service. If they're empty, the selector doesn't match the pod labels, or the pods aren't Ready, and that's the classic cause. Second, I check that the targetPort matches the port the container actually listens on, and that the app binds to all interfaces, not just localhost. Third, readiness: a failing readiness probe removes a pod from endpoints on purpose. Then I test from inside the cluster with a debug pod: curl the service DNS name, then curl the pod IP directly. If the pod IP works but the service doesn't, I suspect kube-proxy or the service config. If DNS fails, it's CoreDNS. If neither works, I look at NetworkPolicies and the CNI. For external traffic I check the Ingress or load balancer layer separately. Most of the time it's a label typo or a port mismatch, so I check endpoints first, because it eliminates half the possibilities in thirty seconds.

Likely follow-ups

- Endpoints are populated but requests still fail. Where next?
- What does kube-proxy actually do in iptables or IPVS mode?

🗺 Service routing check




```
kubectl get endpoints <svc> -n <ns>      # if empty — label selector mismatch
```

```
kubectl describe svc <svc>               # check selector labels
```

```
kubectl get pods --show-labels -n <ns>   # verify pod labels match selector
```

```
kubectl exec -it <pod> -- curl <svc>:<port> # test from inside cluster
```

The most common cause is empty Endpoints — the service selector doesn't match any pod labels. Also check: targetPort matches the container port (not the service port), pods are in Ready state (only Ready pods get endpoints), and kube-proxy / CNI is functioning on the node.

Quick test: `kubectl run tmp --image=curlimages/curl -it --rm -- curl http://<svc>.<ns>.svc.cluster.local` to test DNS resolution and connectivity in one command.

📚 Official docs

- Kubernetes: Debug Services

certs.eknathalabs.com

Q8 DNS resolution is failing inside the cluster — how do you troubleshoot?

Networking

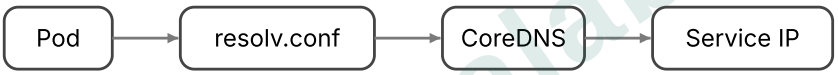
🎙 Spoken answer · ~67 s

I start from inside a pod, because DNS problems are best tested from where the failure is. I exec in and run nslookup for kubernetes.default and for the failing service name, and look at /etc/resolv.conf to make sure it points to the cluster DNS service, usually the ten dot ninety-six dot zero dot ten address. Then I check CoreDNS itself: are the pods running in kube-system, what do their logs say, and is the Corefile in the ConfigMap valid? If DNS works for cluster names but not external ones, the upstream forwarders are suspect. If it works from some pods but not others, I think NetworkPolicy, because a default-deny policy blocks UDP and TCP 53 to CoreDNS unless explicitly allowed. That's a very common trap. Intermittent failures or five-second delays point at the ndots search path generating extra queries, or a conntrack race on UDP. Scaling CoreDNS, enabling NodeLocal DNSCache, and lowering ndots in the pod's dnsConfig are the usual fixes.

Likely follow-ups

- Why do you sometimes see exactly five-second DNS delays in pods?
- What does ndots do?

🗺 In-cluster DNS




```
kubectl exec -it <pod> -- nslookup kubernetes.default  # basic DNS check
```

```
kubectl get pods -n kube-system | grep coredns         # CoreDNS health
```

```
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=50
```

```
kubectl describe cm coredns -n kube-system             # check Corefile config
```

Check CoreDNS pods are Running. Check ConfigMap for Corefile syntax errors. Verify the pod's `/etc/resolv.conf` points to the cluster DNS IP (usually 10.96.0.10). Network policies can block DNS (UDP 53) from pods to CoreDNS — always allow this in custom NetworkPolicies.

Common trap: A NetworkPolicy with `podSelector: {}` (deny all) will break DNS unless you explicitly allow egress to CoreDNS on port 53 UDP/TCP.

📚 Official docs

- Kubernetes: Debugging DNS resolution
- Kubernetes: DNS for Services and Pods
- RFC 1035: DNS implementation and specification

certs.eknathalabs.com

Q9 **Ingress is not routing traffic — how do you debug Ingress issues?** Networking

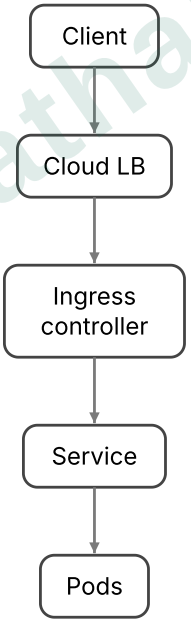
🎙 Spoken answer · ~75 s

I debug Ingress in layers, from the outside in. First, is the Ingress controller running and does it have an external address? Then does the Ingress resource's ingressClassName match that controller, since a mismatch means nobody picks it up. Next, kubectl describe ingress shows the rules and the backend it resolved; I confirm the Service and port exist and that the Service has endpoints, which means pods are Ready. Then I look at the request itself: the host header and path must match a rule, otherwise you get the default backend and a 404. TLS adds another check: is the secret present and does the certificate cover the hostname? The controller logs are the most useful tool, because nginx or Envoy shows exactly which requests arrive and why they fail. A 502 or 503 usually means no healthy backend, and a 504 means the backend is too slow. On GKE the Ingress provisions a cloud load balancer, which can take several minutes, and I check backend health in the console since failing health checks mean readiness or path issues.

Likely follow-ups

- What's the difference between a 502, 503 and 504 from the ingress?
- How does Gateway API improve on Ingress?

🗺 Ingress path




```
kubectl describe ingress <name> -n <ns>           # check rules and backend
```

```
kubectl get pods -n ingress-nginx                  # controller running?
```

```
kubectl logs -n ingress-nginx <controller-pod> -f   # access + error logs
```

```
kubectl get events -n <ns> | grep ingress
```

certs.eknathalabs.com

Troubleshooting layers: (1) Is the Ingress controller running and has an external IP? (2) Does the IngressClass match the controller? (3) Is the backend Service and port correct? (4) Does the host header in the request match the Ingress rule? (5) Is TLS cert valid for the hostname? Check controller logs — nginx/envoy logs show exactly which requests arrive and why they fail.

GKE context: GKE Ingress uses a GCP HTTP(S) Load Balancer. Provisioning takes 5–10 minutes. Check GCP Console → Load Balancers for backend health status — unhealthy backends mean readiness probe failures.

📚 **Official docs**

- Kubernetes: Ingress

Q10 **Network Policy is blocking traffic unexpectedly — how do you debug?** Networking

🎙 **Spoken answer · ~70 s**

When a NetworkPolicy is blocking traffic unexpectedly, I first remember the semantics. Pods are open by default, but as soon as any policy selects a pod for a direction, everything not explicitly allowed in that direction is denied. Allow rules are additive. So I list the policies in the namespace, read which ones select the source and destination pods, and check both sides: ingress rules on the destination and egress rules on the source. I test with a debug pod using nc or curl to the target IP and port, which also tells me if it's DNS or the connection itself. Egress to CoreDNS on port 53 is the classic thing people forget. To confirm a policy is the cause I can temporarily remove or loosen it in a non-production environment and add rules back one at a time. Also, I verify the CNI actually enforces NetworkPolicies, because with plain Flannel they're silently ignored. Cilium and Calico also give flow logs and policy verdicts, which make this far easier.

Likely follow-ups

- How do you allow DNS in a default-deny namespace?
- What tools show you which policy dropped a packet?

```
kubectl get networkpolicy -n <ns> -o yaml   # review all policies in namespace
```

```
kubectl describe networkpolicy <name>        # ingress/egress rules
```

```
# Test connectivity: kubectl exec -it <pod> -- nc -zv <target-ip> <port>
```

NetworkPolicies are additive for allow rules but the default (if any policy selects a pod) is deny-all for unmatched traffic. Debug by temporarily removing NetworkPolicies to confirm they are the cause, then add back one by one. Check both ingress rules on the destination pod AND egress rules on the source pod.

Important: NetworkPolicies are enforced by the CNI plugin (Calico, Cilium, etc.). If your CNI doesn't support them (like Flannel without addons), policies are silently ignored. Verify CNI support first.

📚 **Official docs**

- Kubernetes: Network Policies

certs.eknathalabs.com

Q11 PersistentVolumeClaim is stuck in Pending — how do you fix it?

Storage

🎙 Spoken answer · ~68 s

A PVC stuck in Pending means no volume has been bound to it. I run kubectl describe pvc and read the events, which usually state the reason directly. Common causes: there's no default StorageClass and the claim doesn't name one, the named StorageClass doesn't exist, or its provisioner isn't running, so I check the provisioner or CSI driver pods in kube-system. Another cause is an access mode the storage can't offer, for example ReadWriteMany on block storage that only supports ReadWriteOnce. Quota can also block it, either namespace storage quota or the cloud's disk quota. One important case isn't an error at all: with volumeBindingMode WaitForFirstConsumer the claim stays Pending until a pod using it is scheduled, because the system wants to pick the volume's zone based on where the pod lands. In that case I check why the pod isn't scheduling. Finally, if I'm using static provisioning, I check that a matching PersistentVolume exists with compatible size, access mode, and class.

Likely follow-ups

- What's the difference between Immediate and WaitForFirstConsumer binding?
- What happens to the data when a PVC is deleted?

```
kubectl describe pvc <name> -n <ns>     # provisioner events are here
```

```
kubectl get sc                            # list storage classes
```

```
kubectl describe sc <name>                # check provisioner and parameters
```

```
kubectl get pods -n kube-system | grep provisioner
```

Causes: no default StorageClass defined, StorageClass provisioner pod is not running, access mode mismatch (requesting ReadWriteMany on a block storage that only supports ReadWriteOnce), storage quota exceeded, or the `volumeBindingMode: WaitForFirstConsumer` means PVC binds only when a pod using it is scheduled.

WaitForFirstConsumer: With this binding mode, the PVC stays Pending until a pod claims it. This is correct behavior for topology-aware provisioning (e.g., AWS EBS in a specific AZ).

📚 Official docs

- Kubernetes: Debug Pods
- Kubernetes: Persistent Volumes

certs.eknathalabs.com

Q12 **A pod lost its persistent data after restart — how do you investigate?** Storage

🎙 Spoken answer · ~68 s

If a pod lost its data after a restart, I first work out what kind of storage it was using. Describing the pod shows its volumes. The most common cause is that the application wrote to the container filesystem or to an emptyDir. The container filesystem is lost when the container is recreated, and an emptyDir survives container restarts but is destroyed when the pod is deleted or evicted. If a PVC was used, I check that it's still bound and what the PersistentVolume's reclaim policy is, because with Delete the underlying volume is removed when the claim is deleted. I also confirm the mountPath matches the directory where the application actually writes its data; a mismatch means the app is writing to ephemeral container storage while the volume sits empty next to it. For production stateful workloads I'd use a StatefulSet with PVCs and a Retain reclaim policy, keep tested backups or snapshots, and never rely on emptyDir for anything that must survive.

Likely follow-ups

- What are the PV reclaim policies?
- How do you back up persistent volumes in Kubernetes?

```
kubectl describe pod <pod> | grep -A10 Volumes   # check volume mounts
```

```
kubectl get pvc -n <ns>                           # PVC still bound?
```

```
kubectl describe pv <pv-name>                     # reclaim policy
```

Root causes: pod was using `emptyDir` instead of a PVC (emptyDir is ephemeral — wiped on pod restart), PVC had `reclaimPolicy: Delete` and the pod/PVC was deleted, or the volume was mounted at wrong path. Always use PVCs with `reclaimPolicy: Retain` for production stateful data. Confirm the data directory in the container matches the `mountPath` in the pod spec.

Never use emptyDir for production data. It survives container restarts within the same pod but is destroyed when the pod is evicted or deleted.

📚 Official docs

- Kubernetes: Persistent Volumes

certs.eknathalabs.com

Q13 **How do you secure secrets in Kubernetes — what are the risks of using Kubernetes Secrets natively?**

Security

🎙 Spoken answer · ~66 s

Native Kubernetes Secrets are only base64-encoded, not encrypted, so the real protections are access control and storage. The risks are: anyone with RBAC permission to get or list secrets in a namespace can read them, secrets live in etcd and are readable by anyone with etcd access unless encryption at rest is enabled, and they're easy to leak through environment variables, logs, or Git. My hardening is layered. Turn on etcd encryption at rest, ideally with a KMS provider. Tighten RBAC so only the workloads that need a secret can read it, and audit who can list secrets. Prefer mounting secrets as files over environment variables. Better still, keep the source of truth outside the cluster in a manager such as Vault or the cloud's secret manager, and sync it with External Secrets Operator or the Secrets Store CSI driver, which also gives rotation. And never commit secrets to Git; if GitOps requires it, use sealed or SOPS-encrypted secrets.

Likely follow-ups

- Why are environment variables a weaker way to deliver secrets than files?
- How do you rotate a secret without restarting everything?

🗺 Secret handling



Kubernetes Secrets are base64-encoded (not encrypted) by default. Anyone with RBAC access to `get secrets` can read them. They are stored in etcd — if etcd is compromised without encryption-at-rest, all secrets are exposed.

PRODUCTION HARDENING

```
# Enable etcd encryption at rest (EncryptionConfiguration)
```

```
# Use external secret stores: External Secrets Operator + AWS SSM/GCP Secret Manager
```

```
# Use Vault Agent Injector for sidecar-based secret injection
```

```
kubectl get rolebinding,clusterrolebinding -A | grep secrets  # audit RBAC
```

Best practice: Never store secrets in Git (even encrypted tfvars). Use External Secrets Operator to sync from GCP Secret Manager — secrets live in GCP, K8s Secret is auto-synced and rotated. This is the production standard.

📚 Official docs

- Kubernetes: Secrets
- Kubernetes: Good practices for Secrets

certs.eknathalabs.com

Q14 **A pod is running as root — how do you harden the pod security posture?** Security

🎙 Spoken answer · ~64 s

To harden a pod that runs as root, I set a securityContext. runAsNonRoot true with a specific non-zero user, allowPrivilegeEscalation false, readOnlyRootFilesystem true with writable emptyDir mounts only where needed, and drop all Linux capabilities, adding back only what's required. I also set seccompProfile to RuntimeDefault and avoid privileged containers, hostPath mounts, and host networking. The image has to cooperate, so I build it with a non-root USER and fix file permissions. Then I enforce this cluster-wide rather than trusting each team: Pod Security Admission with the restricted profile enforced on production namespaces, or Kyverno or Gatekeeper policies where I need finer control. To audit what exists already, I query pod specs for their security contexts and run kube-bench for node and cluster checks. For runtime detection I add Falco, which alerts on things like a shell spawned in a container. The baseline profile blocks known escalations, while restricted follows current hardening best practice.

Likely follow-ups

- What are the three Pod Security Standards profiles?
- What breaks when you set readOnlyRootFilesystem?

```
# In pod spec: securityContext
```

```
runAsNonRoot: true
```

```
runAsUser: 1000
```

```
allowPrivilegeEscalation: false
```

```
readOnlyRootFilesystem: true
```

```
capabilities: { drop: ["ALL"] }
```

Use Pod Security Admission (PSA) with `enforce: restricted` label on namespaces in production. Audit existing pods with `kubectl get pods -o json | jq '.items[].spec.containers[].securityContext'`. For CKS exam: know the difference between privileged, baseline, and restricted PSA profiles.

CKS relevance: This is a core CKS domain. Know how to audit with `kube-bench`, configure PSA, and use Falco for runtime security detection.

📚 Official docs

- Kubernetes: Pod Security Standards
- Kubernetes: Security checklist

certs.eknathalabs.com

Q15 **RBAC is misconfigured — a service account has too much access. How do you audit and fix?** Security

🎙 Spoken answer · ~64 s

To audit an over-privileged service account, I start by asking what it can actually do. kubectl auth can-i with the list flag and the as flag for the service account shows its effective permissions. Then I find where they come from by searching RoleBindings and ClusterRoleBindings that reference it, and describing the roles. Tools like rbac-tool or kubectl who-can help answer who can perform a sensitive verb across the cluster. The fix follows least privilege: replace ClusterRoleBindings with namespace-scoped RoleBindings where possible, give each workload its own dedicated service account instead of using default, remove wildcard verbs and resources, and never give cluster-admin to an application. I also disable token auto-mounting for pods that don't call the API. A common source of excess permissions is Helm charts that ship ClusterRoleBindings for convenience, so I review rendered chart RBAC before installing. Then I enforce it by policy and keep reviewing, because permissions drift over time.

Likely follow-ups

- How do you find who can read secrets cluster-wide?
- What is the risk of the default service account?

```
kubectl auth can-i --list --as=system:serviceaccount:<ns>:<sa>
```

```
kubectl get rolebinding,clusterrolebinding -o yaml | grep -A5 <sa-name>
```

```
kubectl describe clusterrolebinding <name>
```

Follow least-privilege principle: avoid ClusterRoleBindings when a RoleBinding scoped to a namespace suffices. Avoid binding to default service accounts. Audit with tools like `rbac-tool` or `kubectl-who-can`. Never use `cluster-admin` for application service accounts.

Common mistake: Helm deployments often create ClusterRoleBindings for convenience. Audit every Helm chart's RBAC manifests before applying to production.

📚 Official docs

- Kubernetes: RBAC
- Kubernetes: RBAC good practices
- Kubernetes: Service accounts

certs.eknathalabs.com

Q16 **How do you detect and prevent privilege escalation in a Kubernetes cluster?** Security

🎙 **Spoken answer · ~63 s**

I treat privilege escalation as something to prevent at several layers, plus detect at runtime. At admission, I enforce Pod Security Admission restricted profile or Kyverno or Gatekeeper policies that reject privileged containers, host namespaces, hostPath mounts, added capabilities, and allowPrivilegeEscalation. At the identity layer, I keep RBAC tight and specifically watch dangerous permissions: the ability to create pods, exec into pods, read secrets, bind roles, or impersonate, since each can be a path to cluster-admin. At the network layer, NetworkPolicies limit lateral movement if a pod is compromised. At runtime, Falco watches syscalls and flags things like a shell in a container, writes under etc, or a privileged container starting. And I enable API server audit logging, because without it there's no forensic trail after an incident; it's often missed on self-managed clusters. Finally, I keep nodes and the runtime patched, since container escapes rely on kernel and runtime vulnerabilities.

Likely follow-ups

- Which RBAC permissions are effectively equivalent to cluster-admin?
- What would Falco alert on that admission control can't catch?

```
# Runtime detection: Falco with default ruleset
```

```
falco -r /etc/falco/falco_rules.yaml  # detects privileged container exec, etc.
```

```
# Admission control: OPA/Gatekeeper or Kyverno policies
```

```
kubectl get constrainttemplate  # OPA Gatekeeper policies
```

Prevention layers: PSA restricted profile blocks privileged containers at admission, NetworkPolicies limit lateral movement, Falco detects anomalous runtime behavior (unexpected exec, file writes in /etc), and audit logging captures API server activity. For CKS: know how to write Falco rules and Kyverno ClusterPolicies.

Audit logging: Enable API server audit policy. Without it, you cannot do forensics after a breach. This is often missed in self-managed clusters.

📚 **Official docs**

- Kubernetes: RBAC good practices

certs.eknathalabs.com

Q17 Pods are not being scheduled on specific nodes — how do you control pod placement?

Scheduling

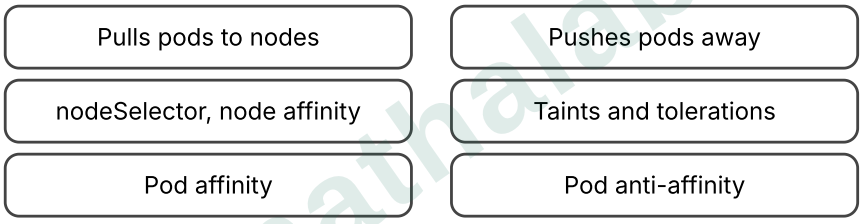
🎙 Spoken answer · ~67 s

Kubernetes gives several placement tools, and I pick the simplest that does the job. nodeSelector matches node labels and is good for basic cases. Node affinity adds richer rules, with required rules that must be met and preferred rules that are best effort. Pod affinity and anti-affinity place pods relative to other pods, for example to co-locate a cache with an app or to keep replicas apart. Taints and tolerations work from the node side: a taint repels pods unless they tolerate it, which is how I reserve GPU or dedicated nodes. For high availability, I prefer topologySpreadConstraints to spread replicas evenly across zones and hosts, which is more predictable than anti-affinity at scale. If pods aren't landing where expected, I describe the pod and read the scheduler event, which lists exactly why each node was rejected. Remember that hard constraints can make pods unschedulable, so I use preferred rules when it's a nice-to-have, and always combine placement with PodDisruptionBudgets.

Likely follow-ups

- When would you use topology spread constraints over anti-affinity?
- How do taints differ from node affinity?

🗺 Placement controls




```
kubectl label node <node> disktype=ssd          # add node label
```

```
kubectl taint node <node> key=value:NoSchedule  # add taint
```

Placement mechanisms (in order of preference for production use): (1) `nodeSelector` — simple label match, (2) Node Affinity — richer rules with preferred vs required, (3) Pod Affinity/AntiAffinity — place pods relative to other pods, (4) Taints + Tolerations — node-side repulsion with pod-side override. For high availability: use `topologySpreadConstraints` to spread pods across zones evenly.

Production pattern: Use AntiAffinity with `topologyKey: kubernetes.io/hostname` to ensure Deployment replicas land on different nodes. Use zone-level spread for HA across AZs.

📚 Official docs

- Kubernetes: Assigning Pods to Nodes

certs.eknathalabs.com

Q18 **Nodes are under resource pressure — how do you handle node eviction and resource management?** Scheduling

🎙 Spoken answer · ~65 s

When a node comes under resource pressure, the kubelet starts evicting pods, and the order depends on QoS class. BestEffort pods, with no requests or limits, go first, then Burstable pods that exceed their requests, and Guaranteed pods, where requests equal limits, are last. So the key preventive step is setting accurate requests on everything, and setting requests equal to limits for critical workloads, especially memory. To investigate, I describe the node and read its conditions for MemoryPressure, DiskPressure, or PIDPressure, check top nodes, and list recently evicted pods. Disk pressure is often image layers or container logs, so I look at image garbage collection and log rotation. For voluntary disruptions like drains and upgrades, PodDisruptionBudgets limit how many replicas go down at once; they don't protect against involuntary eviction. I also set kube-reserved and system-reserved so the kubelet and OS have guaranteed headroom, and use the cluster autoscaler or Karpenter so capacity grows before pressure hits.

Likely follow-ups

- How does the kubelet choose which pod to evict within a QoS class?
- What's the difference between a PodDisruptionBudget and eviction thresholds?

```
kubectl describe node <node> | grep -A10 Conditions   # MemoryPressure, DiskPressure
```

```
kubectl get pods --field-selector=status.phase=Failed -A  # recently evicted pods
```

```
kubectl top nodes  # resource utilization
```

Kubelet evicts pods under memory or disk pressure using QoS classes: BestEffort (no requests/limits) evicted first, then Burstable, then Guaranteed (requests = limits). Set accurate resource requests to avoid BestEffort classification for production pods. Use PodDisruptionBudgets to limit voluntary evictions during node drain/maintenance.

Best practice: Set `requests = limits` for memory on critical pods (Guaranteed QoS). Never leave requests unset in production — the scheduler and eviction manager both rely on them.

📚 Official docs

- Kubernetes: Node-pressure eviction

certs.eknathalabs.com

Q19 **HPA (Horizontal Pod Autoscaler) is not scaling — how do you debug?** Scheduling

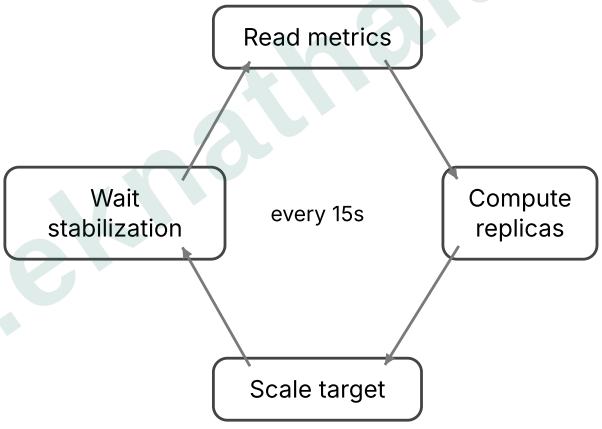
🎙 Spoken answer · ~68 s

When an HPA isn't scaling, I begin with kubectl describe hpa, which shows current metrics, targets, and conditions such as ScalingActive or AbleToScale, and often states the problem directly. Common causes: metrics-server isn't installed or healthy, so I check the metrics API service and run kubectl top pods; the pods have no resource requests, which makes utilization percentage impossible to compute; the target deployment's selector or reference is wrong; or the HPA is already at maxReplicas. Timing matters too: scale-down has a stabilization window of five minutes by default, and scale-up can be limited by behavior policies. If the pod can't actually be scheduled, new replicas stay Pending, which looks like scaling failed, and then the cluster autoscaler is the thing to check. For custom or external metrics, I verify the Prometheus Adapter or KEDA is working and the metric is registered. In production, I scale on a metric that reflects load, such as queue depth or request rate, rather than only CPU.

Likely follow-ups

- Why does HPA need resource requests?
- When would you choose KEDA over the standard HPA?

🗺 HPA loop




```
kubectl describe hpa <name> -n <ns>   # check current metrics and conditions
```

```
kubectl get apiservice v1beta1.metrics.k8s.io   # metrics-server available?
```

```
kubectl top pods -n <ns>               # verify metrics are flowing
```

HPA requires metrics-server to be running. Common failures: metrics-server not installed, pod resource requests not set (HPA computes utilization % of requests — without requests it cannot calculate), scale-down cooldown period active (default 5min), or minReplicas == maxReplicas. For custom metrics HPA, verify Prometheus Adapter is running and the metric is registered as an APIService.

Production tip: Always set both CPU and memory HPA targets. CPU alone misses memory-bound workloads. For GKE, use custom metrics from Cloud Monitoring for queue-depth-based scaling.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Horizontal Pod Autoscaling

Q20 **Node is NotReady — how do you diagnose and recover?** Scheduling

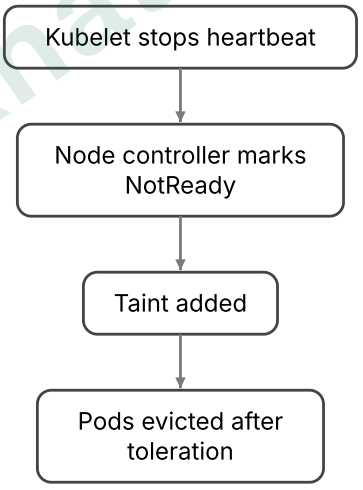
🎙 **Spoken answer · ~68 s**

NotReady means the kubelet isn't reporting healthy to the API server, or the node reports a failing condition. I start with kubectl describe node and read Conditions and Events: it tells me whether it's MemoryPressure, DiskPressure, PIDPressure, or that the kubelet simply stopped posting status. Then I get onto the node if I can: kubelet status and its journal, the container runtime, disk space and inodes, memory, and connectivity to the API server. Frequent causes are a crashed or hung kubelet, a full disk, the container runtime failing, expired certificates, or the VM or its networking being down. Meanwhile the node controller taints the node, and after the default five minutes of toleration, pods are evicted and rescheduled; StatefulSet pods are handled carefully to avoid two copies running. If I can't fix it quickly, I cordon it, drain what I can, and replace the node, treating nodes as cattle, then root-cause from the logs. I'd also alert on NotReady nodes and kubelet health.

Likely follow-ups

- Why do StatefulSet pods not move immediately?
- How would you automate node replacement?

🗺 **Node NotReady**




```
kubectl describe node <node>            # check Conditions and Events
```

```
kubectl get pods -A --field-selector spec.nodeName=<node>  # pods on the node
```

```
# SSH to node:
```

```
systemctl status kubelet                 # kubelet running?
```

```
journalctl -u kubelet -n 100             # kubelet logs
```

certs.eknathalabs.com

```
systemctl status containerd             # container runtime
```

NotReady causes: kubelet stopped/crashed, container runtime (containerd/docker) failed, network plugin (CNI) crashed, node OOM at OS level, disk full, or certificate expired. Fix: restart kubelet and containerd. For a fully unrecoverable node, drain it (`kubectl drain --ignore-daemonsets --delete-emptydir-data`) and replace it.

Node drain before delete: Always drain before deleting/replacing a node to gracefully evict pods and respect PodDisruptionBudgets.

📚 **Official docs**

- Kubernetes: Troubleshooting clusters

Q21 **How do you debug a pod that is Running but application is not responding?** Observability

🎙 **Spoken answer · ~72 s**

A pod in Running state only tells you the main process is alive, not that the application is working. I go through it systematically. First the logs, with follow and tail, to look for panics, repeated errors, or a stuck loop. Then check readiness: if the readiness probe is failing, the pod is removed from the Service endpoints and gets no traffic, which looks like the app not responding. Then I exec into the container and curl the app locally on its port, which separates an app problem from a network problem. If it works locally, the issue is the Service, selector, port, or network policy. If it hangs locally, the app may be deadlocked, out of threads or connections, or blocked on a dependency, so I check connections to the database and cache, resource usage, and thread dumps. I also verify the environment variables and ConfigMaps are what I expect. And I make sure liveness and readiness probes test the right thing, since a bad liveness probe can restart a healthy app.

Likely follow-ups

- How do you tell whether the problem is the pod or the Service in front of it?
- What should a good readiness probe check?

```
kubectl logs <pod> -f --tail=100        # live application logs
```

```
kubectl exec -it <pod> -- sh            # exec into container
```

```
kubectl exec -it <pod> -- curl localhost:<port>/health
```

```
kubectl describe pod <pod> | grep -A5 Readiness  # readiness probe status
```

Running state only means the container process is alive — the application inside may be deadlocked, misconfigured, or healthy-checked incorrectly. Check: readiness probe endpoint is responding (if not, pod is excluded from Service endpoints), application logs for panics/errors, env vars/configmaps mounted correctly, and network connectivity to dependencies (DB, cache).

Readiness vs Liveness: Readiness probe failure removes pod from Service endpoints (traffic stops). Liveness probe failure restarts the container. Use both strategically — don't make liveness probe too aggressive or you'll cause unnecessary restarts under load.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Liveness, readiness and startup probes

Q22 **How do you set up effective observability for a Kubernetes cluster?** Observability

🎙 **Spoken answer · ~66 s**

I build Kubernetes observability around the three pillars plus alerting. For metrics, kube-prometheus-stack gives Prometheus, Grafana, and the standard exporters like kube-state-metrics and node-exporter. For logs, a DaemonSet such as Fluent Bit ships container logs to Loki or Elasticsearch. For traces, applications emit OpenTelemetry to a collector and then to Tempo or Jaeger. Alertmanager routes alerts to paging. What matters more than the tools is what I alert on. I page on symptoms users feel, such as error rate and latency against SLOs, and use cause-level signals as dashboards or lower-urgency alerts: pod restart rates, pods stuck Pending, HPA stuck at max, PVC usage, node pressure, API server and etcd latency, and CoreDNS errors. Dashboards: a cluster overview, namespace resource usage, and per-service golden signals of latency, traffic, errors, and saturation. On a managed platform like GKE, Cloud Monitoring and managed Prometheus remove much of the operational burden. I also watch metric cardinality and log volume to control cost.

Likely follow-ups

- What would you page on versus only put on a dashboard?
- How do you control the cost of metrics and logs?

Production observability has three pillars: metrics, logs, and traces.

```
# Metrics: Prometheus + Grafana (kube-prometheus-stack Helm chart)
```

```
# Logs: Fluent Bit (DaemonSet) → Elasticsearch/Loki → Grafana/Kibana
```

```
# Traces: OpenTelemetry Collector → Jaeger/Tempo
```

```
# Alerting: Prometheus AlertManager with PagerDuty/OpsGenie
```

Key metrics to alert on: pod restart rate, HPA at max replicas, PVC usage %, node CPU/memory %, etcd latency, API server request latency, and coredns error rate. Set up dashboards for: cluster overview, namespace resource usage, and per-deployment golden signals (latency, traffic, errors, saturation).

GKE context: GKE Autopilot includes managed Prometheus. For a GCP-focused role, Google Cloud Monitoring + Cloud Logging reduces operational overhead significantly vs self-managed Prometheus stack.

📚 **Official docs**

- Prometheus: Overview

certs.eknathalabs.com

Q23 Kubernetes API server is slow or unresponsive — how do you troubleshoot?

Observability

🎙 Spoken answer · ~72 s

For a slow API server I look at its own metrics first: request duration by verb and resource, plus in-flight requests and priority and fairness queueing. That tells me whether the problem is broad or one hot resource. Then I look at the usual causes. etcd is the foundation, so I check etcd latency, disk fsync duration, and database size; slow disks are a classic cause. Next, expensive client behaviour: controllers or tools doing large unpaginated LIST calls or restarting watches create storms, and I can identify the culprit from audit logs and metrics by user agent. Then admission webhooks, because every request passes through them; a slow or unavailable webhook with a long timeout and failurePolicy Fail slows or blocks everything, so I check webhook latency metrics and set short timeouts and appropriate failure policies. I also check control plane node CPU and memory, and the object counts, like huge numbers of events or old Jobs. On managed services I can't touch etcd, so I focus on client behaviour and webhooks.

Likely follow-ups

- How can an admission webhook take down a cluster?
- What is API Priority and Fairness?

```
kubectl get --raw /metrics | grep apiserver_request_duration
```

```
kubectl get --raw /healthz                    # API server health
```

```
# Check etcd: etcdctl endpoint health
```

```
kubectl top nodes                             # control plane node pressure
```

Causes: etcd performance degradation (high disk latency — use SSDs for etcd), too many LIST operations (watch storms from controllers), control plane node under memory/CPU pressure, too many objects in etcd (clean up unused resources), or webhook latency (admission webhooks adding round-trip time to every API call).

Webhook timeout: Misconfigured admission webhooks with long timeouts can make every API call slow. Set `failurePolicy: Ignore` and short timeouts for non-critical webhooks. Check webhook latency with `apiserver_admission_webhook_admission_duration_seconds`.

📚 Official docs

- Kubernetes: Troubleshooting clusters
- Kubernetes: API Priority and Fairness

certs.eknathalabs.com

Q24 **How do you perform a zero-downtime Kubernetes cluster upgrade?** Cluster ops

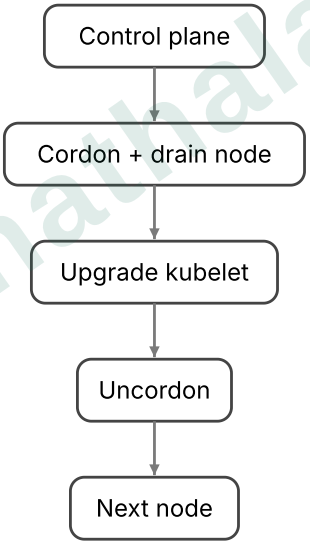
🎙 Spoken answer · ~68 s

I upgrade one minor version at a time, control plane first, then nodes, respecting the version skew policy. Before starting I read the release notes, scan for deprecated or removed APIs with a tool like pluto or kubent, back up etcd, and confirm workloads have multiple replicas, PodDisruptionBudgets, and proper readiness probes. Then I upgrade the control plane; on managed services that's one operation, on kubeadm it's one control plane node at a time. For nodes I prefer a surge approach: add new nodes on the new version, cordon and drain old ones respecting PDBs, and verify each batch before continuing. For critical clusters I might build a parallel cluster and shift traffic instead. Zero downtime really depends on the workloads, not the cluster: a single replica or a missing PDB means drain causes downtime. I also upgrade add-ons like the CNI, CoreDNS, and ingress controller for compatibility, keep a rollback plan, and rehearse the whole thing in a lower environment first.

Likely follow-ups

- A drain hangs forever. What is usually the cause?
- How do you find deprecated APIs before upgrading?

🗺 Zero-downtime upgrade




```
# 1. Upgrade control plane first (one minor version at a time)
```

```
kubectl drain <control-plane-node> --ignore-daemonsets
```

```
# 2. Upgrade kubelet and kubectl on control plane node
```

```
# 3. Upgrade worker nodes one by one:
```

```
kubectl drain <worker-node> --ignore-daemonsets --delete-emptydir-data
```

```
kubectl uncordon <worker-node>   # after upgrade
```

certs.eknathalabs.com

Rules: upgrade only one minor version at a time (1.28 → 1.29, not 1.28 → 1.30). Control plane must be upgraded before workers. Ensure PodDisruptionBudgets are set so draining doesn't violate availability. Test in staging first. For GKE: use GKE's managed upgrade with surge upgrade enabled (adds extra nodes during upgrade).

GKE shortcut: Enable auto-upgrade with maintenance windows and surge upgrade. GKE handles the upgrade orchestration — you define the maintenance policy and it does the rest safely.

📚 **Official docs**

- Kubernetes: Upgrading kubeadm clusters

Q25 **etcd is unhealthy — how do you recover a Kubernetes cluster?** Cluster ops

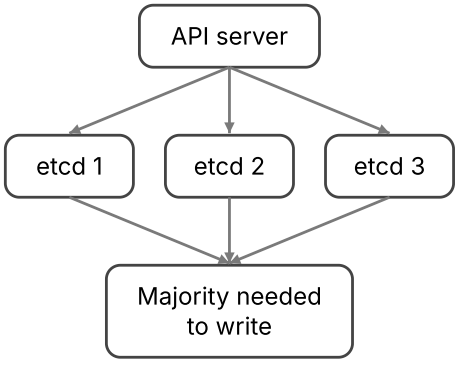
🎙 **Spoken answer · ~71 s**

etcd is the source of truth for the whole cluster, so the first question is whether it still has quorum. With three members you can lose one, with five you can lose two. If quorum is intact, I check member list and endpoint health, find the unhealthy member, and replace it by removing and re-adding it, so it syncs from the healthy ones. If quorum is lost, the cluster can't take writes. Then recovery means restoring from a snapshot: stop the API servers and etcd, restore the snapshot into a new data directory on each member with the new cluster configuration, start etcd, then bring up the API server and verify the nodes and workloads. The workloads already running keep running during this, since the kubelets and containers don't need etcd, but nothing can change. The prerequisite is automated, regular snapshots stored off the node in durable storage, and tested restores, because an untested backup is a hope, not a plan. On managed Kubernetes, the provider handles all of this.

Likely follow-ups

- What happens to running pods while etcd is down?
- How often would you snapshot etcd and how would you test the restore?

🗺 **etcd quorum**




```
etcdctl endpoint health --endpoints=<ip>:2379
```

```
etcdctl member list   # check cluster membership
```

```
# Restore from snapshot:
```

certs.eknathalabs.com

```
etcdctl snapshot restore /backup/etcd.db --data-dir=/var/lib/etcd-restore
```

etcd is the source of truth for the entire cluster. If the majority of etcd members are down, the cluster is read-only (no writes). Recovery steps: restore from a snapshot backup, reconfigure the etcd data directory, restart the etcd service, then restart kube-apiserver. Always take regular etcd snapshots in production (automated, stored in durable object storage).

etcd quorum: A 3-node etcd cluster tolerates 1 failure. A 5-node cluster tolerates 2. Never run single-node etcd in production. For GKE and managed services, etcd is managed for you — this question is more relevant for self-managed clusters (CKA/CKS exam domain).

📚 **Official docs**

- Kubernetes: Operating etcd clusters
- etcd: Disaster recovery
- etcd documentation

Q26 **How do you drain a node safely without causing downtime?** Cluster ops

🎙 **Spoken answer · ~69 s**

To drain a node without downtime I treat it as a controlled sequence. First I check that the cluster can absorb the work: other nodes need enough free capacity, and the workloads need more than one replica. Then I check PodDisruptionBudgets, because drain uses the Eviction API and will wait or hang if a PDB doesn't allow another pod to go, for example a single replica with minAvailable at one hundred percent. I cordon the node so nothing new schedules there, then run kubectl drain with ignore-daemonsets, and delete-emptydir-data if pods use emptyDir, and a sensible grace period so applications can shut down cleanly. Pods get evicted and rescheduled elsewhere, and I watch that they become Ready before moving on. After maintenance I uncordon the node. For production I'd do this during low traffic, drain one node at a time, and make sure applications handle SIGTERM and have preStop hooks so in-flight requests finish. Node pool upgrades on managed clusters follow the same logic automatically.

Likely follow-ups

- What do you do when a drain hangs?
- Why might a pod ignore SIGTERM and delay a drain?

```
kubectl cordon <node>    # prevent new pods from scheduling
```

```
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data --grace-period=60
```

```
kubectl uncordon <node>  # re-enable scheduling after maintenance
```

Before draining: verify PodDisruptionBudgets allow eviction (if PDB has `minAvailable: 100%` and only 1 replica, drain will hang). Ensure other nodes have capacity to absorb the evicted pods. DaemonSet pods are ignored by default (`--ignore-daemonsets`). Pods using `emptyDir` require `--delete-emptydir-data` flag to proceed.

certs.eknathalabs.com

Production checklist: Check HPA max replicas, PDB definitions, and node capacity before draining. Schedule drains during low-traffic windows for safety.

📚 **Official docs**

- Kubernetes: Safely drain a node

Q27 **Certificate expired in the cluster — how do you renew Kubernetes certificates?** Cluster ops

🎙 **Spoken answer · ~67 s**

On kubeadm clusters, control plane certificates expire after one year by default, and when they do the API server rejects connections: kubectl fails with TLS errors, and components can't talk to each other. I check the status with kubeadm certs check-expiration, which lists every certificate and its expiry. To renew, I run kubeadm certs renew all on each control plane node, then restart the static pod components, or move their manifests out and back, so they pick up the new certificates, and restart the kubelet. I also update the admin kubeconfig, because that certificate is embedded in the file. In an HA cluster I repeat this on every control plane node. Better than fixing an outage is preventing it: upgrading with kubeadm renews certificates automatically, I can alert on the x509 certificate expiry metric well ahead of time, and kubelet client certificate rotation should be enabled. On managed services like GKE or EKS, the provider rotates control plane certificates for you.

Likely follow-ups

- Which certificates are not renewed automatically by kubeadm?
- How would you monitor certificate expiry across the cluster?

```
kubeadm certs check-expiration          # list all cert expiry dates
```

```
kubeadm certs renew all                 # renew all control plane certs
```

```
systemctl restart kubelet               # restart to pick up new certs
```

```
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -noout -dates
```

kubeadm-managed clusters: certs expire after 1 year by default. Rotate before expiry using `kubeadm certs renew`. After renewal, restart all control plane components. Set up a monitoring alert for cert expiry (Prometheus `x509_cert_expiry` metric). For managed clusters (GKE/EKS), certificate rotation is handled automatically.

Expired cert symptoms: kubectl commands fail with TLS errors, pods can't pull images, webhooks fail. This is a full cluster outage scenario — certificate monitoring is non-negotiable.

📚 **Official docs**

- Kubernetes: Certificate management with kubeadm

certs.eknathalabs.com

Q28 **How do you implement GitOps for Kubernetes using ArgoCD?** CI/CD & GitOps

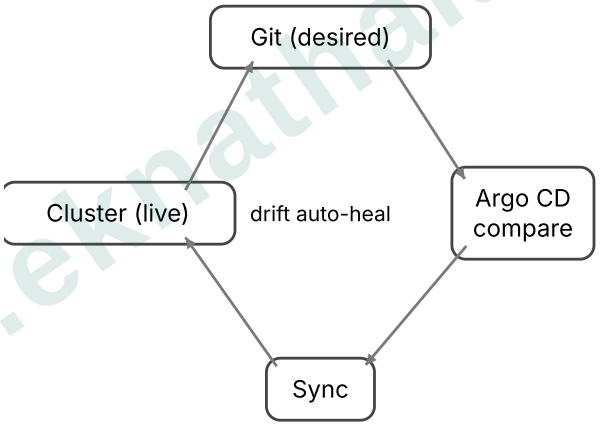
🎙 Spoken answer · ~69 s

GitOps means Git is the source of truth for cluster state, and a controller continuously reconciles the cluster toward it. With Argo CD, I create an Application that points to a repo and path and a destination cluster and namespace. Argo polls the repo, or receives a webhook for instant updates, renders the manifests from plain YAML, Helm, or Kustomize, compares them with live state, and reports whether the app is Synced and Healthy. With automated sync, prune, and self-heal enabled, changes merged to Git are applied automatically, resources removed from Git are deleted, and manual drift in the cluster is reverted. For many clusters or environments, I use ApplicationSets to generate Applications from a template. The practical benefits are an audit trail, easy rollback through git revert, and no cluster credentials in CI, since CI only builds images and updates a manifest. Details that matter: separating app code from config repos, handling secrets with sealed secrets or External Secrets, and using sync waves for ordering.

Likely follow-ups

- How do you handle secrets in a GitOps workflow?
- What does self-heal do and when is it dangerous?

🗺 GitOps reconcile



GitOps principle: Git is the single source of truth for cluster state. ArgoCD continuously reconciles cluster state with the Git repository.

```
argocd app create myapp --repo <git-url> --path k8s/ --dest-server <cluster>
```

```
argocd app sync myapp          # manual sync
```

```
argocd app get myapp           # health and sync status
```

```
argocd app diff myapp          # what would change on sync
```

ArgoCD polls the repo (default 3min) or uses Git webhooks for instant sync. Set sync policy to `automated` with `prune: true` and `selfHeal: true` for fully automated GitOps. Use ApplicationSets for multi-cluster or multi-environment deployments from a single template.

certs.eknathalabs.com

Interview differentiator: Mention the App-of-Apps pattern for bootstrapping cluster infrastructure. ArgoCD manages ArgoCD itself — the entire cluster config is in Git.

📚 **Official docs**

- OpenGitOps principles
- Argo CD documentation

Q29 **How do you implement a blue-green or canary deployment in Kubernetes?** CI/CD & GitOps

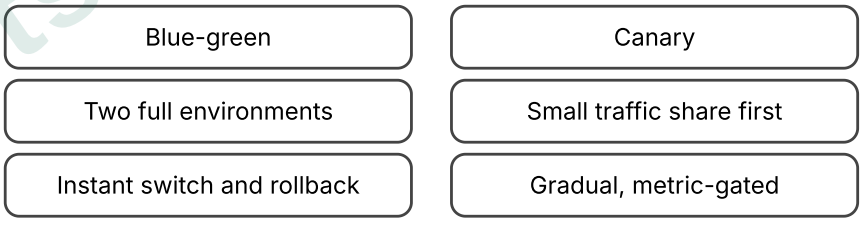
🎙 **Spoken answer · ~69 s**

Blue-green and canary are both ways to reduce deployment risk, with different trade-offs. In blue-green, I run the new version fully alongside the current one, test it, and then switch all traffic at once, in Kubernetes by changing the Service selector or the ingress target. Rollback is instant, just switch back, but it needs double the capacity during the switch. A canary sends a small percentage of real traffic to the new version, watches metrics, and increases gradually. With plain Kubernetes I can approximate it with two Deployments behind one Service, where replica counts set the ratio, but that's crude. For production I use Argo Rollouts or Flagger, which provide weighted steps through an ingress controller or service mesh and, importantly, automated analysis: they query Prometheus for error rate and latency and promote or roll back automatically. I choose canary when I want real-traffic validation with limited blast radius, and blue-green when I need fast, all-or-nothing cutover. Database changes must be backward compatible either way.

Likely follow-ups

- How do database migrations fit into blue-green or canary?
- What metrics would gate a canary promotion?

🗺 **Blue-green vs canary**



Blue-green: two full deployments (blue=current, green=new). Switch traffic at the Service level. Canary: route a percentage of traffic to new version.

```
# Blue-green: patch service selector to switch
```

```
kubectl patch svc myapp -p '{"spec":{"selector":{"version":"green"}}}'
```

```
# Canary with Argo Rollouts:
```

```
kubectl argo rollouts get rollout myapp --watch
```

certs.eknathalabs.com

```
kubectl argo rollouts promote myapp   # advance canary steps
```

Native Kubernetes: use multiple Deployments with weighted Services or Ingress traffic splitting. Production-grade: use Argo Rollouts or Flagger with analysis templates — they automate canary promotion/rollback based on Prometheus metrics (error rate, latency). This is far more sophisticated than manual Deployment switching.

Principal-level answer: Describe analysis-driven progressive delivery — canary receives 10% traffic, if P99 latency stays under threshold for 10min, promote to 25%, then 50%, then 100%. Argo Rollouts automates this entire workflow.

📚 **Official docs**

- Google SRE Workbook: Canarying Releases
- Argo Rollouts
- Martin Fowler: BlueGreenDeployment

Q30 **CI/CD pipeline is failing to deploy to Kubernetes — how do you troubleshoot?** CI/CD & GitOps

🎙 **Spoken answer · ~69 s**

When a pipeline fails to deploy to Kubernetes, I split the problem into authentication, authorisation, artefact, and rollout. First, is the pipeline pointed at the right cluster and namespace? I print the current kubectl context. Then credentials: an expired token or kubeconfig is a frequent cause, so I check how the pipeline authenticates, ideally with short-lived OIDC federation instead of static secrets. Next RBAC: kubectl auth can-i with the pipeline's service account shows whether it may create or patch deployments and secrets. Then the artefact: has the image actually been pushed before apply runs, is the tag the one in the manifest, and can the cluster pull it? Race conditions between build and deploy are common. Then manifest problems: invalid YAML, wrong Helm values for the environment, or an admission webhook or policy rejecting the object. Finally, rollout: the pipeline should wait with kubectl rollout status and a timeout so a deployment that applies but never becomes healthy fails the job and can trigger rollback.

Likely follow-ups

- How do you avoid long-lived cluster credentials in CI?
- What should the pipeline do when rollout status times out?

```
# Check: kubectl is configured with correct kubeconfig
```

```
kubectl config current-context   # verify target cluster
```

```
kubectl auth can-i create deployment --as=system:serviceaccount:cicd:deployer
```

```
# Check: image was pushed to registry before apply
```

```
kubectl rollout status deployment/<name> --timeout=120s
```

certs.eknathalabs.com

Common pipeline failures: kubeconfig not configured or expired token, CI service account lacks RBAC permissions (create/update deployments, secrets), image not yet pushed when apply runs (race condition), wrong namespace or context, or Helm values file referencing wrong environment. Always check `kubectl auth can-i` with the pipeline's service account identity.

GKE pipeline pattern: Use Workload Identity Federation to authenticate GitHub Actions to GCP — no service account key files needed. OIDC token exchange is the production standard.

📚 **Official docs**

- Kubernetes: Service accounts

Q31

How do you handle pod disruption during node maintenance to maintain availability?

Pods & workloads

🎙 **Spoken answer · ~67 s**

To keep an application available during node maintenance, I use PodDisruptionBudgets together with sensible replica counts and spreading. A PDB declares either minAvailable or maxUnavailable for a set of pods, and the Eviction API honours it during voluntary disruptions like kubectl drain, node pool upgrades, and cluster autoscaler scale-down. So with six replicas and maxUnavailable of one, a drain evicts pods one at a time and waits for replacements to become Ready. It has limits: PDBs don't protect against involuntary events like a node crash or an OOM kill. And there's a trap, if minAvailable equals the number of replicas, or there's only one replica, then no pod can ever be evicted and the drain hangs. To make it actually work, I run at least two or three replicas, spread them across nodes and zones with topology spread constraints, make sure readiness probes are accurate so the budget counts only truly ready pods, and handle SIGTERM gracefully so connections drain.

Likely follow-ups

- What happens when a PDB blocks a node upgrade?
- What is the difference between voluntary and involuntary disruptions?

```
# Create PodDisruptionBudget
```

```
kubectl apply -f - <<EOF
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: myapp-pdb }
spec:
  minAvailable: 2
  selector: { matchLabels: { app: myapp } }
EOF
```

```
kubectl get pdb -n <ns>   # check current disruptions allowed
```

PDBs guarantee that at least N pods (or at most X% are unavailable) during voluntary disruptions (drain, eviction). Set `minAvailable: "50%"` or `maxUnavailable: 1`. PDBs only apply to voluntary disruptions — a node crash is involuntary and PDBs don't protect against it. Combine PDBs with proper replica counts and anti-affinity for full HA.

certs.eknathalabs.com

PDB trap: A PDB with `minAvailable` equal to current replicas will block all drains. Always set minAvailable to less than total replicas.

📚 Official docs

- Kubernetes: Disruptions & PodDisruptionBudget

Q32 **How do you manage multi-tenancy in a Kubernetes cluster?** Cluster ops

🎙 Spoken answer · ~68 s

Multi-tenancy means several teams share a cluster safely, and I build it in layers. Each team or environment gets a namespace. On top of that, a ResourceQuota caps total CPU, memory, and object counts per namespace so one tenant can't starve others, and a LimitRange sets default requests and limits so pods don't run unbounded. RBAC scopes each team to its own namespaces. NetworkPolicies, starting with default deny, isolate traffic between tenants. Pod Security Admission or policy engines enforce a security baseline per namespace. Those layers give soft multi-tenancy, which is fine for teams that trust each other. For untrusted tenants I need harder isolation: dedicated node pools using taints and tolerations, sandboxed runtimes such as gVisor or Kata, or separate clusters or virtual clusters. I also add cost attribution through labels, self-service namespace creation with guardrails, and per-tenant observability. The trade-off is always efficiency versus blast radius: more sharing is cheaper, but a bad neighbour or an escape affects more people.

Likely follow-ups

- When would you choose separate clusters over namespaces?
- What does a ResourceQuota do that a LimitRange doesn't?

Multi-tenancy means multiple teams/applications share a cluster safely with isolation and fair resource usage.

```
# Namespace per team/environment with ResourceQuota:
```

```
kubectl create quota team-quota --hard=cpu=10,memory=20Gi,pods=50 -n team-a
```

```
# LimitRange for default resource requests:
```

```
kubectl apply -f limitrange.yaml -n team-a
```

Isolation layers: Namespaces (logical), ResourceQuota (resource limits per namespace), LimitRange (per-pod defaults and limits), NetworkPolicies (network isolation), RBAC (API access isolation), PSA (security posture enforcement per namespace). For stronger isolation between untrusted tenants, use separate node pools with taints/tolerations per tenant, or separate clusters.

Principal recommendation: Soft multi-tenancy (namespace isolation) for trusted teams. Hard multi-tenancy (separate clusters or node pools) for compliance-sensitive or untrusted workloads.

📚 Official docs

- Kubernetes: Multi-tenancy

certs.eknathalabs.com

Q33 **How do you troubleshoot high latency in a microservices application running on Kubernetes?**

Observability

🎙 Spoken answer · ~70 s

For high latency in a microservice system I work from the user-facing symptom down. First I establish where the time goes. With distributed tracing in Jaeger or Tempo I find which hop in the call chain is slow, and the golden signals per service tell me if it's latency alone or also errors and saturation. Once I have the slow service, I check its resources: CPU throttling is a classic hidden cause, visible in the throttled seconds metric even when CPU usage looks modest, along with memory pressure and garbage collection. Then the platform: DNS lookup latency, where the ndots setting multiplies queries; connection pool exhaustion; network policy or service mesh sidecar overhead; and noisy neighbours on the node. Then dependencies: the database, cache, or an external API, checking their latency and queueing. I look at percentiles, not averages, because tail latency hides in p99. And I correlate with recent deploys or traffic changes. Once the cause is found, I fix it and add an alert on that signal.

Likely follow-ups

- How would you find CPU throttling when usage looks low?
- What does a p99 spike with a flat p50 suggest?

```
kubectl top pods -n <ns> --sort-by=cpu   # CPU-saturated pods
```

```
# Check Prometheus golden signals per service:
```

```
# http_request_duration_seconds (latency), http_requests_total (traffic), error rate
```

```
kubectl exec -it <pod> -- curl -s /metrics | grep duration
```

Systematic approach: (1) Identify which service in the call chain is slow using distributed traces (Jaeger/Tempo), (2) Check if the slow pod is CPU-throttled (increase limits or HPA), (3) Check DNS resolution latency (ndots:5 config causes multiple DNS lookups), (4) Check network policy overhead, (5) Check external dependency (DB, cache) latency. Use service mesh (Istio/Linkerd) for automatic tracing and traffic metrics.

DNS latency fix: Set `dnsConfig.options: [{name: ndots, value: "1"}]` to reduce spurious DNS lookups. This alone can reduce P99 latency by hundreds of milliseconds in chatty microservices.

📚 Start here (official docs)

- Kubernetes documentation

certs.eknathalabs.com

Q34 **How do you implement image security scanning in a Kubernetes cluster?** Security

🎙 Spoken answer · ~71 s

I secure images in layers across the lifecycle rather than relying on one scan. In CI, I scan every image with a tool like Trivy and fail the build on critical vulnerabilities, and I also scan the Dockerfile and dependencies. I use minimal base images, such as distroless, to reduce the attack surface, and I pin images by digest rather than a mutable tag so what I tested is what runs. I sign images and generate an SBOM. In the registry, I enable continuous scanning, because new CVEs appear after an image is built, so re-scanning deployed images matters as much as the initial scan. At admission, I use Kyverno or Gatekeeper to allow only images from approved registries, with valid signatures, and optionally with a clean scan result. At runtime, Falco detects suspicious behaviour like a shell in a container or unexpected network activity. And I make sure there's a process for patching: rebuilding base images regularly and rolling out fixes, since scanning without remediation just creates a list.

Likely follow-ups

- Why pin images by digest instead of tag?
- How do you handle a critical CVE with no available fix?

```
# CI: trivy image <image>:<tag>  # scan before push
```

```
# Admission: OPA/Gatekeeper or Kyverno to block unscanned images
```

```
# Runtime: Falco to detect anomalous container behavior
```

```
# Registry: GCR/Artifact Registry with Container Analysis API
```

Defense in layers: (1) Scan in CI pipeline — block PRs with critical CVEs, (2) Admission webhook — only allow images from approved registries with scan results, (3) Use digest pinning (`image: repo/app@sha256:abc...`) not mutable tags in production, (4) Enable runtime scanning with Falco, (5) Regularly re-scan deployed images for newly discovered CVEs.

GCP Artifact Registry: Enables automatic vulnerability scanning with Container Analysis. Combine with Binary Authorization to enforce only attested images run in production — a cryptographic proof of scan completion.

📚 Official docs

- Trivy

certs.eknathalabs.com

Q35 **How do you design a production-grade Kubernetes platform for multiple teams — what does a Principal Platform Engineer own here?**

CI/CD & GitOps

🎙 Spoken answer · ~71 s

I'd frame this as building a product for internal developers, not just running clusters. At the infrastructure layer, clusters are provisioned with Terraform and configured through GitOps with Argo CD, so everything is reproducible and auditable; I plan for multiple clusters or regions, with a defined upgrade cadence. The platform then offers golden paths: opinionated templates and Helm or Kustomize bases that include sensible defaults for probes, resources, autoscaling, and security, with self-service namespace creation and CI/CD integration. Guardrails come from policy as code, using Kyverno or Gatekeeper, plus quotas, network policies by default, and RBAC tied to the identity provider. Reliability: SLOs for the platform itself, observability built in so every team gets dashboards and alerts for free, and capacity planning with autoscaling. Governance and cost: labels for attribution, showback, and security scanning. As the owner, I'd measure success by developer outcomes such as time to first deploy and deployment frequency, run it like a product with documentation and a feedback loop, and avoid becoming a bottleneck by favouring self-service.

Likely follow-ups

- How do you stop the platform team becoming a bottleneck?
- How do you measure whether the platform is successful?

This is the Principal-level systems design question. The answer spans infrastructure, developer experience, and organizational patterns.

```
# Infrastructure: IaC (Terraform) for cluster provisioning, GitOps (ArgoCD) for config
```

```
# Developer experience: internal developer platform, self-service namespace creation
```

```
# Reliability: multi-region clusters, cross-cluster traffic routing
```

```
# Governance: OPA/Kyverno policies, admission webhooks, cost attribution
```

A Principal Platform Engineer owns: cluster lifecycle (provisioning, upgrades, capacity planning), golden paths (opinionated templates teams use to deploy safely), reliability SLOs for the platform itself, security posture enforcement, observability standards, and developer productivity metrics. The platform should make the right thing the easy thing — teams deploy to production without thinking about Kubernetes internals.

Interview context: Frame your answer around mentoring — a Principal defines the standards other engineers follow. Show you think in terms of platforms and APIs, not just individual cluster operations. That's what separates a Staff/Principal from a Senior engineer.

📚 Official docs

- CNCF: Platforms white paper

certs.eknathalabs.com

Q36 **Explain init containers vs sidecar containers. When do you use each in production?** Pods & workloads

🎙 Spoken answer · ~69 s

Both add containers to a pod, but their lifecycles differ. Init containers run sequentially to completion before the application containers start. If one fails, the pod restarts it, and the app doesn't start. They're right for one-time setup: waiting for a dependency to be reachable, running database migrations, fetching configuration or secrets, setting kernel parameters, or cloning content into a shared volume. Sidecars run alongside the main container for the whole life of the pod and extend or support it: log shippers, service-mesh proxies such as Envoy, metrics exporters, or credential refreshers. They share the pod's network namespace and volumes, so they communicate over localhost or shared files. A historical problem was sidecar ordering and Job completion, since a sidecar kept the pod from finishing; newer Kubernetes versions support native sidecars, which are init containers with restartPolicy Always, so they start before the app and stop after it. I'd use an init container when something must finish first, and a sidecar when something must keep running.

Likely follow-ups

- How do native sidecars solve startup and shutdown ordering?
- Why not run database migrations in an init container for every replica?

🗺 Init then main containers



Both add containers to a pod beyond the main app, but they differ in lifecycle. Init containers run to completion **before** app containers start, sequentially; sidecars run **alongside** the main container for its whole life.

INIT CONTAINER — RUNS ONCE, BEFORE APP

```
initContainers:
  - name: wait-for-db
    image: busybox
    command: ['sh','-c','until nc -z db 5432; do sleep 2; done']
```

Use init containers for: waiting on a dependency, running DB migrations, fetching config/secrets, setting kernel params, cloning a git repo into a shared volume.

SIDECAR — RUNS FOR THE POD'S LIFETIME

Use sidecars for: log shippers, service-mesh proxies (Envoy), metrics exporters, config reloaders. Since K8s 1.28+ native sidecars are `initContainers` with `restartPolicy: Always` — they start before the app and shut down after it, fixing the old job-sidecar termination problem.

Interview tip: The native sidecar (1.29 GA) solved two long-standing pain points: sidecars now start before the main container (so the proxy is ready first) and don't block Job completion. Mention this — it shows you track recent K8s.

📚 Official docs

- Kubernetes: Init containers
- Kubernetes: Sidecar containers

certs.eknathalabs.com

Q37 **What is the difference between resources.requests and resources.limits? What happens at each boundary?**

Pods & workloads

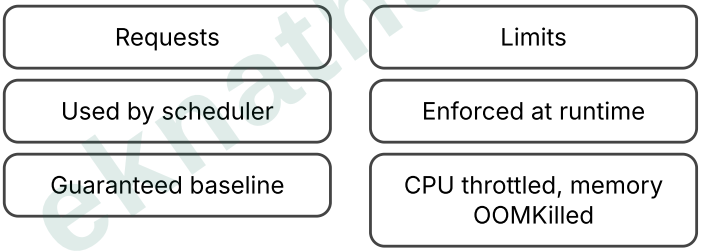
🎙 Spoken answer · ~69 s

Requests are what the scheduler uses to place the pod, because it reserves that amount on a node, and they also set the guaranteed share under contention. Limits are the ceiling enforced at runtime, and they behave differently by resource. CPU is compressible: exceed the limit and the container is throttled through the CFS quota, which hurts latency but doesn't kill it. Memory is not compressible: exceed the limit and the container is OOMKilled. Requests and limits also determine the QoS class: Guaranteed when requests equal limits for every container, Burstable when some are set, and BestEffort when none are, and under node pressure BestEffort pods are evicted first, then Burstable. In practice I set memory requests equal to limits to avoid surprise kills, set CPU requests from observed usage like p90 or p95, and often leave CPU limits loose or off to avoid needless throttling, while LimitRanges and quotas keep namespaces sane. I use VPA in recommendation mode to tune the numbers from real data.

Likely follow-ups

- Why can CPU throttling happen when usage looks low?
- Would you set CPU limits in production? Why or why not?

🗺 Requests vs limits



Requests are what the scheduler reserves; limits are the hard ceiling the runtime enforces. They drive scheduling, QoS class, and eviction order.

```
resources:
  requests: { cpu: "250m", memory: "256Mi" }   # scheduler reserves this
  limits:   { cpu: "500m", memory: "512Mi" }   # hard cap
```

WHAT HAPPENS AT THE BOUNDARY

CPU is compressible: exceeding the CPU limit **throttles** the container (CFS quota), it is not killed. Memory is incompressible: exceeding the memory limit gets the container **OOMKilled** (exit 137). Requests affect which node it lands on and the QoS class.

QOS CLASSES (EVICTION PRIORITY)

```
Guaranteed  : requests == limits for all resources  (evicted last)
Burstable   : requests < limits                       (evicted middle)
BestEffort  : no requests/limits set                   (evicted first)
```

certs.eknathalabs.com

Production gotcha: Setting a CPU limit can cause latency spikes from CFS throttling even when the node has spare CPU. Many teams set CPU *requests* but omit CPU *limits* for latency-sensitive workloads, while always setting memory limits to prevent node OOM.

📚 **Official docs**

- Kubernetes: Resource management for Pods & containers

Q38 **Walk through the full pod lifecycle and termination sequence. Why do graceful shutdowns fail?**

Pods & workloads

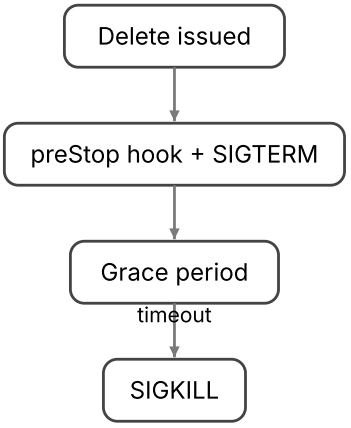
🎙 **Spoken answer · ~65 s**

When a pod is deleted, it's marked Terminating and removed from Service endpoints, while the kubelet runs the preStop hook if there is one, then sends SIGTERM to PID 1 of each container. The pod gets terminationGracePeriodSeconds, thirty by default, to exit, and after that the kubelet sends SIGKILL. Graceful shutdowns fail for a few common reasons. The app ignores SIGTERM, often because it isn't PID 1 and a shell wrapper swallows the signal. Or there's a race: endpoint removal and SIGTERM happen in parallel, so traffic may still arrive at a pod that's shutting down. The fixes are a short preStop sleep of a few seconds, an app that stops accepting new connections and drains in-flight requests on SIGTERM, using exec in the entrypoint or a small init like tini, and a grace period longer than your longest request. I'd pair this with a PodDisruptionBudget so voluntary disruptions don't take too many replicas down at once.

Likely follow-ups

- Why does a preStop sleep help?
- What happens to long-lived connections like websockets during a rollout?

🗺 **Pod termination**



Understanding termination is key to zero-downtime deploys. The sequence is not just "send SIGTERM."

TERMINATION SEQUENCE

certs.eknathalabs.com

```
1. Pod marked Terminating; removed from Endpoints (stops getting new traffic)
2. preStop hook runs (if defined)
3. SIGTERM sent to PID 1 in each container
4. Wait up to terminationGracePeriodSeconds (default 30s)
5. SIGKILL if still running
```

WHY GRACEFUL SHUTDOWN FAILS

The endpoint removal (step 1) and SIGTERM (step 3) happen **in parallel**, not in order — kube-proxy/ingress may still route traffic for a moment after SIGTERM. The app can get requests after it started shutting down.

```
lifecycle:
  preStop:
    exec: { command: ["sh","-c","sleep 5"] }   # let endpoint removal propagate
terminationGracePeriodSeconds: 45
```

Interview tip: The classic fix is a `preStop` sleep of a few seconds so load balancers de-register the pod before SIGTERM, plus an app that handles SIGTERM by draining in-flight requests. Also ensure PID 1 actually receives SIGTERM (use an init like `tini` if your entrypoint is a shell that swallows signals).

📚 **Official docs**

- Kubernetes: Pod lifecycle & termination

Q39

Deployment vs StatefulSet vs DaemonSet vs Job/CronJob — when do you choose each?

Pods & workloads

🎙 **Spoken answer · ~66 s**

I pick the controller by the workload's characteristics. A Deployment is for stateless, interchangeable replicas: web services and APIs, with rolling updates and easy scaling. A StatefulSet is for workloads that need stable identity: each pod gets a predictable name and DNS entry, its own persistent volume through volumeClaimTemplates, and ordered startup and shutdown. That fits databases, Kafka, and ZooKeeper. A DaemonSet runs one pod per node, or per selected node, which suits node-level agents like log collectors, monitoring exporters, CNI plugins, and storage drivers. A Job runs a task to completion, with retries, and a CronJob schedules Jobs on a cron expression, for batch processing, backups, and reports. A few practical notes: for CronJobs I set the concurrency policy and deadlines so overlapping or missed runs behave sensibly, and I don't put a database in a StatefulSet casually. Running stateful systems well takes operators, backups, and failover handling, so often a managed database is the better choice.

Likely follow-ups

- When is a StatefulSet the wrong answer for a database?
- What does a CronJob concurrencyPolicy control?

Picking the right workload controller is a common design question.

THE FOUR

certs.eknathalabs.com

```
Deployment  : stateless, interchangeable replicas; rolling updates
StatefulSet : stable identity + ordered start/stop + per-pod storage
DaemonSet   : one pod per node (agents)
Job/CronJob : run-to-completion / scheduled batch
```

STATEFULSET GUARANTEES

Stable network ID (`pod-0`, `pod-1`...), stable storage (PVC per pod via `volumeClaimTemplates`), ordered rollout and scale-down (reverse order). Use for databases, Kafka, ZooKeeper, anything needing identity or per-replica disks.

DAEMONSET EXAMPLES

Log collectors (Fluent Bit), node monitoring (node-exporter), CNI agents, storage daemons, security agents — anything that must run on every node.

Interview tip: A frequent trap: "can I run a database as a Deployment?" Technically yes with a single replica, but you lose stable identity and ordered operations. For HA stateful systems use a StatefulSet (or better, an operator like CloudNativePG / Strimzi that manages the StatefulSet plus failover logic).

📚 **Official docs**

- Kubernetes: StatefulSets
- Kubernetes: DaemonSet
- Kubernetes: Jobs

Q40 **How do liveness, readiness, and startup probes differ? What's the danger of a bad liveness probe?** Pods & workloads

🎙 **Spoken answer · ~70 s**

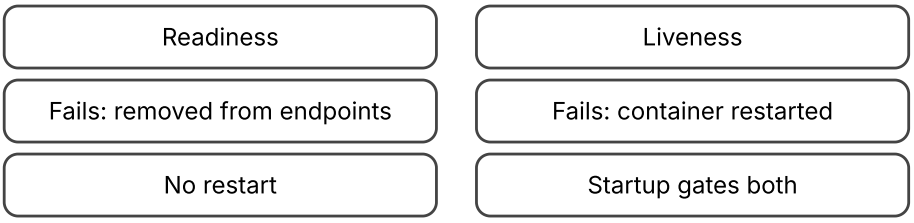
Liveness asks: is this container stuck beyond recovery? If it fails, the kubelet restarts the container. Readiness asks: can this pod take traffic right now? If it fails, the pod is removed from Service endpoints but not restarted. Startup asks: has the app finished booting? It holds off the other two until it passes, which protects slow starters. The danger of a bad liveness probe is that it turns a partial problem into an outage. If the probe checks a downstream dependency like the database, a database blip makes every pod fail liveness, and Kubernetes restarts the whole fleet at once, a self-inflicted cascading failure. Aggressive timeouts and thresholds do the same under load: a busy pod misses a probe, gets killed, and dumps its traffic on the others. So I keep liveness cheap and local, just asking whether the process is responsive. Dependency checks belong in readiness, if anywhere. I use a startup probe for slow apps and tune periods, timeouts, and thresholds from real behavior under load.

Likely follow-ups

- Should readiness check the database? What are the trade-offs?
- What happens if there is no readiness probe at all?

certs.eknathalabs.com

🗺 **Probes**



The three probe types serve different purposes and conflating them causes outages.

```
livenessProbe   : is it alive? FAIL -> restart the container
readinessProbe  : ready for traffic? FAIL -> remove from Service endpoints
startupProbe    : has it finished booting? gates the other two for slow starters
```

THE DANGER OF A BAD LIVENESS PROBE

If a liveness probe is too aggressive (short timeout, low threshold) or depends on a downstream service, a transient blip restarts healthy containers. Under load this creates a **restart storm** — the very thing you didn't want. Liveness should check only "is this process wedged," never "is my database up."

```
readinessProbe:
  httpGet: { path: /healthz/ready, port: 8080 }
  periodSeconds: 5
  failureThreshold: 3
livenessProbe:
  httpGet: { path: /healthz/live, port: 8080 }
  initialDelaySeconds: 15
  failureThreshold: 3
```

Rule: Readiness can depend on dependencies (so you stop serving when DB is down); liveness must NOT — it should only detect a deadlocked process. A liveness probe that pings the DB will restart-loop your whole fleet during a DB incident.

📚 **Official docs**

- Kubernetes: Liveness, readiness and startup probes

certs.eknathalabs.com

Q41 **Explain HPA, VPA, and how they conflict. How does HPA actually compute replica count?** Pods & workloads

🎙 Spoken answer · ~69 s

The Horizontal Pod Autoscaler changes the number of replicas, while the Vertical Pod Autoscaler changes each pod's resource requests and limits. The HPA formula is: desired replicas equals the ceiling of current replicas times current metric divided by target metric. For example, four pods at eighty percent CPU with a fifty percent target gives ceiling of six point four, so seven replicas. It reads metrics from metrics-server for CPU and memory, or from adapters like Prometheus Adapter or KEDA for custom and external metrics, and uses stabilization windows to avoid flapping. Utilization is relative to requests, so requests must be set. The conflict arises when both HPA and VPA act on the same CPU or memory metric: VPA changes requests, which changes the utilization percentage the HPA sees, and they oscillate. Safe combinations are HPA on CPU with VPA on memory, or running VPA in recommendation-only mode and applying its suggestions deliberately, or scaling horizontally on custom metrics like queue depth while VPA handles sizing.

Likely follow-ups

- Why does VPA usually restart pods to apply changes?
- When would you scale on queue depth rather than CPU?

🗺 HPA formula



Horizontal Pod Autoscaler scales replica count; Vertical Pod Autoscaler adjusts requests/limits. They fight if both target CPU/memory.

HPA FORMULA

```
desiredReplicas = ceil( currentReplicas * (currentMetric / targetMetric) )

# e.g. 4 pods at 80% CPU, target 50%:
# ceil(4 * 80/50) = ceil(6.4) = 7 replicas
```

HPA reads metrics from `metrics-server` (resource metrics) or custom/external metrics adapters (Prometheus Adapter, KEDA). It has a stabilization window to avoid flapping and won't scale on missing metrics.

THE HPA/VPA CONFLICT

Both reacting to CPU/memory will oscillate. Safe combos: HPA on CPU + VPA on memory in `recommendation`-only mode, or HPA on custom metrics (RPS, queue depth) + VPA on resources. Never both on the same signal in `Auto` mode.

Interview tip: For event-driven scaling (queue length, Kafka lag, scale-to-zero), name **KEDA** — it extends HPA with external scalers. For node-level scaling mention Cluster Autoscaler or Karpenter. Showing the formula and the HPA/VPA conflict is what separates a real answer.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Horizontal Pod Autoscaling
- Kubernetes Autoscaler: Vertical Pod Autoscaler

Q42 **Explain taints, tolerations, node affinity, and pod affinity/anti-affinity with a real use case.** Scheduling

🎙 **Spoken answer · ~70 s**

These are the scheduling controls, and they work from different directions. A taint sits on a node and repels pods unless the pod has a matching toleration. A typical use is a GPU node pool: I taint the nodes dedicated equals gpu with NoSchedule, and only GPU workloads that tolerate it land there. The effect matters: NoSchedule blocks new pods, PreferNoSchedule is a soft hint, and NoExecute also evicts running pods. Note a toleration only permits scheduling; it doesn't attract, so GPU pods also need node affinity or a selector to choose those nodes. Node affinity is how a pod selects nodes by label, with required or preferred rules. Pod affinity places pods near other pods, for example a web tier close to its cache. Pod anti-affinity does the opposite, spreading replicas of the same app across different nodes or zones using a topologyKey like hostname, which is how I avoid all replicas sharing a failure domain. For even spreading at scale, I prefer topology spread constraints.

Likely follow-ups

- Does a toleration guarantee a pod lands on the tainted node?
- What is the difference between required and preferred affinity?

These control where pods land. Taints repel; affinities attract; anti-affinity spreads.

TAINTS + TOLERATIONS (NODE REPELS UNLESS POD TOLERATES)

```
kubectl taint nodes gpu-node-1 dedicated=gpu:NoSchedule
# pod must tolerate to land there:
tolerations:
- { key: dedicated, operator: Equal, value: gpu, effect: NoSchedule }
```

Effects: `NoSchedule` (don't place), `PreferNoSchedule` (soft), `NoExecute` (evict existing pods too).

NODE AFFINITY (POD SELECTS NODES) + POD ANTI-AFFINITY (SPREAD)

```
affinity:
  podAntiAffinity:          # spread replicas across nodes
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector: { matchLabels: { app: web } }
      topologyKey: kubernetes.io/hostname
```

REAL USE CASE

GPU nodes tainted so only ML workloads (with tolerations + node affinity for `gpu=true`) land there. Web tier uses pod anti-affinity across `topology.kubernetes.io/zone` so a zone failure never takes all replicas.

Interview tip: For simple even spreading prefer `topologySpreadConstraints` over anti-affinity — it scales better and supports `maxSkew`. Anti-affinity (especially `required`) can leave pods Pending if there aren't enough distinct nodes.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Taints and tolerations
- Kubernetes: Assigning Pods to Nodes

Q43 **What are topologySpreadConstraints and why are they preferred over pod anti-affinity?**

Scheduling

🎙 **Spoken answer · ~66 s**

Topology spread constraints tell the scheduler to distribute matching pods evenly across failure domains such as zones or nodes, with a tunable maxSkew, which is the allowed difference in pod count between the most and least loaded domain. I specify the topologyKey, for example the zone label, a label selector for the pods, and whenUnsatisfiable: DoNotSchedule for a hard rule or ScheduleAnyway for a soft one. They're preferred over pod anti-affinity for several reasons. Anti-affinity is binary: a pod either can or can't be co-located, so with more replicas than domains, required anti-affinity leaves pods Pending, whereas spread constraints keep things balanced and still schedule. Anti-affinity is also more expensive to evaluate, because each pod is checked against others, which hurts scheduler performance at scale. Spread constraints express the actual intent, which is an even distribution across availability zones for resilience. I commonly use both: a zone-level constraint with maxSkew one and a node-level one as a soft rule.

Likely follow-ups

- What happens with maxSkew when a zone has no capacity?
- How do spread constraints behave during a rolling update?

Topology spread constraints distribute pods evenly across failure domains (zones, nodes) with a tunable skew, scaling far better than anti-affinity.

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule    # or ScheduleAnyway (soft)
  labelSelector: { matchLabels: { app: web } }
```

certs.eknathalabs.com

WHY PREFERRED OVER ANTI-AFFINITY

Pod anti-affinity is O(N^2) to evaluate (every pod checked against every other) and is binary — a pod either can or can't co-locate. Spread constraints are evaluated more efficiently and let you say "keep zones within 1 pod of each other" with `maxSkew`, giving even distribution instead of just "not together."

WHENUNSATISFIABLE

`DoNotSchedule` = hard (pod stays Pending if it would violate skew); `ScheduleAnyway` = best-effort. Combine zone-level and node-level constraints for resilient placement.

Gotcha: With `DoNotSchedule` and a cluster that lacks enough zones/nodes, pods go Pending. Use `ScheduleAnyway` for soft spreading, or ensure your node pools actually span the topology keys you reference.

📚 **Official docs**

- Kubernetes: Assigning Pods to Nodes

Q44 **How does the kube-scheduler decide where to place a pod? Explain the filter and score phases.** Scheduling

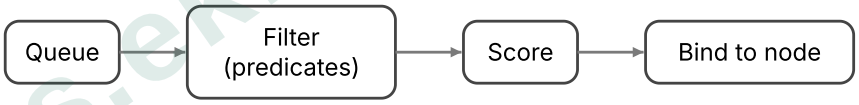
🎙 **Spoken answer · ~69 s**

The kube-scheduler handles each pending pod in two phases: filtering and scoring. In filtering, it eliminates nodes that can't run the pod. It checks whether the node has enough allocatable CPU and memory for the pod's requests, whether node selectors and affinity match, whether the pod tolerates the node's taints, whether the node is cordoned, and whether the required volumes can bind there, along with port conflicts. If no node passes, the pod stays Pending and the event lists why each node was rejected. In scoring, the remaining nodes are ranked by plugins: balanced resource allocation, image locality so nodes that already have the image score higher, inter-pod affinity and topology spread, and a strategy like least-allocated, which spreads load, or most-allocated, which bin-packs for cost. The highest score wins, with ties broken randomly. Then the scheduler binds the pod to the node. Importantly, scheduling uses requests, not actual usage, so inaccurate requests lead to poor placement. If needed, preemption can evict lower priority pods.

Likely follow-ups

- Why does the scheduler look at requests rather than real usage?
- How do you make the scheduler bin-pack instead of spread?

🗺 **Scheduler pipeline**



The scheduler runs a two-phase pipeline per pending pod: filter (which nodes *can* run it) then score (which is *best*).

FILTER (PREDICATES) — ELIMINATE INFEASIBLE NODES

```
- NodeResourcesFit   : enough allocatable CPU/mem for requests?
- NodeAffinity       : node selector / affinity match?
- TaintToleration    : tolerates the node's taints?
- NodeUnschedulable  : node cordoned?
- VolumeBinding      : can required PVCs bind here?
```

SCORE (PRIORITIES) — RANK FEASIBLE NODES

```
- NodeResourcesBalancedAllocation : even CPU/mem usage
- ImageLocality                   : node already has the image
- InterPodAffinity / TopologySpread
- LeastAllocated vs MostAllocated  : spread vs bin-pack
```

Highest-scoring node wins; the pod is bound (a write to the API setting `nodeName`), then kubelet on that node starts it.

certs.eknathalabs.com

Interview tip: Pending almost always means the *filter* phase rejected every node — `kubectl describe pod` Events shows which predicate failed (insufficient cpu, untolerated taint, no matching affinity, unbound PVC). You can customize scoring via the scheduler framework or run multiple scheduler profiles.

📚 **Official docs**

- Kubernetes: kube-scheduler

Q45 **Explain PodDisruptionBudgets. How do they interact with node drains and upgrades?** Scheduling

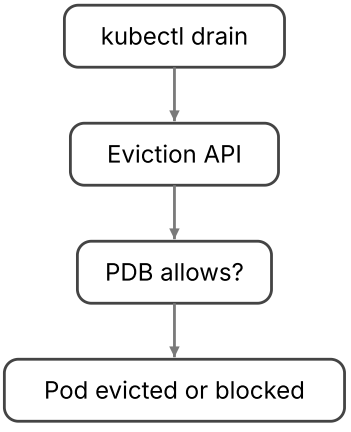
🎙 **Spoken answer · ~69 s**

A PodDisruptionBudget limits how many pods of an application can be unavailable during voluntary disruptions. You define either minAvailable or maxUnavailable, as a number or percentage, plus a selector. It applies to actions that go through the Eviction API: kubectl drain, node pool upgrades, and cluster autoscaler scale-down. It does not cover involuntary events such as a node crash, hardware failure, or an OOM kill. During a drain, if evicting a pod would violate the budget, the eviction is refused and drain retries until the budget allows it, which keeps the service above its minimum. The classic failure is a budget that can never be satisfied: one replica with minAvailable one, or minAvailable equal to replicas, which blocks drains and upgrades indefinitely. So I align the budget with the replica count, for instance maxUnavailable one for most services, and make sure readiness probes are accurate, since the budget counts ready pods. For upgrades I monitor for blocked evictions and alert on PDBs with zero allowed disruptions.

Likely follow-ups

- How do you diagnose a node upgrade stuck on a PDB?
- Do PDBs apply to deleting a pod directly?

🗺 **PDB during drain**



A PodDisruptionBudget limits how many pods of an app can be voluntarily disrupted at once, protecting availability during drains, upgrades, and autoscaling.

certs.eknathalabs.com

```
apiVersion: policy/v1
kind: PodDisruptionBudget
spec:
  minAvailable: 2        # or maxUnavailable: 1
  selector: { matchLabels: { app: web } }
```

VOLUNTARY VS INVOLUNTARY

PDBs only govern **voluntary** disruptions: `kubectl drain`, cluster autoscaler scale-down, node pool upgrades. They do **not** protect against involuntary events (node crash, OOM, kernel panic).

INTERACTION WITH DRAINS

`kubectl drain` uses the Eviction API, which respects PDBs. If evicting a pod would violate `minAvailable`, the eviction is blocked and the drain waits. A misconfigured PDB (e.g. `minAvailable` equal to replica count) makes drains hang forever.

Production gotcha: `minAvailable: 100%` or a PDB on a single-replica Deployment deadlocks node maintenance — the node can never be safely drained. Always leave headroom (e.g. `maxUnavailable: 1`) and run >=2 replicas for anything with a PDB.

📚 **Official docs**

- Kubernetes: Safely drain a node

Q46 **What is a ConfigMap vs Secret, and how do you safely roll config changes without downtime?** Pods & workloads

🎙 **Spoken answer · ~72 s**

ConfigMaps hold non-sensitive configuration, and Secrets hold sensitive values; both can be injected as environment variables or mounted as files. Secrets are only base64-encoded by default, so encryption at rest and tight RBAC matter. The important operational difference is how updates propagate. Environment variables are read once when the container starts, so changing the ConfigMap does nothing until the pod restarts. Mounted files are eventually updated in place by the kubelet, but the application has to re-read them, and subPath mounts never update. For safe rollouts without downtime, I make config changes trigger a normal rolling update: include a checksum of the config as an annotation in the pod template, which Helm can do with a sha256 function, or use a tool like Reloader, or run kubectl rollout restart. That way new pods start with the new config, readiness probes gate traffic, and old pods are removed gradually, and rollback is simple. Another approach is immutable ConfigMaps with versioned names, so each change is a new object and the Deployment references it explicitly.

Likely follow-ups

- Why doesn't a subPath mounted ConfigMap update?
- What are the advantages of immutable ConfigMaps?

ConfigMaps hold non-sensitive config; Secrets hold sensitive data (base64-encoded, not encrypted by default). Both can be mounted as volumes or injected as env vars.

ENV VAR VS VOLUME MOUNT — A KEY DIFFERENCE

Env vars are read **once at container start** — changing the ConfigMap does NOT update them; you must restart the pod. Volume-mounted ConfigMaps/Secrets are **updated in place** (eventually, via kubelet sync) but the app must re-read the file.

certs.eknathalabs.com

ROLLING CONFIG SAFELY

```
# Trigger a rollout when config changes: hash the config into an annotation
kubectl rollout restart deployment/web        # simplest manual trigger
# or use a checksum annotation in the pod template so the hash change rolls pods
```

Security note: Secrets are only base64-encoded in etcd by default. Enable **encryption at rest** (EncryptionConfiguration with a KMS provider) and restrict RBAC on secrets. Better: pull from an external store (Vault, cloud Secret Manager) via CSI Secrets Store or External Secrets Operator so secrets never live in etcd/git.

Interview tip: Mention the `immutable: true` field on ConfigMaps/Secrets — it improves performance (kubelet stops watching) and prevents accidental edits; you roll a new named object instead.

📚 **Official docs**

- Kubernetes: ConfigMaps

Q47 **How do you debug a pod that's running but the application inside is misbehaving (no shell in image)?** Pods & workloads

🎙 **Spoken answer · ~72 s**

Minimal images such as distroless or scratch have no shell, so kubectl exec can't open one. The modern answer is ephemeral debug containers. With kubectl debug I attach a temporary container, for example busybox or netshoot, to the running pod, and with the target flag it shares the process namespace of the application container, so I can see its processes, read its proc entries, and use the pod's network namespace to test connections, run tcpdump, or curl localhost. That doesn't restart or change the original container. If I need to experiment more deeply, kubectl debug can copy the pod into a new debug pod with a shell, optionally with the image swapped. For node-level issues, kubectl debug node starts a privileged pod with the host filesystem mounted. Alongside that, there are the basics: logs with the previous flag for crashed instances, port-forward to hit the app directly, kubectl cp to extract files, and crictl or nsenter on the node. Ephemeral containers need to be permitted by RBAC, which is also a security consideration.

Likely follow-ups

- Why are ephemeral containers restricted by RBAC?
- How would you capture network traffic from a pod with no tcpdump?

Distroless/scratch images have no shell, so `kubectl exec` fails. Use ephemeral debug containers.

KUBECTL DEBUG — EPHEMERAL CONTAINER

```
kubectl debug -it <pod> --image=busybox --target=<container> -- sh
# shares the target container's process namespace — see its PIDs, /proc, network
```

DEBUG A NODE

```
kubectl debug node/<node> -it --image=ubuntu   # host PID/net/IPC, /host mount
```

COPY A POD WITH A SHELL FOR DEEPER WORK

certs.eknathalabs.com

```
kubectl debug <pod> -it --copy-to=debug-pod --image=nicolaka/netshoot --share-processes
```

Other tools: `kubectl logs --previous` for crashed instances, `kubectl cp` to pull files, `kubectl port-forward` to hit the app locally, and `nsenter`/`crictl` on the node for runtime-level inspection.

Interview tip: `kubectl debug` with `--target` and process-namespace sharing is the modern answer for distroless images. `netshoot` is the go-to debug image (dig, curl, tcpdump, ss). This shows you operate hardened, shell-less production images.

📚 **Start here (official docs)**

- Kubernetes documentation

Q48 **Explain rolling update strategy parameters (maxSurge, maxUnavailable) and how to do a safe canary.** Pods & workloads

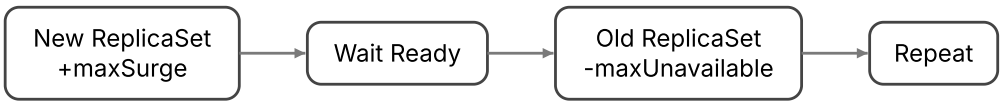
🎙 **Spoken answer · ~72 s**

Two parameters control a rolling update. maxSurge is how many extra pods above the desired count may exist during the rollout, and maxUnavailable is how many pods may be missing. Setting maxUnavailable to zero with maxSurge of one or a small percentage gives the safest, zero-downtime behaviour: a new pod must become Ready before an old one is removed. A larger surge makes the rollout faster but uses more capacity. This depends on correct readiness probes, otherwise Kubernetes considers a pod ready too early, and on graceful shutdown. For a canary, without extra tooling you can run a second small Deployment with the new image behind the same Service selector. With nine stable pods and one canary, roughly ten percent of traffic reaches the new version, based on pod counts. You watch error rates and latency and then promote. That's crude, because the split depends on replica ratios. With Argo Rollouts or Flagger you get explicit traffic weights, pauses, and automated analysis against Prometheus metrics, with automatic rollback if the canary degrades.

Likely follow-ups

- Why does a rolling update need accurate readiness probes?
- How does Argo Rollouts decide to roll back automatically?

🗺 **Rolling update**



Deployment rollout speed and safety are governed by two knobs.

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 25%          # extra pods above desired during rollout
    maxUnavailable: 0      # never drop below desired (safest)
```

certs.eknathalabs.com

`maxUnavailable: 0` + `maxSurge: 1` gives zero-downtime: a new pod must be Ready before an old one is removed. Larger surge = faster but more resource use.

CANARY WITHOUT EXTRA TOOLING

```
# Run a small second Deployment with the new image + same Service selector
# 1 canary pod alongside 9 stable -> ~10% traffic, watch metrics, then promote
```

CANARY WITH TOOLING

Argo Rollouts or Flagger give progressive delivery: automated step weights (10%->50%->100%), metric analysis, and auto-rollback on SLO breach — far safer than manual.

Interview tip: Tie in the earlier termination point — zero-downtime needs `maxUnavailable: 0` AND readiness probes AND preStop drain. All three together. Mention DB migrations must be backward-compatible (expand/contract) since old and new pods run simultaneously mid-rollout.

📚 **Official docs**

- Kubernetes: Deployments
- Google SRE Workbook: Canarying Releases
- Martin Fowler: CanaryRelease

Q49 **What is pod priority and preemption? How can it cause cascading evictions?** Scheduling

🎙 **Spoken answer · ~70 s**

Pod priority lets me tell the scheduler which pods matter more, using PriorityClass objects with a numeric value. If a high-priority pod can't be scheduled because the cluster is full, the scheduler can preempt: it finds a node where evicting lower-priority pods would make room, evicts the minimum necessary, respecting PodDisruptionBudgets where it can, and schedules the important pod. That's useful for protecting critical system components and revenue-critical services over batch jobs. The risk is cascading effects. If many high-priority pods appear at once, they can evict a wave of lower-priority pods, which then go Pending, possibly triggering autoscaler scale-ups, and if they in turn have other priorities they can displace further pods, causing churn. Misuse is the usual cause: if everything is marked high priority, priority loses meaning and preemption thrashes. My approach is a small number of well-defined classes, reserving the highest for system-critical pods, setting a low default for batch workloads, using preemptionPolicy Never where I want ordering but not eviction, and keeping enough headroom.

Likely follow-ups

- What is preemptionPolicy Never used for?
- How would you give batch jobs lower priority than services?

PriorityClasses let higher-priority pods preempt (evict) lower-priority ones when the cluster is full.

certs.eknathalabs.com

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata: { name: high-priority }
value: 1000000
globalDefault: false
```

When a high-priority pod can't schedule, the scheduler finds the lowest-priority pods whose removal would make room and evicts them (respecting PDBs where possible).

CASCADING EVICTION RISK

If many high-priority pods arrive, they can evict a wave of low-priority pods, which then re-queue and may themselves trigger node scale-up or further churn. Misused priorities (everything "high") defeat the purpose and cause thrash.

Gotcha: System-critical components use reserved classes `system-cluster-critical` / `system-node-critical`. Don't assign these to app workloads. Preemption can disrupt lower-priority pods even when they have PDBs if no other placement exists. Keep a sane priority hierarchy and use it sparingly.

Interview tip: Good real use: give batch/CI jobs low priority so they yield to production traffic during spikes, and use Cluster Autoscaler with priority expander so preemption is a bridge until new nodes come online.

📚 **Official docs**

- Kubernetes: Node-pressure eviction
- Kubernetes: Pod priority and preemption

Q50 **A pod was OOMKilled but the app's heap looks fine. What else could consume the memory limit?** Pods & workloads

🎙 **Spoken answer · ~74 s**

The container memory limit is enforced by the cgroup, and it counts far more than the application heap. So when a pod is OOMKilled while the heap looks fine, I look for the other memory consumers. Page cache from heavy file I/O counts toward the cgroup, although it is reclaimable, so it usually matters less. Off-heap and native memory is a big one: in the JVM that means direct buffers, metaspace, JNI libraries, and memory-mapped files. Thread stacks add up when an application creates many threads. Volumes of type emptyDir with medium Memory, and tmpfs mounts, count against the container's limit too. Also, older runtimes that aren't container-aware size their heap from the host's memory rather than the limit. To diagnose, kubectl describe confirms OOMKilled and exit code 137, kubectl top with containers shows live usage, and on the node I read the cgroup memory statistics to see rss, cache, and mapped file. Then I fix it by setting the heap to a fraction of the limit, leaving headroom, and raising the limit based on measured peak.

Likely follow-ups

- How would you size a JVM heap inside a container limit?
- Does page cache count toward the OOM kill decision?

The cgroup memory limit counts **everything** in the container, not just your app's heap. OOMKills with a "healthy" heap usually come from overlooked memory users.

certs.eknathalabs.com

HIDDEN MEMORY CONSUMERS

```
- Page cache from heavy file I/O (counts toward cgroup memory)
- Off-heap / native allocations (JNI, mmap, direct ByteBuffers in JVM)
- tmpfs / emptyDir{medium: Memory} volumes count against the limit
- Thread stacks (many threads x stack size)
- The JVM/runtime itself not being container-aware (old runtimes)
```

DIAGNOSIS

```
kubectl describe pod <pod>            # Last State: OOMKilled, exit 137
kubectl top pod <pod> --containers   # live usage
# on node: cat /sys/fs/cgroup/memory.max  and  memory.current
```

JVM gotcha: Pre-Java 10 JVMs ignored cgroup limits and sized the heap from host RAM -> instant OOM. Use a container-aware JVM and `-XX:MaxRAMPercentage`. For Node.js, set `--max-old-space-size` below the limit.

Interview tip: Mention that `emptyDir` with `medium: Memory` and writable page cache both count against the limit — a surprising number of OOMKills are tmpfs or log files, not the heap.

📚 **Official docs**

- Kubernetes: Debug Pods
- Kubernetes: Determine the reason for Pod failure
- Kubernetes: Resource management for Pods & containers

Q51 **How does the Cluster Autoscaler work, and how is Karpenter different?** Scheduling

🎙 **Spoken answer · ~68 s**

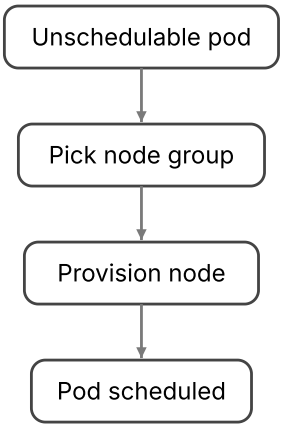
The Cluster Autoscaler adds nodes when pods are Pending because no node has room, and removes nodes that are underused and whose pods can move elsewhere. It works with predefined node groups, such as AWS Auto Scaling groups or GCP managed instance groups, so it scales by increasing the size of a group of identical machines. Scale-down respects PodDisruptionBudgets, local storage, and annotations. Karpenter takes a different approach. It has no node groups: it watches pending pods, calculates what they need in aggregate, and launches the right instance types directly, choosing from a wide range, including spot. That makes it faster, since nodes come up in seconds without waiting on group abstractions, and better at bin-packing and cost optimisation. It also does consolidation, actively repacking workloads onto fewer or cheaper nodes, and handles node expiry for patching. The trade-offs are that Karpenter is most mature on AWS, and you need to define constraints in NodePools carefully. For both, accurate pod requests are essential.

Likely follow-ups

- Why might the autoscaler not scale up even though pods are Pending?
- How does Karpenter consolidation save cost?

certs.eknathalabs.com

🗺 **Cluster Autoscaler**



Cluster Autoscaler (CA) adds/removes **nodes** when pods can't schedule or nodes are underused. Karpenter is a newer, more flexible alternative.

CLUSTER AUTOSCALER

Watches for Pending pods that fail only due to insufficient resources -> scales up a node group (ASG/MIG). Scales down nodes that are underutilized and whose pods can move elsewhere (respecting PDBs, local storage, etc.). Works on fixed, pre-defined node groups.

KARPENTER — GROUP-LESS, JUST-IN-TIME

```
- No node groups: picks instance types per pending pods' actual needs
- Faster: provisions nodes in seconds, bin-packs across instance types
- Consolidation: actively repacks workloads onto fewer/cheaper nodes
- Native spot handling and instance diversification
```

Scale-down blockers: CA won't remove a node running a pod without a controller, a pod with local storage, a pod that can't be moved due to PDB, or kube-system pods lacking a PDB. A single such pod can pin an expensive node up indefinitely.

Interview tip: Karpenter's consolidation + per-pod instance selection usually cuts cost vs CA's fixed groups. Mention the scale-down blockers — they're the #1 reason "autoscaling isn't saving money."

📚 **Official docs**

- Kubernetes Autoscaler: Cluster Autoscaler
- Karpenter documentation

certs.eknathalabs.com

Q52 **Explain how a Service selects pods and what happens when none are Ready. How do headless Services differ?**

Pods & workloads

🎙 Spoken answer · ~71 s

A Service is a stable virtual IP and DNS name in front of a changing set of pods, selected by a label selector. The link between them is the EndpointSlice, which lists the IPs of matching pods that are Ready. Only ready pods are included, which is why readiness probes matter: a pod that fails readiness is removed from the Service's endpoints, though the Service itself remains. If no pods are ready, the endpoint list is empty and connections fail with refused or time out. So when debugging a Service I check the selector matches the labels, that targetPort matches the container port, and that the EndpointSlice has addresses. A headless Service sets clusterIP to None, so there's no virtual IP and no kube-proxy load balancing. DNS returns the individual pod IPs directly, as multiple A records. That's used by StatefulSets, where each pod needs a stable DNS name, and by clients that do their own load balancing or need to reach specific pods, such as databases and gRPC clients.

Likely follow-ups

- Why would a Service exist but have no endpoints?
- How do StatefulSet pods get individual DNS names?

A Service is a stable virtual IP that load-balances to a dynamic set of pod IPs chosen by a label selector. The glue is the Endpoints/EndpointSlice object.

SELECTOR -> ENDPOINTSLICE

```
kubectl get endpointslices -l kubernetes.io/service-name=<svc>
# only READY pods appear here; not-ready pods are excluded from routing
```

If no pods are Ready, the EndpointSlice is empty and connections fail (connection refused / timeout). This is why readiness probes matter — a failing readiness check pulls the pod from the endpoints, not the Service.

HEADLESS SERVICE (CLUSTERIP: NONE)

```
spec:
  clusterIP: None      # no virtual IP / no proxying
  selector: { app: db }
```

DNS returns the **individual pod IPs** (A records) instead of one VIP. Used with StatefulSets so clients can address specific pods (`db-0.db.ns.svc`) — essential for databases and peer discovery.

Interview tip: Distinguish the Service types: ClusterIP (internal VIP), NodePort (port on every node), LoadBalancer (cloud LB), ExternalName (CNAME), and Headless (no VIP, direct pod DNS). Headless + StatefulSet is the standard pattern for stateful peer-aware systems.

📚 Official docs

- Kubernetes: Service

certs.eknathalabs.com

Q53 **Walk through exactly how a packet travels from one pod to another pod on a different node.** Networking

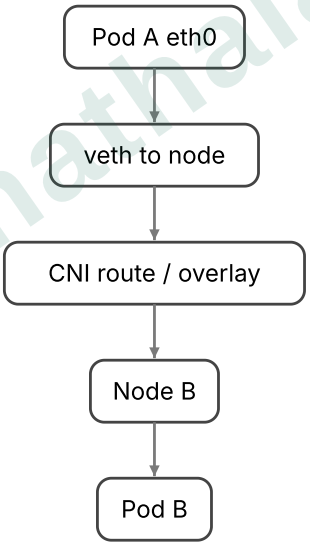
🎙 Spoken answer · ~76 s

Take pod A on node one talking to pod B on node two. Pod A sends to pod B's IP through its eth0, which is one end of a virtual ethernet pair. The other end lives in node one's root network namespace, attached to a bridge or handled directly by the CNI datapath. Node one then needs to know that B's IP belongs to node two's pod CIDR, which it learns through routes or the CNI's control plane. Now there are two common designs. With an overlay, such as Flannel VXLAN or Calico IP-in-IP, the packet is encapsulated inside another packet addressed to node two's IP, sent across the node network, and decapsulated on arrival. With routed or native networking, as in Calico BGP, or cloud VPC-native CNIs like the AWS and GKE ones, pod IPs are routable on the network, so there's no encapsulation, with better performance and easier debugging. On node two the packet is routed to B's veth and into its namespace. If the destination is a Service IP, kube-proxy or eBPF rewrites it to a pod IP first.

Likely follow-ups

- What are the trade-offs between overlay and routed CNIs?
- Where does a Service IP get translated to a pod IP?

🗺 Pod to pod



This is the canonical CNI question. The path crosses veth pairs, the node's network namespace, and an overlay or routed fabric.

THE PATH (POD A ON NODE1 -> POD B ON NODE2)

```
1. Pod A sends to Pod B's IP; leaves via eth0 (one end of a veth pair)
2. Other veth end is in node1's root netns, attached to a bridge / CNI datapath
3. Routing: node knows Pod B's IP belongs to node2's pod CIDR
4. Encapsulation (overlay: VXLAN/Geneve) OR direct routing (no overlay)
5. Across the node network to node2
6. Decapsulate -> node2 routes to Pod B's veth -> Pod B eth0
```

certs.eknathalabs.com

OVERLAY VS ROUTED

Overlay (Flannel VXLAN, Calico IP-in-IP): pod traffic is encapsulated; works anywhere but adds overhead. Routed/native (Calico BGP, GKE/EKS VPC-native): pod IPs are real routable addresses in the VPC; lower overhead, integrates with cloud routing.

Interview tip: The K8s networking model requires: every pod gets its own IP, pods communicate without NAT, and nodes can reach all pods. CNI plugins implement this. Mention eBPF-based Cilium replacing kube-proxy/iptables for performance, and that the veth pair + per-node pod CIDR is the core mechanism. Knowing overlay-vs-routed trade-offs signals depth.

📚 **Official docs**

- Kustomize reference

Q54 **How does kube-proxy implement Services? Compare iptables vs IPVS vs eBPF modes.** Networking

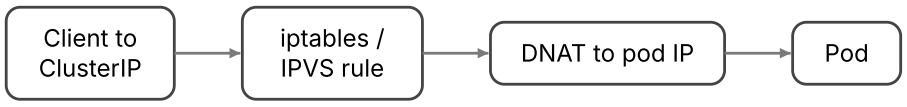
🎙 **Spoken answer · ~69 s**

kube-proxy makes Service virtual IPs work by programming each node's datapath so traffic to a Service IP is load-balanced to one of its backend pods. In iptables mode, it installs DNAT rules that pick a backend randomly. It's simple and reliable, but rules are evaluated linearly, so with thousands of Services the chains get huge and updates slow down. IPVS mode uses the kernel's IPVS with hash tables, giving constant-time lookups and proper balancing algorithms like round robin and least connections, which scales much better. eBPF, as in Cilium's kube-proxy replacement, goes further: it does Service load balancing in eBPF programs, often at the socket level, so there's no per-packet NAT chain, with better performance, observability, and consistent policy enforcement. My rule of thumb: iptables is fine for small to medium clusters, IPVS for large Service counts, and eBPF when I'm already running Cilium or want the best performance and visibility. Whichever mode, I watch for conntrack table limits, which cause drops under heavy load.

Likely follow-ups

- What problems occur when the conntrack table fills up?
- Why does iptables mode degrade with many Services?

🗺 **kube-proxy**



kube-proxy programs the node's datapath so a Service's virtual IP load-balances to backend pods. The mode determines how.

IPTABLES MODE (DEFAULT HISTORICALLY)

Installs iptables rules: Service VIP DNAT'd to a random backend pod IP. Simple and robust, but rule evaluation is O(N) — thousands of Services create huge rule chains and slow updates.

IPVS MODE

Uses the kernel's IPVS (hash tables) for O(1) lookup and real LB algorithms (round-robin, least-conn, etc.). Scales to thousands of Services far better than iptables.

certs.eknathalabs.com

EBPF (CILIUM, KUBE-PROXY REPLACEMENT)

```
# Cilium can REPLACE kube-proxy entirely:
# Service load-balancing done in eBPF at the socket/XDP layer
# - no iptables rule explosion, lower latency, better observability
```

Interview tip: The progression iptables -> IPVS -> eBPF is about scaling Service count and cutting latency. In large clusters, iptables rule churn becomes a real bottleneck (slow Service updates, CPU on rule eval). eBPF/Cilium is the modern direction — name it and explain *why* (no linear rule chains).

📚 **Official docs**

- Kubernetes: Service
- Kubernetes: kube-proxy
- Linux kernel docs: BPF

Q55 **Explain Kubernetes DNS. How is a Service name resolved and what are common DNS failure modes?** Networking

🎙 **Spoken answer · ~69 s**

CoreDNS resolves Service and pod names in the cluster. A Service is reachable at its name, namespace, svc, cluster.local. Each pod's resolv.conf lists search domains for its namespace and the cluster, the CoreDNS Service IP as nameserver, and ndots set to five. That ndots setting is a well-known trap. Any name with fewer than five dots is tried against each search domain first, so a lookup for api.example.com generates several failing queries before the real one succeeds. High-traffic applications generate a lot of extra DNS load and latency. Common failure modes: CoreDNS pods overloaded or crashing, a bad Corefile, upstream resolvers failing for external names, a NetworkPolicy blocking port 53 to CoreDNS, and the conntrack race on UDP that causes five-second delays. My fixes are: use fully qualified names with a trailing dot for external hosts, lower ndots in the pod dnsConfig, scale CoreDNS, and deploy NodeLocal DNSCache to cache close to the pod and avoid the race. I monitor CoreDNS latency and error metrics.

Likely follow-ups

- Why does a trailing dot help a DNS lookup?
- What does NodeLocal DNSCache change?

CoreDNS resolves Service and pod names cluster-wide. Understanding the resolution path and its pitfalls is key to debugging "intermittent" failures.

RESOLUTION

```
svc.namespace.svc.cluster.local   # fully qualified Service name
# A pod's /etc/resolv.conf:
search ns.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.96.0.10   # the CoreDNS Service ClusterIP
options ndots:5
```

certs.eknathalabs.com

THE NDOTS:5 PERFORMANCE TRAP

`ndots:5` means any name with fewer than 5 dots is first tried against every search-domain suffix. `api.example.com` (2 dots) becomes 4+ failed lookups before the real one. High-QPS apps generate massive DNS load and latency from this.

COMMON FAILURE MODES

```
- CoreDNS pods under-resourced / not enough replicas (use autoscaler)
- ndots:5 amplifying lookups (use FQDN with trailing dot, or NodeLocal DNSCache)
- conntrack race on UDP (old kernels) -> intermittent 5s timeouts
- NetworkPolicy accidentally blocking egress to CoreDNS
```

Interview tip: Name **NodeLocal DNSCache** as the standard fix — a per-node DNS cache that cuts CoreDNS load and avoids the UDP conntrack 5s-timeout bug. The `ndots:5` trap and using fully-qualified names (trailing dot) for external hosts is a classic senior optimization.

📚 **Official docs**

- Kubernetes: DNS for Services and Pods

Q56 **What is an Ingress vs a Service of type LoadBalancer vs the Gateway API? When use each?** Networking

🎙 **Spoken answer · ~70 s**

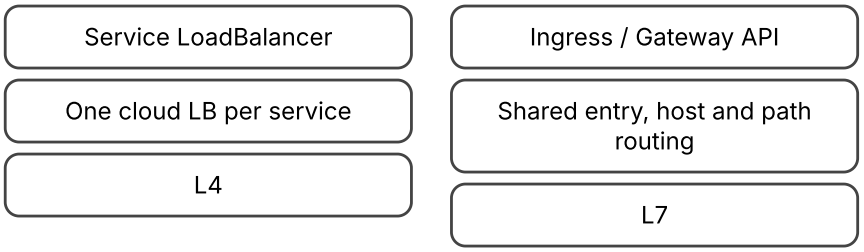
These expose workloads at different layers. A Service of type LoadBalancer works at layer four and provisions one cloud load balancer per Service. It's simple and works for any TCP or UDP protocol, but it costs one load balancer per Service and has no HTTP routing. Ingress works at layer seven: a shared controller, such as NGINX, Traefik, or a cloud load balancer, routes by host and path, terminates TLS, and lets many Services share one entry point. Its limitation is a minimal spec, so vendors add features through annotations, which makes configurations non-portable. Gateway API is the successor. It separates roles: GatewayClass for the infrastructure provider, Gateway for the cluster operator, and routes like HTTPRoute for application teams. It supports header and method matching, traffic weights, cross-namespace routing, and more protocols, in a portable, typed way. My guidance: use LoadBalancer for non-HTTP traffic or a single service, Ingress for existing HTTP setups, and Gateway API for new designs that need richer routing and clean team boundaries.

Likely follow-ups

- How does Gateway API support multi-team ownership?
- When is an Ingress annotation a portability risk?

certs.eknathalabs.com

🗺 **Exposing apps**



These expose workloads externally at different layers and maturity levels.

SERVICE TYPE LOADBALANCER

L4 (TCP/UDP). Provisions one cloud load balancer per Service — simple but one LB (and cost) per Service, no HTTP routing.

INGRESS

L7 HTTP(S) routing through a shared ingress controller (NGINX, Traefik, cloud LB). Host/path-based routing, TLS termination, one LB for many Services. The long-standing standard, but the spec is limited and vendors bolt on features via annotations.

GATEWAY API — THE SUCCESSOR

```
# Role-oriented: GatewayClass (infra) / Gateway (cluster-op) / HTTPRoute (app dev)
# Expressive routing (header/method/weight), protocol-aware, portable across vendors
# Replaces annotation-soup of Ingress; supports traffic splitting natively
```

Interview tip: Use LoadBalancer for raw TCP/non-HTTP; Ingress for HTTP routing today; and know the **Gateway API** is the future (GA, role separation, native traffic splitting and multi-protocol). Mentioning Gateway API and why it exists — to fix Ingress's expressiveness and annotation portability problems — signals current knowledge.

📚 **Official docs**

- Kubernetes: Service
- Kubernetes: Ingress
- Kubernetes: Gateway API

certs.eknathalabs.com

Q57 **Explain NetworkPolicies. Why is a pod 'open' by default and how do you implement zero-trust?** Networking

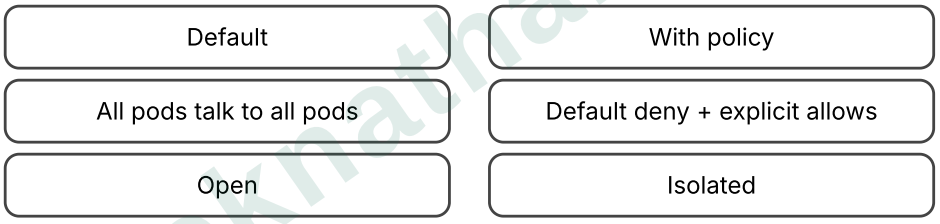
🎙 Spoken answer · ~75 s

By default Kubernetes networking is flat: any pod can talk to any other pod, in any namespace. NetworkPolicies add firewall rules for pod traffic, but only when the CNI enforces them, as Calico and Cilium do and plain Flannel doesn't. The key semantic is that a pod starts open, but as soon as any policy selects it for ingress or egress, that direction becomes deny by default, and only the rules listed are allowed; rules are additive. To implement zero trust, I start each namespace with a default-deny policy that selects all pods, with an empty podSelector and no rules. Then I add explicit allows per application: frontend to API on its port, API to database, and so on, using pod and namespace selectors. Egress needs care: I must explicitly allow DNS to CoreDNS and any required external destinations. I roll out in stages, using tools with flow visibility such as Cilium Hubble to see what traffic exists before enforcing, and I test in staging. NetworkPolicies are layer three and four, so identity-based mTLS in a mesh complements them.

Likely follow-ups

- Why does default-deny egress often break DNS?
- How would you roll out default deny safely on a live namespace?

🗺 NetworkPolicy



By default, all pods can talk to all pods — the cluster is flat and open. NetworkPolicies add firewall rules, but only if the CNI enforces them.

DEFAULT-OPEN, THEN DEFAULT-DENY

```
# Start zero-trust: deny all ingress in a namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny-ingress }
spec:
  podSelector: {}           # all pods in namespace
  policyTypes: [Ingress]    # no ingress rules = deny all ingress
```

Then add explicit allow rules per app. The moment a pod is selected by any policy, it switches from allow-all to allow-only-what's-listed.

ALLOW SPECIFIC TRAFFIC

certs.eknathalabs.com

```
ingress:
- from:
  - podSelector: { matchLabels: { app: frontend } }
  ports: [{ protocol: TCP, port: 8080 }]
```

Critical: NetworkPolicies are enforced by the CNI — Flannel (basic) ignores them; Calico/Cilium enforce them. A policy with no enforcing CNI is a silent no-op. Also remember to allow egress to CoreDNS (port 53) in default-deny-egress namespaces, or DNS breaks.

Interview tip: Zero-trust = default-deny + explicit allows per flow. Cilium adds L7-aware policies (allow only `GET /api`) and identity-based rules via eBPF, going beyond the L3/L4 stock NetworkPolicy.

📚 **Official docs**

- Kubernetes: Network Policies
- NIST Zero Trust Architecture (SP 800-207)

Q58 **You have intermittent 5-second latency on some pod-to-service calls. What's the likely cause?** Networking

🎙 **Spoken answer · ~72 s**

Intermittent delays of almost exactly five seconds are the signature of the DNS conntrack race. On some older kernels, a client sends the A and AAAA queries in parallel over UDP from the same socket, and a race in the kernel's connection tracking and NAT insertion causes one packet to be dropped. The resolver then waits for its default five-second timeout before retrying, so the request succeeds but slowly. The ndots five setting makes it more likely because it multiplies the number of lookups. I confirm by testing DNS from the affected pods and seeing occasional five-second lookups, checking conntrack insert-failed counters, and capturing packets. Fixes: deploy NodeLocal DNSCache so pods query a local cache and avoid the race; add resolv.conf options such as single-request-reopen, or use TCP for DNS; reduce ndots or use fully qualified names for external hosts; and upgrade to a kernel where the race is fixed. It's also worth checking CoreDNS capacity and upstream health, since overload produces similar timeouts, but the fixed five-second value is the giveaway.

Likely follow-ups

- How do you prove it's the conntrack race and not CoreDNS overload?
- What does single-request-reopen do?

Intermittent ~5s delays on DNS or new connections are a fingerprint of the classic conntrack/DNS race on UDP.

THE ROOT CAUSE

On older kernels, parallel DNS lookups (A and AAAA sent simultaneously over UDP) hit a race in the kernel conntrack DNAT insertion; one packet is dropped, and the client waits for the **5s DNS retransmit timeout**. The `ndots:5` setting multiplies lookups, increasing the chance.

FIXES

cons.eknathalabs.com

```
- NodeLocal DNSCache (queries go to a local cache over TCP-ish path; avoids the race)
- single-request-reopen / use-vc in resolv.conf options
- reduce ndots / use FQDN for external names
- upgrade kernel (the conntrack race is fixed in newer kernels)
```

HOW TO CONFIRM

```
kubectl exec <pod> -- sh -c 'time nslookup my-svc'   # ~5s = the tell
# tcpdump on the node for DNS retransmits; check CoreDNS latency metrics
```

Interview tip: The 5-second-exactly pattern is the giveaway — it maps to the DNS UDP retransmit timer. NodeLocal DNSCache is the canonical fix and a great thing to name. This question tests whether you've actually operated clusters, not just read docs.

📚 **Start here (official docs)**

- Kubernetes documentation

Q59 **Explain the PV / PVC / StorageClass model and dynamic provisioning end to end.** Storage

🎙 **Spoken answer · ~71 s**

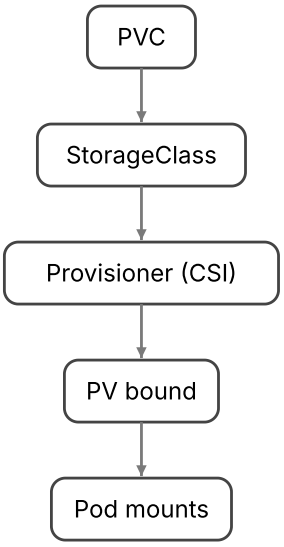
Kubernetes separates what an application asks for from how storage is provided. A StorageClass describes a type of storage and the provisioner behind it, for example gp3 or SSD. A PersistentVolumeClaim is a pod's request: a size, an access mode, and a class. A PersistentVolume is the actual provisioned disk, bound one to one to a claim. With dynamic provisioning, the flow is: a pod references a PVC; the PVC names a StorageClass; the CSI provisioner for that class creates a real disk in the cloud; a PV object is created and bound to the claim; and the volume is attached and mounted into the pod. Two details matter in practice. Access modes: ReadWriteOnce means one node at a time, which is what block disks give you, while ReadWriteMany needs shared file storage like NFS or EFS. And volumeBindingMode WaitForFirstConsumer delays creation until the pod is scheduled, so the disk lands in the right zone. The reclaim policy decides whether the disk is deleted or retained when the claim goes.

Likely follow-ups

- What does WaitForFirstConsumer solve?
- Delete versus Retain reclaim policy: which would you use for production data?

certs.eknathalabs.com

🗺 **Dynamic provisioning**



Kubernetes decouples storage requests (PVC) from storage implementation (PV) via StorageClass-driven dynamic provisioning.

THE OBJECTS

```
StorageClass : the "type" of storage + provisioner (e.g. gp3, ssd)
PVC          : a pod's REQUEST for storage (size, access mode, class)
PV           : the actual provisioned volume (bound 1:1 to a PVC)
```

DYNAMIC PROVISIONING FLOW

```
1. Pod references a PVC
2. PVC names a StorageClass
3. The CSI provisioner for that class creates a real disk (EBS/PD/etc.)
4. A PV is created and BOUND to the PVC
5. The volume is attached + mounted into the pod
```

ACCESS MODES & BINDING

`ReadWriteOnce` (one node), `ReadOnlyMany`, `ReadWriteMany` (needs NFS/EFS-style backend), `ReadWriteOncePod` (single pod). `volumeBindingMode: WaitForFirstConsumer` delays provisioning until a pod is scheduled, so the disk lands in the right zone.

Interview tip: The `WaitForFirstConsumer` binding mode is a key production detail — without it, a disk can be created in zone-a while the pod schedules to zone-b and can never mount it. Always use it for zonal block storage.

📚 **Official docs**

- Kubernetes: Persistent Volumes
- Kubernetes: StorageClasses

certs.eknathalabs.com

Q60 **What is the CSI architecture? Walk through what happens when a volume is attached to a pod.** Storage

🎙 Spoken answer · ~66 s

CSI, the Container Storage Interface, standardises how Kubernetes talks to storage systems through out-of-tree drivers, so vendors ship their own driver without changing Kubernetes. A driver has two parts. The controller plugin, usually a Deployment with sidecars such as external-provisioner and external-attacher, talks to the cloud or storage API to create, delete, attach, and detach volumes. The node plugin, a DaemonSet, runs on every node and does the local work, registered through the node-driver-registrar. When a pod using a volume is scheduled, the sequence is: first, ControllerPublishVolume attaches the disk to the chosen node through the cloud API. Then NodeStageVolume formats the disk if needed and mounts it at a global staging path on the node. Then NodePublishVolume bind-mounts that path into the pod's container directory. Unmounting reverses it. Understanding the split helps debugging: attach failures show up in VolumeAttachment objects and controller logs, while mount and permission failures show up in the node plugin and kubelet logs.

Likely follow-ups

- Where would you look if a volume attaches but the pod can't mount it?
- What do the CSI sidecar containers do?

The Container Storage Interface standardizes how K8s talks to storage backends via out-of-tree drivers. It has a controller plane and a node plane.

COMPONENTS

```
CSI Controller (Deployment): Provision/Delete + Attach/Detach (talks to cloud API)
CSI Node (DaemonSet)       : Stage + Publish (mounts on the actual node)
Sidecars: external-provisioner, external-attacher, node-driver-registrar
```

ATTACH -> MOUNT SEQUENCE

```
1. ControllerPublishVolume : attach disk to the node (cloud API call)
2. NodeStageVolume         : format + mount to a global staging path on node
3. NodePublishVolume       : bind-mount staging path into the pod's mount namespace
```

Detach reverses it on pod deletion. The split (stage once per node, publish per pod) lets multiple pods share a staged volume where allowed.

Interview tip: CSI also enables snapshots, volume expansion, and cloning as standardized features. A common failure: a volume stuck attaching because the previous node didn't cleanly detach (force-detach / `VolumeAttachment` cleanup needed). Knowing the controller-vs-node split explains why some failures are cloud-side (attach) vs node-side (mount).

📚 Start here (official docs)

- Kubernetes documentation

certs.eknathalabs.com

Q61 **A StatefulSet pod is stuck and its PVC won't move to a new node. Why, and how do you resolve it?** Storage

🎙 Spoken answer · ~77 s

This is stickiness by design. A StatefulSet pod is bound to its own PVC, and for block storage that PVC is a zonal, ReadWriteOnce disk, which can be attached to only one node at a time and only in its own availability zone. If the old node died uncleanly, the cloud or the VolumeAttachment object may still show the disk attached there, so the new node can't attach it, and the pod stays in ContainerCreating with multi-attach or attach timeout errors. Another cause is zone mismatch: the pod is scheduled to a node in another zone from the disk. To resolve it, I look at the pod events and kubectl get volumeattachment to find the stale attachment. If the old node is truly gone, I delete the Node object, or force delete the pod if I'm certain the node is dead, then the attach controller detaches the volume and the pod can start. I'm careful because forcing this while the old node is still writing can corrupt data. Prevention: use topology-aware scheduling, replicate data at the application layer, and make sure nodes drain cleanly.

Likely follow-ups

- Why is force-deleting a StatefulSet pod risky?
- How does application-level replication reduce this problem?

StatefulSet pods bind to specific PVCs and (for block storage) specific zones — that stickiness is by design but causes stuck-pod scenarios.

WHY IT'S STUCK

```
- ReadWriteOnce block volume can attach to only ONE node at a time
- If the old node died uncleanly, the volume is still "attached" to it
- The pod can't start on a new node until the volume detaches
- Zonal disk (EBS/PD) can only mount in its own zone -> pod pinned to that zone
```

RESOLUTION

```
kubectl get volumeattachment           # find the stale attachment
kubectl get pods -o wide               # confirm pod can't bind
# If node is truly gone: delete the Node object so the attach-detach
# controller force-detaches; then the pod can reschedule + mount.
kubectl delete node <dead-node>
```

Danger: Never force-detach a volume that might still be written to by a live node — RWO exists to prevent two writers corrupting data. Only force-detach when you're certain the old node is dead. This is why split-brain protection matters for stateful workloads.

Interview tip: Mention that the cloud's node-not-ready timeout (often ~6 min) gates automatic force-detach, which is why a failed-node stateful pod takes minutes to recover. For faster failover, operators (e.g. CloudNativePG) handle promotion at the app layer rather than relying on volume reattach.

📚 Official docs

- Kubernetes: Debug Pods
- Kubernetes: Persistent Volumes
- Kubernetes: StatefulSets

certs.eknathalabs.com

Q62 How do you expand a PVC, take snapshots, and back up stateful data in Kubernetes?

Storage

🎙 Spoken answer · ~70 s

CSI covers all three. For expansion, the StorageClass needs allowVolumeExpansion set to true, then I patch the PVC to request a larger size. The CSI driver grows the underlying disk, and on modern drivers the filesystem is resized online, often without restarting the pod; some need a restart. Expansion is one way only, you can't shrink. For snapshots, I create a VolumeSnapshot referencing the PVC and a VolumeSnapshotClass, and the driver takes a point-in-time snapshot of the disk. To restore, I create a new PVC whose dataSource points at the snapshot. But a storage snapshot is crash-consistent, not application-consistent, so for databases I either quiesce or flush before the snapshot, or use application-aware backups such as logical dumps or the database operator's backup feature. For real backup and disaster recovery, I use a tool like Velero, which captures Kubernetes objects and volume snapshots and stores them off-cluster in object storage, ideally in another region. And I test restores regularly, because that is the only proof the backup works.

Likely follow-ups

- Why isn't a volume snapshot enough for a database backup?
- What does Velero back up that a snapshot doesn't?

CSI standardizes volume expansion, snapshots, and (via tools) backup of stateful workloads.

ONLINE VOLUME EXPANSION

```
# StorageClass must have allowVolumeExpansion: true
kubectl patch pvc <pvc> -p '{"spec":{"resources":{"requests":{"storage":"50Gi"}}}}'
# CSI grows the disk; filesystem resize often happens online (no pod restart on modern CSI)
```

VOLUME SNAPSHOTS

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
spec:
  volumeSnapshotClassName: csi-snapclass
  source: { persistentVolumeClaimName: data-db-0 }
# restore by creating a PVC with dataSource pointing at the snapshot
```

APPLICATION-CONSISTENT BACKUPS

Snapshots are crash-consistent, not app-consistent — quiesce the DB (or use its native backup) for consistency. **Velero** backs up cluster objects + PV snapshots together and restores to another cluster (DR).

Gotcha: You can only *grow* a PVC, never shrink it. And a snapshot of a running database without flushing/locking may be unrestorable — pair CSI snapshots with the DB's own consistent-backup hooks.

Interview tip: Name Velero for cluster-level backup/DR and stress crash-consistent vs application-consistent — the distinction interviewers look for in anyone who's restored real stateful data.

📚 Official docs

- Kubernetes: Persistent Volumes

certs.eknathalabs.com

Q63 **How does service mesh mTLS work in Kubernetes, and what's the cost of running a mesh?**

Networking

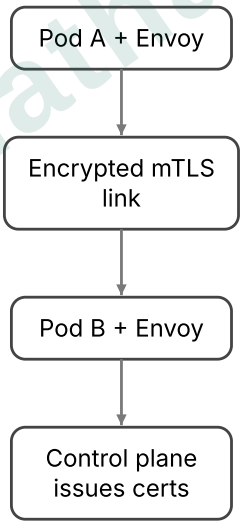
🎙 Spoken answer · ~73 s

A service mesh such as Istio or Linkerd injects a sidecar proxy into each pod, or uses node-level proxies in newer ambient designs, and uses it to secure and manage service-to-service traffic transparently. For mutual TLS, the control plane acts as a certificate authority and issues short-lived certificates to each workload, tied to a SPIFFE identity based on its service account. Traffic is redirected through the proxy using iptables or eBPF. When service A calls B, proxy A presents its certificate, proxy B verifies it and presents its own, and the traffic is encrypted, with authentication in both directions. The application code doesn't change. The mesh also provides retries, timeouts, circuit breaking, traffic splitting for canaries, and uniform L7 metrics and tracing. The costs are real: extra CPU and memory per pod, added latency per hop, more moving parts to operate and upgrade, a steep learning curve, and harder debugging because traffic passes through proxies. So I adopt a mesh when I need its capabilities, like compliance-driven mTLS across many services, not by default.

Likely follow-ups

- When would you not adopt a service mesh?
- What is the ambient mesh model trying to fix?

🗺 Mesh mTLS



A service mesh (Istio, Linkerd) injects a sidecar proxy per pod that transparently encrypts and authenticates service-to-service traffic with mutual TLS.

HOW MTLS WORKS

```
- Control plane issues short-lived per-workload certs (SPIFFE identity)
- Sidecar proxies intercept all pod traffic (via iptables/eBPF redirect)
- Proxy A presents its cert, proxy B verifies + presents its own -> mutual auth
- Traffic encrypted in transit; app code unchanged
```

certs.eknathalabs.com

WHAT THE MESH ADDS BEYOND MTLS

Uniform retries/timeouts/circuit-breaking, traffic splitting (canary), L7 observability (golden signals for every call), and authorization policies — all without app changes.

THE COST

```
- Per-pod sidecar: extra CPU/memory + ~1 hop of latency per call
- A control plane to operate and upgrade
- Debugging complexity (traffic goes through proxies)
- Overkill for a handful of services
```

Interview tip: Be balanced: mesh gives free mTLS + observability + traffic control, but the sidecar tax (resources, latency, ops) is real. Mention **Linkerd** (lighter) vs **Istio** (feature-rich) and the move to sidecar-less / ambient / eBPF (Cilium) meshes to cut the per-pod overhead.

📚 **Official docs**

- Cloudflare Learning: mutual TLS
- Istio documentation

Q64 **Explain dual-stack, IP exhaustion, and pod CIDR sizing problems at scale.** Networking

🎙 **Spoken answer · ~71 s**

At scale, IP address planning becomes a hard constraint because pod and service ranges are difficult to change later, often requiring a cluster rebuild. Each node is given a slice of the cluster's pod CIDR; a slash twenty-four per node gives about two hundred fifty pods per node, and the overall CIDR limits the number of nodes times pods per node. On VPC-native clusters, such as GKE alias IPs or the AWS VPC CNI, pods use real VPC addresses, so a large cluster can exhaust the VPC or subnet itself. Mitigations: size the ranges generously at creation for peak nodes and density; reduce the maximum pods per node if addresses are tight; use secondary ranges or non-routable pod ranges with masquerading; use prefix delegation on AWS; and plan the service CIDR separately. Dual-stack, supporting IPv4 and IPv6 together, is another way to escape IPv4 limits, but it adds operational complexity across load balancers, policies, and tooling, so it needs testing. The practical lesson: do IP capacity planning early and document it.

Likely follow-ups

- How does the AWS VPC CNI use IP addresses differently from an overlay?
- Can you change the pod CIDR on a running cluster?

At scale, pod/Service IP ranges become a hard constraint that must be planned up front — you can't easily resize them later.

THE EXHAUSTION PROBLEM

```
- Each node gets a slice of the pod CIDR (e.g. /24 = 254 pods/node)
- Cluster pod CIDR (e.g. /16) caps total nodes x pods-per-node
- VPC-native clusters consume REAL VPC IPs for pods -> can exhaust the VPC
- Resizing the cluster CIDR usually means rebuilding the cluster
```

MITIGATIONS

certs.eknathalabs.com

```
- Size the pod CIDR generously at creation (plan for peak nodes x density)
- Reduce per-node max pods if IPs are tight
- Use non-VPC-routable pod ranges / IP masquerade where appropriate
- Secondary ranges / custom CNI IPAM to extend address space
```

DUAL-STACK

IPv4/IPv6 dual-stack assigns both families to pods/Services, easing IPv4 exhaustion. Requires CNI + cloud support and careful Service `ipFamilyPolicy` config.

Interview tip: The headline is "CIDR sizing is a day-0 decision you can rarely undo." On EKS, the classic fix for VPC IP exhaustion is prefix delegation or custom networking; on GKE, planning the secondary ranges. Showing you'd capacity-plan IPs before cluster creation signals real scale experience.

📚 **Official docs**

- RFC 4632: CIDR

Q65 **When would you use emptyDir, hostPath, local PV, and network storage? Trade-offs of each?** Storage

🎙 **Spoken answer · ~73 s**

Each volume type fits a different need. emptyDir is scratch space tied to the pod's life: it survives container restarts but is deleted with the pod. It's fine for caches and temporary files, and with medium Memory it's a RAM-backed tmpfs that counts against the container's memory limit. hostPath mounts a directory from the node. It's tied to that node, behaves differently across nodes, and is a security risk because it can expose the host filesystem, so I limit it to node agents. A local PersistentVolume uses a node's local disk through the PV mechanism, with node affinity so scheduling is correct. It gives the best performance, such as NVMe for databases and caches, but the data is lost if the node is lost, and the pod is pinned to that node, so it needs application-level replication. Network storage, such as EBS, Persistent Disk, NFS, or Ceph, is durable and survives node loss, and can be reattached elsewhere, but has network latency and, for block, zonal attachment limits. My default for important data is network-backed PVCs.

Likely follow-ups

- When is a local PV worth the operational cost?
- Why does emptyDir with medium Memory risk OOM kills?

Different volume types suit different needs — ephemeral scratch vs node-local performance vs durable network storage.

THE OPTIONS

```
emptyDir   : ephemeral, lives with the pod; gone on pod delete (scratch, cache)
hostPath   : mounts a node directory; tied to that node; risky (security, portability)
local PV   : node-local disk as a PV; high perf, but pod pinned to that node
network PV : EBS/PD/NFS/Ceph; durable, survives node loss, portable across nodes
```

corrs.eknathalabs.com

TRADE-OFFS

emptyDir is fast and simple but non-durable (and `medium: Memory` counts against the mem limit). **hostPath** breaks scheduling portability and is a security risk (don't use in multi-tenant). **local PV** gives NVMe-class performance for databases but loses HA — node dies, data is stranded; rely on app-level replication. **Network PV** is durable and reschedulable but slower and zone-bound.

Interview tip: Local PV is the right call for performance-critical stateful systems that handle their own replication (Cassandra, Kafka) — you trade K8s-level durability for speed and let the app replicate. Network PV is the default for everything that needs the volume to follow the pod. Flag hostPath as a smell outside of node-agent DaemonSets.

📚 **Official docs**

- Kubernetes: Persistent Volumes

Q66

How do you expose a gRPC or long-lived connection service correctly behind a Kubernetes LB?

Networking

🎙 **Spoken answer · ~74 s**

gRPC uses long-lived HTTP/2 connections that multiplex many requests, and that interacts badly with connection-level load balancing. A Kubernetes Service, or a layer four cloud load balancer, balances connections, not requests. So a gRPC client opens one connection, it is pinned to a single backend pod, and all its requests go there. Load becomes uneven, and when you scale up, new pods get no traffic until clients reconnect. There are three fixes. First, use a layer seven proxy that understands HTTP/2 and balances per request: an Envoy-based ingress or gateway, or a service mesh like Linkerd or Istio, which does it transparently. Second, client-side load balancing: use a headless Service so DNS returns all pod IPs, and have the gRPC client use a round-robin policy and re-resolve DNS periodically. Third, limit connection lifetime with settings like max connection age on the server so clients reconnect and rebalance, with graceful draining. I also make sure the ingress or cloud load balancer is configured for HTTP/2 and has suitable idle timeouts, and that health checks use gRPC health checking.

Likely follow-ups

- Why does scaling up not help a gRPC service behind a ClusterIP Service?
- What does max connection age do?

L4 load balancers and HTTP/2 (gRPC) interact badly with the default Service round-robin, causing uneven load.

THE PROBLEM

gRPC multiplexes many requests over one long-lived HTTP/2 connection. A connection-level (L4) LB pins that connection to one backend pod, so all requests from a client hit one pod — load is uneven, and new pods get no traffic until clients reconnect.

FIXES

```
- Use an L7 proxy that load-balances per-REQUEST (Envoy/Ingress with HTTP/2,
  or a service mesh) instead of per-connection
- Client-side load balancing: headless Service -> client sees all pod IPs,
  balances requests itself (gRPC name resolver + round_robin policy)
- Periodic connection recycling (MAX_CONNECTION_AGE) to redistribute
```

certs.eknathalabs.com

Gotcha: Scaling up a gRPC service often doesn't relieve load because existing connections never move. You must force rebalancing (request-level LB, headless+client LB, or connection max-age) — adding replicas alone does nothing.

Interview tip: This is a favorite "have you actually run gRPC on K8s" question. The crisp answer: per-connection L4 balancing breaks HTTP/2 multiplexing; solve with request-level L7 balancing, client-side LB via headless Service, or connection age limits.

📚 **Official docs**

- gRPC documentation

Q67 **What is the Operator pattern and how does it manage stateful applications via CRDs?** Storage

🎙 **Spoken answer · ~72 s**

An Operator packages human operational knowledge for an application into a controller. You define a CustomResourceDefinition, which adds a new resource type, for example a PostgresCluster. A user creates an instance of it, stating the desired outcome: three nodes, version sixteen, daily backups. The operator watches those custom resources and continuously reconciles reality toward that spec, creating the StatefulSets, Services, and Secrets, but also doing the domain-specific work. That's why operators suit stateful applications: a StatefulSet gives stable identity and storage but knows nothing about the application. An operator adds primary election and failover, replica configuration, backups and point-in-time recovery, rolling upgrades in the right order, scaling, and certificate handling. Examples are CloudNativePG for Postgres and Strimzi for Kafka. The pattern is the same control loop that Kubernetes itself uses. When choosing one, I check maturity, how it handles failure and upgrades, and its backup story, since an immature operator on production data is a risk. Building my own is possible with Kubebuilder or Operator SDK, but I would avoid it unless necessary.

Likely follow-ups

- What is a reconcile loop and why must it be idempotent?
- How do you evaluate whether an operator is production ready?

An Operator encodes human operational knowledge (deploy, backup, failover, upgrade) into a controller that reconciles a Custom Resource toward desired state.

THE MECHANISM

```
CRD: defines a new type, e.g. kind: PostgresCluster
CR : an instance: "I want a 3-node Postgres, v16, with backups"
Operator (controller): watches CRs -> reconciles reality to match
  (creates StatefulSet, Services, secrets, runs failover, schedules backups)
```

WHY FOR STATEFUL APPS

StatefulSets give identity + storage but not application logic. An Operator adds the domain smarts: primary election, automated failover, point-in-time restore, version upgrades with data migration, scaling with rebalancing.

EXAMPLES

CloudNativePG / Zalando (Postgres), Strimzi (Kafka), Elastic Operator, Prometheus Operator. The reconcile loop (observe -> diff -> act) is the same control-loop pattern as built-in controllers.

certs.eknathalabs.com

Interview tip: Frame Operators as "automating the on-call runbook." The CRD extends the K8s API; the controller is just another reconcile loop. For production databases on K8s, an Operator is strongly preferred over a hand-rolled StatefulSet because it handles the hard parts — failover and backups — that raw StatefulSets don't.

📚 **Official docs**

- Kubernetes: Custom Resources
- Kubernetes: Operator pattern

Q68 **How does external traffic reach a pod end-to-end (cloud LB to container)? What is externalTrafficPolicy?**

Networking

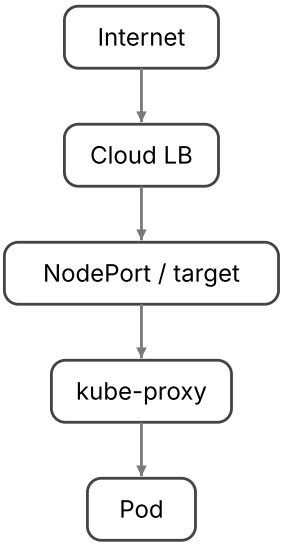
🎙 **Spoken answer · ~72 s**

With a Service of type LoadBalancer, traffic goes from the internet to the cloud load balancer, which targets a NodePort on the cluster's nodes, then kube-proxy on that node, and finally to a pod. The setting externalTrafficPolicy controls the behaviour. With Cluster, the default, the load balancer can send traffic to any node, and kube-proxy may forward it to a pod on a different node. That adds a hop and, because it SNATs the packet, hides the real client IP from the application, but load is spread evenly. With Local, only nodes that actually run a backend pod accept the traffic. There's no second hop, and the client source IP is preserved, which matters for logging, allow-listing, and rate limiting. The downside is that load can be uneven if pods are spread unevenly across nodes, and the load balancer's health checks must correctly exclude nodes without local pods. During rolling updates, connection draining matters too. Some clouds can bypass NodePort entirely by sending traffic directly to pod IPs, which improves latency and visibility.

Likely follow-ups

- How do you preserve the client IP behind a cloud load balancer?
- What problem does Local policy create for pod distribution?

🗺 **External traffic**



certs.eknathalabs.com

External ingress traverses a cloud LB, a node port, kube-proxy, and finally the pod. `externalTrafficPolicy` changes the hop behavior and source-IP visibility.

THE PATH

```
Internet -> Cloud LB -> NodePort on a node -> kube-proxy -> pod
# With type=LoadBalancer, the cloud LB targets node ports across the fleet
```

EXTERNALTRAFFICPOLICY: CLUSTER VS LOCAL

```
Cluster (default): LB hits any node; kube-proxy may forward to a pod on ANOTHER
  node -> extra hop, SNAT hides the real client source IP, even load
Local: only nodes that HAVE a backend pod accept traffic; no second hop,
  PRESERVES client source IP, but load can be uneven (depends on pod placement)
```

IMPLICATIONS

`Local` preserves client IP (needed for allowlists, geo, logging) and avoids a hop, but requires the LB health check to only target nodes with pods, and pod distribution affects balance.

Interview tip: The source-IP question is the heart of it: default `Cluster` policy SNATs and loses the client IP; switch to `Local` to preserve it (or use proxy protocol / X-Forwarded-For at L7). Knowing the extra-hop and load-balance trade-off of `Local` shows real network operations experience.

📚 **Official docs**

- Kubernetes: Service

Q69 **How do you run a database on Kubernetes safely? What are the real risks and mitigations?** Storage

🎙 **Spoken answer · ~73 s**

Running a database on Kubernetes is viable, but the risks centre on data safety during failures, so I'd be deliberate. The real risks are: when a node fails, the volume may take minutes to detach and reattach, causing downtime; misuse of force detach on a ReadWriteOnce volume can lead to split-brain and corruption; a pod could be rescheduled away from its zonal disk; an aggressive liveness probe can restart a healthy but busy database; and noisy neighbours or control plane problems cause churn. Mitigations: use a mature operator such as CloudNativePG or Strimzi for failover, backups, and upgrades rather than a hand-rolled StatefulSet; run at least three replicas with application-level replication so losing a node doesn't lose data; spread them across zones; use anti-affinity and PDBs; give the database dedicated nodes via taints, with Guaranteed QoS and fast storage; use conservative probes; and, most importantly, keep tested backups with point-in-time recovery stored off-cluster. And be honest about the alternative: a managed database often moves these risks to the provider, which is frequently the better choice.

Likely follow-ups

- When would you recommend a managed database instead?
- Why are liveness probes dangerous on databases?

certs.eknathalabs.com

Running stateful databases on K8s is viable but demands care — the risks are around data safety during node/volume failures.

THE REAL RISKS

```
- Volume reattach delay on node failure (minutes) -> downtime
- RWO split-brain if force-detach is misused -> data corruption
- Naive rescheduling moving a pod away from its zonal disk
- An aggressive liveness probe restarting a healthy-but-slow DB
- etcd/control-plane issues cascading to data-plane churn
```

MITIGATIONS

```
- Use an Operator (CloudNativePG, Strimzi) for failover + backups, not raw StatefulSet
- App-level replication (>=3 replicas) so a lost node/volume isn't data loss
- StatefulSet + headless Service + per-pod PVC + anti-affinity across zones
- Conservative probes; PDBs; tested restore (Velero / native backups)
- Local NVMe PV + app replication for performance-critical stores
```

Honest take: For mission-critical OLTP, many teams still use managed DBs (Cloud SQL, RDS) and keep K8s for stateless tiers. Running DBs on K8s is best when an Operator handles failover/backup and the DB replicates at the application layer.

Interview tip: The mature answer weighs build-vs-buy: K8s-hosted DB (control, portability, cost) vs managed DB (operational safety). If on K8s, an Operator + app-level replication + tested backups are non-negotiable.

📚 **Start here (official docs)**

- Kubernetes documentation

Q70 **Explain Kubernetes RBAC. How do Roles, ClusterRoles, and bindings actually combine?** Security

🎙 **Spoken answer · ~71 s**

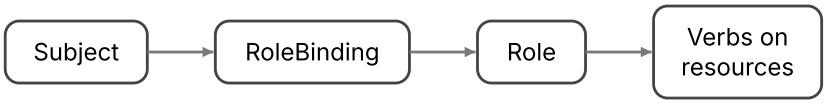
RBAC answers who can do what to which resources, using four objects. A Role grants permissions inside one namespace: verbs on resources. A ClusterRole is cluster-wide permissions, and also covers cluster-scoped resources like nodes and non-resource URLs. A RoleBinding grants a Role or a ClusterRole to subjects, which are users, groups, or service accounts, within one namespace. A ClusterRoleBinding grants a ClusterRole across the entire cluster. The subtle combination that confuses people is that a RoleBinding can reference a ClusterRole. The permissions come from the ClusterRole, but they apply only inside the binding's namespace. That lets me define a permission set once, like view or edit, and reuse it per namespace. RBAC is additive: there are no deny rules, so the union of all bindings applies, and I audit with kubectl auth can-i. Good practice is least privilege, namespace-scoped bindings, groups instead of individual users, avoiding wildcards, and never binding cluster-admin to applications. Also, some permissions, such as creating pods or reading secrets, are effectively powerful and need extra care.

Likely follow-ups

- Why is there no deny rule in RBAC?
- Which verbs or resources are effectively privilege escalation?

certs.eknathalabs.com

🗺 **RBAC**



RBAC governs who can do what on which resources. Four objects compose: Role/ClusterRole (permissions) and RoleBinding/ClusterRoleBinding (who gets them).

THE FOUR OBJECTS

```
Role               : permissions WITHIN a namespace
ClusterRole        : cluster-wide permissions (or reusable across namespaces)
RoleBinding        : grant a Role (or ClusterRole) to subjects in a namespace
ClusterRoleBinding : grant a ClusterRole cluster-wide
```

A SUBTLE COMBINATION

A **RoleBinding** can reference a **ClusterRole** — this grants that ClusterRole's permissions but scoped to the binding's namespace. This is how you define a permission set once (ClusterRole) and apply it per-namespace.

```
rules:
- apiGroups: [""]
  resources: ["pods","pods/log"]
  verbs: ["get","list","watch"]
```

RBAC is additive only — there are no deny rules. Permissions accumulate from all bindings a subject has. You restrict by granting less, never by denying. Audit with `kubectl auth can-i --list --as=<user>`.

Interview tip: Least privilege: prefer namespaced Roles over ClusterRoles, bind to specific ServiceAccounts not `default`, and avoid wildcards (`verbs: ["*"]`). Mention `kubectl auth can-i` for verification — it shows you actually audit RBAC.

📚 **Official docs**

- Kubernetes: RBAC

certs.eknathalabs.com

Q71 **What replaced PodSecurityPolicy? Explain Pod Security Admission and its three levels.** Security

🎙 Spoken answer · ~70 s

PodSecurityPolicy was removed in Kubernetes one point twenty-five. Its replacement is Pod Security Admission, a built-in admission controller driven by labels on namespaces, which enforces the Pod Security Standards. There are three standards. Privileged is unrestricted. Baseline blocks known privilege escalations: no privileged containers, no host namespaces, restricted host paths and capabilities. Restricted follows current hardening best practice: must run as non-root, drop all capabilities, use a seccomp profile, and disallow privilege escalation. There are three modes, set per namespace: enforce rejects violating pods, audit records violations in the audit log, and warn shows a warning to the user. A good rollout is to start with warn and audit on restricted to see what would break, fix the workloads, then switch on enforce. PSA is deliberately simple. It can't do exceptions per workload or custom rules, so for finer control, or mutation, I pair it with Kyverno or Gatekeeper, or ValidatingAdmissionPolicy. I'd apply restricted to application namespaces and baseline or privileged only to system namespaces that need it.

Likely follow-ups

- How do you migrate from PSP to PSA without breaking workloads?
- What can't PSA do that Kyverno can?

PodSecurityPolicy (PSP) was removed in 1.25. Pod Security Admission (PSA) is the built-in replacement — a namespace-label-driven admission controller enforcing the Pod Security Standards.

THREE STANDARDS

```
privileged : unrestricted (no constraints)
baseline   : prevents known privilege escalations (no hostNetwork, no privileged)
restricted : hardened best-practice (non-root, drop ALL caps, seccomp, RO rootfs)
```

THREE MODES, SET VIA NAMESPACE LABELS

```
kubectl label ns prod \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/warn=restricted \
  pod-security.kubernetes.io/audit=restricted
# enforce = block; warn = allow but warn; audit = log to audit only
```

You can enforce baseline while warning on restricted, easing migration.

Limitation: PSA is namespace-level and coarse (3 fixed profiles). For fine-grained or custom policy (registry allowlists, required labels, specific seccomp profiles), use a policy engine — OPA Gatekeeper or Kyverno — alongside or instead of PSA.

Interview tip: Know the timeline (PSP deprecated 1.21, removed 1.25 -> PSA). The `restricted` profile maps to the hardening you'd describe anyway: runAsNonRoot, drop all capabilities, seccomp RuntimeDefault, no privilege escalation, read-only root FS.

📚 Official docs

- Kubernetes: Pod Security Standards

certs.eknathalabs.com

Q72 How do you harden a pod's securityContext for a CKS-level restricted workload?

Security

🎙 Spoken answer · ~73 s

A hardened securityContext reduces the damage if a container is compromised. The core settings are: runAsNonRoot true with an explicit non-zero user, so the process can't run as UID zero; allowPrivilegeEscalation false, which blocks gaining more privileges than the parent, such as via setuid binaries; readOnlyRootFilesystem true, so an attacker can't modify the container's files, with writable emptyDir mounts only where needed, such as a temp directory; capabilities dropped entirely, adding back only specific ones, like binding a low port if truly required; and a seccomp profile of RuntimeDefault, which blocks dozens of rarely needed, dangerous system calls. I'd also avoid privileged mode, host namespaces, and hostPath mounts, set an fsGroup when volumes need group access, and disable service account token mounting if the API isn't needed. The image must support this, so it has a non-root user and doesn't write to unexpected places. I then enforce it with Pod Security Admission at the restricted level, and validate with kube-bench and runtime tools. For the CKS, I'd know each field and what it prevents.

Likely follow-ups

- What typically breaks when you enable readOnlyRootFilesystem?
- Which capability would you add back to bind port 80 as non-root?

A hardened securityContext minimizes the blast radius if a container is compromised. This is core CKS material.

THE HARDENED SPEC

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop: ["ALL"]
  seccompProfile:
    type: RuntimeDefault
```

WHAT EACH DOES

```
runAsNonRoot             : refuse to run as UID 0
allowPrivilegeEscalation: block setuid/gaining more privs than parent
readOnlyRootFilesystem   : container can't write its own FS (use emptyDir for tmp)
drop ALL capabilities    : remove all Linux caps, add back only what's needed
seccompProfile           : restrict syscalls to the runtime default allowlist
```

Never: `privileged: true` (full host access), mounting the Docker socket, `hostPID/hostNetwork/hostIPC` unless an agent truly needs it, or running as root. Each is a container-escape vector.

Interview tip: If the app needs to bind a low port, add `NET_BIND_SERVICE` back rather than running as root. Mention AppArmor/SELinux profiles and a read-only rootfs + writable emptyDir for `/tmp` as the standard pattern. This is exactly what the PSA `restricted` profile enforces.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Security context

Q73 **Explain how a container escape could happen and the layered defenses against it.** Security

🎙 **Spoken answer · ~69 s**

A container is just a process with namespaces and cgroups, sharing the host kernel, so an escape exploits a misconfiguration or kernel bug to reach the host. The common vectors are: privileged containers, which have nearly full host access; mounting the container runtime socket, which lets you control the runtime and start privileged containers; hostPath mounts of sensitive directories; powerful capabilities like SYS_ADMIN; sharing the host PID or network namespace; and kernel vulnerabilities, since the kernel is shared. The defence is layered. Prevent: don't grant privilege, drop capabilities, avoid host mounts and namespaces, run as non-root with a read-only filesystem, and enforce this by Pod Security Admission and policy engines. Reduce kernel exposure: seccomp, AppArmor or SELinux, and sandboxed runtimes such as gVisor or Kata Containers for untrusted workloads. Limit what an escape yields: separate node pools for sensitive workloads, minimal service account permissions, network policies, and no cloud credentials on nodes. Detect: Falco for suspicious runtime behaviour. And keep the kernel and runtime patched.

Likely follow-ups

- Why is mounting the Docker or containerd socket so dangerous?
- When do gVisor or Kata Containers justify their overhead?

A container is a process with namespaces + cgroups, not a VM. Escapes exploit misconfigurations or kernel bugs to reach the host.

COMMON ESCAPE VECTORS

```
- privileged: true              -> near-full host access
- Mounting the host Docker/containerd socket -> control the runtime
- hostPath mount of / or /var    -> read/write host files
- CAP_SYS_ADMIN and friends       -> many escape primitives
- Kernel exploit (shared kernel)  -> break out via a vuln
- Writable /proc or host PID ns    -> tamper with host processes
```

LAYERED DEFENSES (DEFENSE IN DEPTH)

```
1. Don't grant privilege: drop caps, no privileged, no host mounts/namespaces
2. seccomp + AppArmor/SELinux: restrict syscalls and file access
3. Run as non-root + read-only rootfs
4. Stronger isolation: gVisor (userspace kernel) or Kata (microVM) per pod
5. Runtime detection: Falco/Tetragon alert on suspicious syscalls
6. Patch nodes (kernel) promptly
```

Interview tip: The headline: containers share the host kernel, so isolation is only as strong as the kernel + your constraints. For hostile multi-tenant workloads, name **gVisor** or **Kata Containers** (VM-level isolation) as the answer when namespaces aren't enough, and **Falco** for runtime threat detection.

certs.eknathalabs.com

📚 **Official docs**

- Docker: Engine security

Q74 **How does Workload Identity (GKE) / IRSA (EKS) work and why is it better than node service accounts?** Security

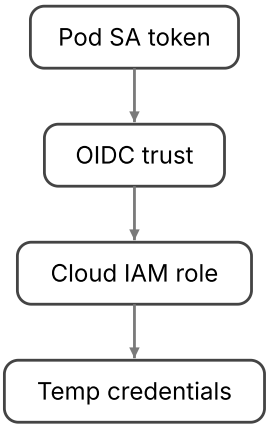
🎙 **Spoken answer · ~73 s**

Workload Identity on GKE, and IRSA on EKS, let a pod assume a cloud identity without any stored credentials. You map a Kubernetes ServiceAccount to a cloud IAM identity, a Google service account or an AWS role, using an annotation and a trust relationship. The pod receives a projected, audience-scoped service account token, an OIDC JWT, and the cloud's security token service exchanges it for short-lived cloud credentials, scoped to that role and rotated automatically. This is better than the alternatives for three reasons. Node service accounts mean every pod on the node inherits the node's cloud permissions, so a compromised pod gets broad access. Static keys stored in Secrets or environment variables are long-lived, can leak, and need manual rotation. With workload identity, permissions are per workload and least-privilege, credentials are short-lived with nothing to leak or rotate, and the audit logs show which workload did what. Operationally, I give each workload its own service account, restrict the node's own permissions to the minimum, and block pod access to the node metadata server.

Likely follow-ups

- How do you stop pods reaching the node metadata endpoint?
- What is the trust relationship in IRSA?

🗺 **IRSA / Workload Identity**



Workload Identity binds a Kubernetes ServiceAccount to a cloud IAM identity so pods get short-lived, scoped cloud credentials without static keys.

THE MECHANISM

```
# K8s SA -> annotated to map to a cloud IAM role/SA
# Pod requests a projected SA token (OIDC) -> exchanged for cloud creds
# Creds are short-lived, auto-rotated, scoped to that workload's IAM role
```

WHY BETTER THAN NODE SA / STATIC KEYS

certs.eknathalabs.com

```
Node SA       : EVERY pod on the node inherits the node's cloud permissions
                (over-broad; a compromised pod gets node-level cloud access)
Static keys   : long-lived secrets in env/Secret -> leak risk, manual rotation
Workload Id   : per-pod least-privilege, no stored secret, auto-rotated
```

Anti-pattern: Giving the node's service account broad cloud IAM and relying on it for pod access — any pod (and any attacker who lands in one) gets those permissions. Disable legacy metadata access and use Workload Identity / IRSA so each workload gets only its own role.

Interview tip: This maps directly to your earlier secrets point — Workload Identity solves the bootstrapping problem (how a pod proves who it is) without a stored secret. On EKS it's IRSA (IAM Roles for Service Accounts) via OIDC; on GKE it's Workload Identity. Per-pod least privilege is the win.

📚 **Official docs**

- Kubernetes: Service accounts
- Amazon EKS: IAM roles for service accounts (IRSA)
- Azure: Managed identities

Q75 **What is admission control? Compare validating/mutating webhooks, OPA Gatekeeper, and Kyverno.** Security

🎙 **Spoken answer · ~71 s**

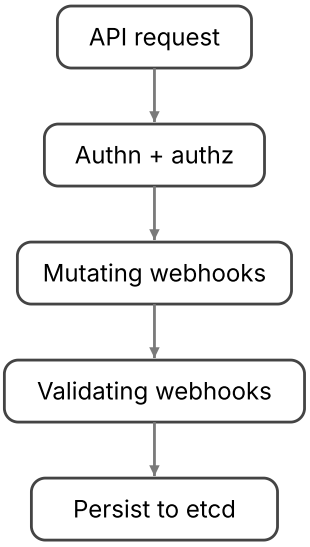
Admission control is the stage where the API server intercepts a request after authentication and authorisation, but before it is saved to etcd. It can mutate objects and validate them, which makes it the enforcement point for policy. There are two webhook types. Mutating webhooks can modify the object, for example injecting a sidecar, adding labels, or setting defaults. Validating webhooks run afterwards and can only accept or reject. For policy, OPA Gatekeeper uses the Rego language, with ConstraintTemplates and Constraints. It's very powerful for complex logic but has a steeper learning curve. Kyverno uses Kubernetes-native YAML policies and can validate, mutate, generate resources, and verify image signatures, so it is easier for most teams. There's also the built-in ValidatingAdmissionPolicy, which uses CEL expressions inside the API server, avoiding webhook latency and availability risk. Operationally, webhooks are a risk: a slow or unavailable webhook with failurePolicy Fail can block the cluster. So I set short timeouts, exclude system namespaces, run multiple replicas, and begin policies in audit mode before enforcing.

Likely follow-ups

- How can a failing webhook break cluster operations?
- When would you pick ValidatingAdmissionPolicy over Kyverno?

certs.eknathalabs.com

🗺 **Admission**



Admission controllers intercept API requests after authn/authz but before persistence, to validate or mutate objects — the enforcement point for policy.

TWO WEBHOOK TYPES

```
Mutating webhook   : can MODIFY the object (inject sidecar, add labels, set defaults)
Validating webhook : can ACCEPT or REJECT (enforce policy), runs after mutating
```

POLICY ENGINES

```
OPA Gatekeeper : policies in Rego (powerful, steeper learning curve)
                 ConstraintTemplates + Constraints; great for complex logic
Kyverno        : policies as YAML/Kubernetes-native (easier to read/write)
                 validate, mutate, generate, and verifyImages in one tool
```

TYPICAL POLICIES

Require resource limits, block `:latest` tags, enforce allowed registries, require labels/probes, disallow privileged, verify image signatures (cosign).

Gotcha: A failing/slow validating webhook with `failurePolicy: Fail` can block ALL API writes (including system controllers) — a self-inflicted outage. Scope webhooks narrowly (namespaceSelector, objectSelector) and consider `failurePolicy: Ignore` for non-critical policies.

Interview tip: Kyverno (YAML-native, simpler) vs Gatekeeper/OPA (Rego, more powerful) is the comparison to make. Both replace the removed PSP for custom policy and integrate supply-chain checks (image signature verification).

📚 **Official docs**

- Kubernetes: Admission controllers
- Google SRE Book: Handling Overload
- OPA Gatekeeper

certs.eknathalabs.com

Q76 **Explain supply-chain security for containers: image scanning, signing, SBOM, and admission.**

Security

🎙 Spoken answer · ~74 s

Supply-chain security is making sure that what runs in the cluster is exactly what you built from trusted sources, with known contents. It has four pillars. Scanning: tools like Trivy or Grype check images for known vulnerabilities in CI, failing the build on critical findings, and re-scan in the registry as new CVEs appear. SBOM: generate a software bill of materials with a tool like Syft, so I know every component inside an image and can find which images are affected when a vulnerability drops. Signing: sign images and attestations, such as build provenance, with cosign from Sigstore, ideally with keyless signing tied to the CI identity, aiming at SLSA levels. Admission: at the cluster, a policy engine such as Kyverno or the Sigstore policy controller rejects any image that is unsigned, comes from an unapproved registry, or lacks the required attestations. I also pin by digest, use minimal base images, and protect the build system itself, since the pipeline is part of the attack surface. Together this gives a verifiable chain from source to running container.

Likely follow-ups

- What does keyless signing with cosign mean?
- Why pin an image by digest rather than a tag?

🗺 Supply chain



Supply-chain security ensures only trusted, verified images run. The pillars are scanning, SBOM, signing, and admission enforcement.

THE PIPELINE

```
1. SCAN   : Trivy/Grype scan image for CVEs in build CI (fail on criticals)
2. SBOM   : generate a Software Bill of Materials (Syft) -> know what's inside
3. SIGN   : cosign signs the image + attestations (provenance, SLSA)
4. ADMIT  : cluster admission verifies signature before allowing the image to run
```

ADMISSION-TIME VERIFICATION

```
# Kyverno / Gatekeeper / sigstore policy-controller:
# reject any image that is unsigned or not from an allowed registry,
# and verify the cosign signature + SLSA provenance attestation
```

HARDENING THE IMAGE ITSELF

Minimal/distroless base, non-root user, pinned digests (not tags), multi-stage builds to drop build tooling, no secrets baked in.

Interview tip: Name the concrete tools: **Trivy/Grype** (scan), **Syft** (SBOM), **cosign/sigstore** (sign + verify), **SLSA** (provenance framework), and admission verification via Kyverno/policy-controller. Pinning by digest and verifying signatures at admission closes the "someone pushed a malicious :latest" gap.

certs.eknathalabs.com

📚 **Official docs**

- SLSA specification
- SPDX specification
- CycloneDX

Q77 **How do you manage secrets securely in Kubernetes beyond the built-in Secret object?** Security

🎙 **Spoken answer · ~70 s**

Native Secrets are only base64-encoded in etcd and are easy to over-expose, so I improve this in layers. First, encrypt etcd at rest using an EncryptionConfiguration with a KMS provider, so etcd holds ciphertext. Second, tight RBAC on the secrets resource and audit logging of access. Third, keep the source of truth outside the cluster, in Vault or a cloud secret manager, and sync it with the External Secrets Operator, which creates Kubernetes Secrets automatically and supports rotation. Fourth, for the strongest model, the Secrets Store CSI driver mounts secrets directly into the pod at runtime, so nothing is persisted in etcd. Fifth, use dynamic secrets, such as Vault generating short-lived database credentials per workload that expire automatically, which limits the damage of a leak. For GitOps, never commit plain secrets: use Sealed Secrets or SOPS so only encrypted values are in Git, or reference external secrets. Also, prefer file mounts over environment variables, rotate regularly, and give workloads access through workload identity rather than long-lived tokens.

Likely follow-ups

- What are the trade-offs between External Secrets Operator and the CSI driver?
- How do dynamic secrets limit blast radius?

Native Secrets are only base64-encoded in etcd and easily over-exposed. Production setups keep secrets out of etcd/git and inject them just-in-time.

LAYERED IMPROVEMENTS

```
1. Encryption at rest: EncryptionConfiguration with a KMS provider
   (so etcd holds ciphertext, not base64)
2. Tight RBAC on the secrets resource + audit access
3. External Secrets Operator: sync from Vault/cloud Secret Manager into K8s Secrets
4. CSI Secrets Store: mount secrets directly from Vault/cloud at pod runtime
   (nothing persisted in etcd)
5. Vault dynamic secrets: short-lived, per-request DB creds that auto-expire
```

GITOPS + SECRETS

Never commit plaintext secrets. Use Sealed Secrets (encrypted, safe in git, decrypted in-cluster) or reference an external store via External Secrets Operator so git holds only a pointer.

Anti-pattern: Plaintext secrets in a Git repo, in container env baked into the image, or shared across all namespaces. Any of these turns one leak into total compromise.

Interview tip: The strongest answer mirrors your secrets-management design point: prefer dynamic, short-lived secrets (Vault) and runtime injection (CSI Secrets Store) so credentials never sit in etcd or git, and solve identity via Workload Identity rather than a stored token.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Good practices for Secrets

Q78 **Walk through securing the Kubernetes API server and control plane (CKS perspective).** Security

🎙 **Spoken answer · ~75 s**

The API server is the front door of the cluster, so I harden it and the control plane in several areas. For access: strong authentication, using OIDC or cloud IAM, with anonymous auth disabled and no shared static credentials; RBAC kept to least privilege, with no casual cluster-admin; and the node authorizer and admission controllers enabled. Visibility: enable audit logging with a sensible policy and ship it to a central store, since without it there's no forensics. Network: keep the control plane private or limit it with authorised networks, and use TLS everywhere with certificate rotation. etcd holds all data including secrets, so I encrypt it at rest with a KMS provider, require mTLS between the API server and etcd, restrict network access to etcd, and take encrypted backups. On the nodes, I secure the kubelet by disabling anonymous access, setting authorisation mode to Webhook, and turning on rotation. I protect component config file permissions, and then verify against the CIS benchmark with kube-bench. On managed clusters, much of this is the provider's job, but access and audit are still mine.

Likely follow-ups

- What would you put in an audit policy and where would the logs go?
- How do you secure the kubelet API?

The API server is the front door to the cluster; hardening it and the control plane is foundational CKS content.

API SERVER HARDENING

```
- Strong authn (OIDC / cloud IAM), no anonymous auth
- Least-privilege RBAC; no wildcard cluster-admin handouts
- Enable audit logging (who did what, when)
- Restrict network exposure: private control plane / authorized networks
- Disable insecure port; TLS everywhere; rotate certs
```

ETCD — THE CROWN JEWELS

```
- Encrypt etcd at rest (KMS); etcd holds ALL cluster data + secrets
- mTLS between API server and etcd; restrict etcd network access
- Regular encrypted etcd backups (and test restore)
```

NODE + KUBELET

Kubelet authn/authz on (no anonymous), rotate kubelet certs, restrict kubelet API, run CIS Benchmark checks (`kube-bench`), keep nodes patched.

Interview tip: Name **kube-bench** (CIS Benchmark) and **audit logging** explicitly. Stress that etcd is the highest-value target — its compromise = all secrets — so encryption at rest + mTLS + restricted access + tested backups are non-negotiable. On managed clusters (GKE/EKS) the provider handles much of this; know what's yours vs theirs.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Operating etcd clusters
- Kubernetes: kubelet

Q79 **What runtime security threats exist after a pod is running, and how do Falco/Tetragon help?** Security

🎙 **Spoken answer · ~75 s**

Build-time scanning cannot catch what happens after a container is running: an attacker exploiting an application vulnerability, a stolen credential, or a zero-day. Runtime threats include a reverse shell started in a container, unexpected processes such as a crypto miner or recon tools, reading sensitive files like etc shadow, writing to system directories, unexpected outbound connections to command-and-control servers, and privilege escalation or escape attempts. Falco watches system calls using eBPF or a kernel module and matches them against rules, for example shell spawned in a container, write below etc, or outbound connection to an unusual address, then raises alerts that I route to the SIEM or incident channel. Tetragon, from Cilium, also uses eBPF, but can go beyond detecting: it can enforce, for instance killing a process or blocking an action in the kernel, with low overhead and Kubernetes-aware context. The value is detection and fast response, such as isolating the pod with a network policy. The challenge is tuning rules to reduce noise, so I start with the default ruleset, review alerts, and tailor for my workloads.

Likely follow-ups

- How do you reduce false positives from Falco?
- What's the difference between detection and enforcement at runtime?

Build-time scanning misses runtime threats — a compromised running container behaving maliciously. Runtime security detects and responds to that.

RUNTIME THREATS

```
- Reverse shell spawned in a container
- Unexpected process execution (crypto miner, recon tools)
- Sensitive file reads (/etc/shadow), writes to system dirs
- Unexpected outbound network connections (C2, exfiltration)
- Privilege escalation attempts, container escape behavior
```

FALCO / TETRAGON

```
Falco    : rules over syscalls (kernel module/eBPF) -> alert on suspicious behavior
           e.g. "shell spawned in a container", "write below /etc"
Tetragon : eBPF-based observability + ENFORCEMENT (can kill offending processes)
```

These complement, not replace, prevention: even a hardened container can be exploited via an app vuln, and runtime detection catches the post-exploitation behavior.

Interview tip: Frame the layers: prevent (scan, sign, harden) + detect/respond (Falco/Tetragon at runtime). eBPF is the modern data source — low overhead, kernel-level visibility. Tetragon's ability to enforce (kill the process) not just alert is a notable distinction to mention.

certs.eknathalabs.com

📚 **Official docs**

- Falco documentation

Q80 **How do you implement multi-tenancy securely in a shared cluster? What are the isolation boundaries?** Security

🎙 **Spoken answer · ~71 s**

Securing a multi-tenant cluster means stacking isolation layers, and being honest that it's weaker than separate clusters. Per tenant I use a namespace as the basic boundary, with RBAC limiting the tenant's service accounts and users to their own namespaces. ResourceQuota caps CPU, memory, and object counts to stop noisy neighbours, and LimitRange sets default and maximum requests per pod. NetworkPolicies default to deny cross-namespace traffic. Pod Security Admission, or policy engines, enforce restricted security settings per namespace, and policies restrict what tenants may create, such as no hostPath, no privileged pods, only approved registries. For hostile or untrusted tenants, I add node-level isolation with dedicated node pools using taints and tolerations, sandboxed runtimes such as gVisor or Kata via RuntimeClass, and separate secrets management and identity. The weak points to acknowledge are the shared control plane and shared kernel: a noisy tenant can load the API server, and a kernel escape affects everyone. For strong guarantees, such as regulated or unrelated customers, I use separate clusters or virtual clusters.

Likely follow-ups

- When is a namespace per tenant not enough?
- How do virtual clusters compare to separate clusters?

Multi-tenancy on one cluster trades cost-efficiency for weaker isolation than separate clusters. You stack namespace-level controls to approximate isolation.

ISOLATION LAYERS PER TENANT

```
Namespace        : the basic boundary (one+ per tenant)
RBAC             : tenant SAs scoped to their namespaces only
ResourceQuota    : cap CPU/mem/object counts per namespace (noisy-neighbor)
LimitRange       : default + max requests/limits per pod
NetworkPolicy    : default-deny cross-namespace traffic
PSA / policy     : enforce restricted pod security per namespace
Node isolation   : taints + RuntimeClass (gVisor/Kata) for hostile tenants
```

THE HARD TRUTH ABOUT THE BOUNDARY

Namespaces are NOT a hard security boundary — they share the same kernel, control plane, and (by default) nodes. A kernel exploit or control-plane bug crosses namespaces.

When soft multi-tenancy isn't enough: For untrusted/hostile tenants, use separate clusters, dedicated node pools per tenant (taints + node affinity), and sandboxed runtimes (gVisor/Kata). "Hard" multi-tenancy of untrusted code on shared nodes is genuinely difficult.

Interview tip: Distinguish soft multi-tenancy (trusted internal teams — namespace + quota + RBAC + NetworkPolicy suffices) from hard multi-tenancy (untrusted — separate clusters or VM-isolated runtimes). Stating that a namespace is not a security boundary is the senior insight.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Multi-tenancy

Q81 **Explain ServiceAccount tokens, token projection, and why auto-mounting can be a risk.** Security

🎙 **Spoken answer · ~70 s**

Every pod runs as a ServiceAccount, and its token authenticates to the API server with that account's RBAC permissions. By default, the token is automatically mounted into every pod under var run secrets. If the pod doesn't need to call the Kubernetes API, which is true for most application pods, that mounted token is an unnecessary credential. An attacker who gains code execution in the pod can read it and use it against the API server, so it's an easy lateral movement path, especially if the service account is over-privileged. Mitigations: set automountServiceAccountToken to false on the service account or the pod when the API isn't needed; give workloads their own service accounts with minimal RBAC instead of default; and use bound, projected tokens. Since Kubernetes one point twenty-two, tokens are short-lived, audience-scoped, bound to the pod, and rotated automatically, unlike the old non-expiring token Secrets, so a stolen token is far less valuable. Projected tokens are also what powers workload identity to cloud providers, with a specific audience.

Likely follow-ups

- What was wrong with the old long-lived service account token Secrets?
- How does token projection enable workload identity?

Every pod gets a ServiceAccount; its token authenticates to the API server. Mismanaged, it's an easy lateral-movement path.

THE RISK OF AUTO-MOUNT

```
# By default the SA token is mounted into every pod at:
/var/run/secrets/kubernetes.io/serviceaccount/token
# If the pod doesn't need the API, that's an unnecessary credential an
# attacker can steal to talk to the API server with the SA's RBAC.
```

MITIGATIONS

```
automountServiceAccountToken: false   # on the SA or the pod, if API access unneeded
# Use bound, projected tokens (default in 1.22+): short-lived, audience-scoped,
# auto-rotated -- vs the old never-expiring Secret-based tokens.
```

BOUND SERVICE ACCOUNT TOKENS

Modern tokens are time-bound, audience-bound, and tied to the pod's lifetime (projected volume), so a leaked token expires and can't be replayed elsewhere — unlike legacy long-lived Secret tokens.

Interview tip: Two crisp points: (1) disable token auto-mount for pods that don't call the API (least privilege), and (2) bound/projected tokens (1.22+) replaced the old non-expiring Secret tokens, shrinking the blast radius of a leak. Pair with tight RBAC on the SA so even a stolen token does little.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Service accounts

Q82 **A CVE drops for your base image fleet-wide. Walk through your response process.** Security

🎙 **Spoken answer · ~73 s**

I'd run it as a short incident process. First, triage and assess exposure. Is the vulnerability exploitable in our context: is the vulnerable component reachable, is there a known exploit, are the workloads internet-facing or privileged? I use SBOMs and registry scans, with Trivy across all images, to inventory exactly which images and running workloads are affected, and prioritise by severity and exposure. Second, contain if it's actively exploited: tighten NetworkPolicies, scale down or isolate exposed services, add an admission rule blocking the vulnerable image from new scheduling, and turn up runtime detection with Falco for signs of exploitation. Third, remediate: update the base image, rebuild and re-scan the affected images through CI, and roll them out by priority using the normal pipeline with canary or progressive delivery, which is much easier if base images and build templates are centralised. Fourth, verify: re-scan deployed workloads and confirm none still runs the vulnerable image. Finally, a short retrospective: how quickly did we find affected images, and how do we automate rebuilds and keep an up-to-date inventory?

Likely follow-ups

- How do you decide whether a CVE is actually exploitable in your environment?
- How do you make base image updates automatic?

This tests incident process for supply-chain/runtime vulnerabilities — triage, assess, remediate, verify.

1. TRIAGE / ASSESS EXPOSURE

```
- Is it exploitable in our context? (network-reachable? privileged? present in the path?)
- Inventory affected images via SBOMs / registry scan (Trivy across all images)
- Severity + exploit availability -> prioritize
```

2. CONTAIN (IF ACTIVELY EXPLOITED)

```
- NetworkPolicy to limit blast radius, or scale down exposed services
- Runtime detection (Falco) for exploitation signs
- Admission rule to block the vulnerable image from new scheduling
```

3. REMEDIATE + ROLL OUT

```
- Patch base image, rebuild, rescan to confirm fixed
- Roll out via normal CD (canary -> fleet), backward-compatible
- For urgent: prioritized rollout of the most-exposed services first
```

4. VERIFY + PREVENT RECURRENCE

Re-scan the fleet to confirm zero affected images remain; add/enforce the CVE gate in CI; consider digest pinning and signed images.

certs.eknathalabs.com

Interview tip: Lead with **exposure assessment** — not every critical CVE is reachable in your setup, and triage prevents a needless fire drill. SBOMs make "which images are affected" answerable in minutes instead of days; mention them as the reason you can respond fast.

📚 **Official docs**

- Argo CD: ApplicationSet
- Trivy

Q83 **What's the difference between authentication, authorization, and admission in the API request flow?** Security

🎙 **Spoken answer · ~70 s**

Every API request goes through three gates in order. Authentication asks who you are: client certificates, OIDC tokens, service account tokens, or cloud IAM. Authorisation asks whether you are allowed to do this action: RBAC mostly, plus the node authoriser and webhooks. Admission asks whether this particular object should be accepted or changed: first mutating admission, then schema validation, then validating admission such as Pod Security Admission and policy engines, and only then is the object written to etcd. Knowing the order helps with diagnosis from the error message. Unauthorized, a 401, means authentication failed, like an expired token or certificate. Forbidden, a 403, saying a user cannot list pods means authorisation failed, so I check RBAC with auth can-i. An error saying a request was denied by a webhook or policy, or that a pod violates a security standard, is admission. It also tells me where to put controls: identity in authentication, who-can-do-what in RBAC, and content rules, like no privileged containers or required labels, in admission.

Likely follow-ups

- Which HTTP status codes map to authentication and authorisation failures?
- Why do mutating webhooks run before validating ones?

Every API request passes through three sequential gates. Knowing the order clarifies where a given control lives.

THE REQUEST PIPELINE

```
Request -> [1 AuthN] -> [2 AuthZ] -> [3 Admission] -> persisted to etcd

1. Authentication : WHO are you? (certs, OIDC tokens, SA tokens, cloud IAM)
2. Authorization  : are you ALLOWED? (RBAC / ABAC / Webhook / Node authorizer)
3. Admission      : SHOULD this specific object be allowed/mutated?
                    (mutating webhooks, then validating webhooks/PSA/policy)
```

WHERE CONTROLS LIVE

```
"User X can't authenticate"        -> AuthN (bad/expired cert or token)
"User X forbidden to list pods"     -> AuthZ (RBAC missing the verb)
"Pod rejected: must run non-root"   -> Admission (PSA/policy engine)
"Sidecar auto-injected"             -> Admission (mutating webhook)
```

Interview tip: The crisp framing: authN = identity, authZ = permission, admission = policy on the object itself. Mutating admission runs before validating admission. This mental model lets you instantly place where any security control or failure belongs — a clean

certs.eknathalabs.com

way to structure a security answer.

📚 **Official docs**

- Kubernetes: Admission controllers

Q84 **How do you scan and harden a cluster against the CIS Benchmark? What does kube-bench check?** Security

🎙 **Spoken answer · ~70 s**

The CIS Kubernetes Benchmark is a widely used hardening checklist, and kube-bench automates checking a cluster against it. It inspects the control plane component flags for the API server, controller manager, scheduler, and etcd; file permissions and ownership for configuration files and certificates; kubelet settings such as anonymous auth being off, authorisation mode Webhook, and certificate rotation; etcd encryption and TLS; and whether audit logging is enabled. I run it as a job on the nodes, targeting control plane, node, etcd, and policies, and it outputs pass, fail, or warn for each control with remediation guidance. I then work through failures by risk, fix the configuration, and re-run, ideally in CI or on a schedule to catch drift. On managed services such as GKE, EKS, and AKS the provider secures the control plane, so I focus on the worker node checks and the policy items, and use provider-specific benchmark variants. kube-bench covers configuration, not workloads, so I pair it with image scanning, policy enforcement, and runtime detection.

Likely follow-ups

- Which kube-bench results matter on a managed cluster?
- How would you track CIS compliance over time?

The CIS Kubernetes Benchmark is the standard hardening checklist; `kube-bench` automates checking a cluster against it.

WHAT KUBE-BENCH CHECKS

```
- Control plane component flags (api-server, controller-manager, scheduler, etcd)
- File permissions/ownership of config files and certs
- kubelet configuration (anonymous auth off, authz mode, cert rotation)
- etcd encryption + TLS settings
- Logging/auditing enabled
```

RUNNING IT

```
kube-bench run --targets master,node,etcd,policies
# outputs PASS/FAIL/WARN per CIS control with remediation steps
```

MANAGED VS SELF-MANAGED

On GKE/EKS/AKS the provider hardens the control plane (you can't set those flags), so node/policy checks are your responsibility. On self-managed (kubeadm), you own all of it.

Interview tip: Pair **kube-bench** (CIS config hardening) with **kube-hunter** (active pen-test of the cluster) and Trivy (workload/image scanning) for a full posture check. Knowing that managed clusters shift control-plane checks to the provider — and what remains yours (nodes, RBAC, policies, workloads) — shows real operational understanding.

Certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Security checklist
- kube-bench (CIS Kubernetes benchmark)
- CIS Benchmarks

Q85 **Explain seccomp, AppArmor, and SELinux in the context of containers. When use each?** Security

🎙 **Spoken answer · ~73 s**

These are Linux kernel security mechanisms that restrict what a container can do, which adds defence in depth below the Kubernetes layer. Seccomp filters system calls: a process can only invoke the syscalls the profile allows. The RuntimeDefault profile blocks dozens of rarely needed and dangerous syscalls and is a cheap, widely compatible win; custom profiles can be tighter. AppArmor is a path-based mandatory access control system: a profile restricts which files a process can access and which capabilities and network operations it can use. It's the default on Ubuntu and Debian. SELinux is label-based mandatory access control: policy is enforced using security labels on processes and files, the default on Red Hat family systems, and it's very strong but more complex. In Kubernetes I set seccompProfile in the securityContext, and AppArmor and SELinux through their respective fields and options. Which to use depends largely on the host operating system, since a node uses AppArmor or SELinux but not both. My baseline is RuntimeDefault seccomp everywhere, plus the OS-native MAC profile for sensitive workloads.

Likely follow-ups

- Why is RuntimeDefault seccomp a good default?
- Can a node run both AppArmor and SELinux?

These are Linux kernel security modules that constrain what a container can do — defense in depth below the K8s layer.

THE THREE

```
seccomp   : filters which SYSCALLS a process may make (allowlist/denylist)
AppArmor  : path-based MAC -> restrict file access, capabilities, network per profile
SELinux   : label-based MAC -> enforce policy via security labels on processes/files
```

IN KUBERNETES

```
securityContext:
  seccompProfile: { type: RuntimeDefault }   # blocks ~dangerous syscalls
# AppArmor via annotation/profile; SELinux via seLinuxOptions
```

`RuntimeDefault` seccomp blocks dozens of rarely-needed, dangerous syscalls with near-zero effort — the easiest high-value hardening. Custom profiles tighten further but need testing (a missing syscall crashes the app).

WHEN EACH

seccomp: always enable RuntimeDefault as a baseline. AppArmor: Debian/Ubuntu nodes, path-based file/cap restriction. SELinux: RHEL/CoreOS nodes, label-based enforcement (default on OpenShift).

Interview tip: The pragmatic answer: turn on `seccompProfile: RuntimeDefault` everywhere (huge win, trivial cost), then layer AppArmor or SELinux depending on your node OS. These operate at the kernel boundary, complementing

certs.eknathalabs.com

RBAC/NetworkPolicy which operate at the K8s API/network layers.

📚 **Official docs**

- Kubernetes: Seccomp tutorial
- Kubernetes: AppArmor tutorial
- RHEL 9 documentation

Q86 **How would you detect and respond to a compromised pod that's mining crypto or beaconing out?** Security

🎙 **Spoken answer · ~72 s**

I'd treat it as a security incident and follow detect, contain, preserve, eradicate, and recover. Detection signals are CPU pinned with no legitimate workload change, unexpected outbound connections to unknown IPs or mining pools, Falco alerts for unexpected processes or a shell in a container, and files or processes that aren't in the image. Containment first: apply a deny-all NetworkPolicy to that pod to cut command-and-control and exfiltration without destroying evidence. Preserve evidence before killing anything: capture logs, process lists, network connections, and a snapshot of the container filesystem or the node disk. If I suspect an escape, cordon the node and check other pods on it. Then investigate how it got in: a vulnerable application, an exposed dashboard, a leaked credential, or a poisoned image. Eradicate: delete and redeploy from a known-good image, rotate any secrets or tokens the pod could access, and review the service account's permissions and cloud credentials. Then fix the root cause, tighten policies, and add detection so it can't happen silently again, and write a postmortem.

Likely follow-ups

- Why isolate with a network policy instead of deleting the pod immediately?
- What credentials would you rotate after this?

This blends runtime detection, network controls, and incident response — a realistic security scenario.

DETECTION SIGNALS

```
- CPU pegged at 100% with no legitimate workload change (mining)
- Unexpected outbound connections to unknown IPs/pools (beaconing/C2)
- Falco alerts: unexpected process spawn, shell in container, outbound to mining pool
- New processes not in the image; writes to unexpected paths
```

RESPONSE

```
1. Isolate: apply a deny-all NetworkPolicy to the pod (cut C2/exfil)
2. Preserve: snapshot for forensics BEFORE killing (capture evidence)
3. Cordon the node if escape suspected; check for lateral movement
4. Terminate the pod; rotate any credentials it could access (SA token, secrets)
5. Root cause: which CVE / misconfig / leaked cred let them in?
6. Remediate the entry vector; redeploy clean; verify
```

PREVENTION GOING FORWARD

Egress NetworkPolicies (default-deny egress stops beaconing), runtime detection (Falco/Tetragon), least-privilege SA + tight RBAC so a compromised pod can't pivot, hardened securityContext to limit escape.

certs.eknathalabs.com

Interview tip: Order matters: **isolate (network) and preserve evidence before you kill the pod** — terminating first destroys forensics and the attacker may have persistence elsewhere. Then rotate credentials the pod could reach and hunt for lateral movement. Egress controls are the prevention headline: default-deny egress would have blocked the beaconing outright.

📚 **Official docs**

- Kubernetes: Network Policies

Q87 **How do you monitor a Kubernetes cluster? What are the layers and the key metrics at each?** Observability

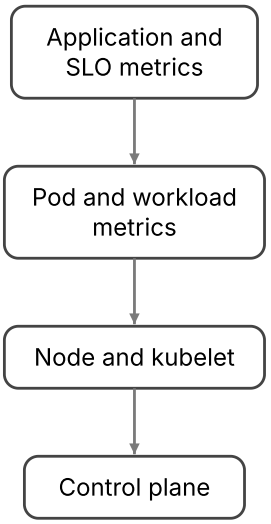
🎙 **Spoken answer · ~69 s**

I monitor in layers, because each has its own signals. For the control plane: API server request latency and error rate, etcd health and disk latency, and the scheduler and controller queues. For nodes: CPU, memory, disk and network usage, node conditions like pressure and Ready, and kubelet health. For Kubernetes workloads: pod restarts, OOMKills, Pending pods, readiness failures, and HPA behaviour. For applications: the golden signals of latency, traffic, errors, and saturation, or the RED method. The data sources are node-exporter for OS metrics, kube-state-metrics for object state such as desired versus ready replicas and PVC status, cAdvisor and the kubelet for container metrics, and the applications' own instrumentation, all scraped by Prometheus and shown in Grafana. Alerting matters more than dashboards: page on symptoms that affect users, like error rate and latency against SLOs, and send cause-level signals, such as restarts and pressure, to lower-urgency channels. I'd also watch the monitoring system itself and control cardinality so costs and query performance stay sane.

Likely follow-ups

- Which alerts would page someone and which would only ticket?
- What does kube-state-metrics provide that node-exporter doesn't?

🗺 **Monitoring layers**



Cluster monitoring spans several layers; each has signals you must watch. Prometheus + Grafana is the de-facto stack.

THE LAYERS

certs.eknathalabs.com

```
Cluster/control plane : apiserver latency, etcd health, scheduler/controller queues
Nodes                 : CPU/mem/disk/network, node conditions, kubelet health
Workloads             : pod restarts, OOMKills, readiness, HPA behavior
Application           : RED (rate, errors, duration) / golden signals
Kubernetes objects    : kube-state-metrics (deployment replicas, PVC status...)
```

WHERE METRICS COME FROM

```
node-exporter        : node-level OS metrics
kube-state-metrics   : object state (desired vs ready replicas, pod phase)
cAdvisor (in kubelet): per-container resource usage
metrics-server       : live CPU/mem for HPA + kubectl top
app /metrics         : your service's RED metrics (instrumented)
```

KEY ALERTS

Pods crash-looping, nodes NotReady, PVCs near full, HPA at max replicas, apiserver/etcd latency high, certificate expiry, persistent OOMKills.

Interview tip: Structure the answer by layer and cite the exporters by name (node-exporter, kube-state-metrics, cAdvisor) — that specificity shows hands-on monitoring experience. Tie app monitoring to RED/golden signals and mention kube-prometheus-stack as the batteries-included Helm chart.

📚 **Official docs**

- Prometheus: Instrumentation best practices

Q88 **Explain how Prometheus discovers and scrapes targets in Kubernetes. What is ServiceMonitor?** Observability

🎙 **Spoken answer · ~70 s**

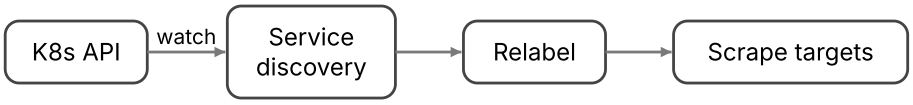
Prometheus pulls metrics, and in Kubernetes it finds targets dynamically using service discovery. It watches the Kubernetes API through kubernetes_sd_configs, with roles such as pod, service, endpoints, node, and ingress. Relabeling rules then decide which discovered targets to scrape and rewrite their labels, for instance keeping only pods with an annotation, or attaching namespace and pod labels. Prometheus then scrapes the metrics path on each target at the configured interval, and as pods come and go the target list updates automatically. With the Prometheus Operator, you don't hand-write that configuration. A ServiceMonitor is a custom resource that selects Services by label and describes which port, path, and interval to scrape. A PodMonitor does the same directly for pods. The operator translates these into Prometheus configuration and reloads it, which lets each application team own the monitoring definition for its own service, alongside its deployment manifests. Common problems are a selector that doesn't match, a port name mismatch, or the Prometheus instance not selecting the ServiceMonitor's namespace or labels.

Likely follow-ups

- Why might a ServiceMonitor exist but Prometheus show no target?
- What does relabeling do in a scrape config?

certs.eknathalabs.com

🗺 **Prometheus discovery**



Prometheus uses Kubernetes service discovery to find scrape targets dynamically as pods come and go.

SERVICE DISCOVERY + SCRAPING

```
# Prometheus watches the K8s API for pods/services/endpoints
# kubernetes_sd_configs roles: pod, service, endpoints, node, ingress
# Relabeling picks which targets to scrape and rewrites labels
# Then it pulls /metrics from each target on an interval
```

PROMETHEUS OPERATOR: SERVICEMONITOR / PODMONITOR

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
spec:
  selector: { matchLabels: { app: web } }
  endpoints:
  - { port: metrics, interval: 30s, path: /metrics }
```

The Operator turns ServiceMonitor/PodMonitor CRDs into Prometheus scrape config automatically — you declare "scrape services with this label" instead of editing prometheus.yml.

Scale gotcha: Pull model + high-cardinality labels (per-pod, per-request-id) explode memory and TSDB size. Keep label cardinality bounded. For huge fleets use sharding/federation or a remote-write backend (Thanos, Mimir, Cortex) for long-term storage and global query.

Interview tip: Explain relabeling as the filter mechanism, ServiceMonitor as the declarative Operator-native way to register scrape targets, and cardinality as the #1 thing that kills Prometheus — mirroring the TSDB cardinality point from system design.

📚 **Official docs**

- kubernetes-mixin (monitoring mixins)
- Prometheus Operator
- Prometheus: Overview

certs.eknathalabs.com

Q89

What are the three pillars of observability and how do you implement them in Kubernetes?

Observability

🎙 Spoken answer · ~73 s

Metrics, logs, and traces answer different questions, and I need all three to debug a distributed system. Metrics tell me what is happening and how much: aggregated numbers that are cheap to store and ideal for dashboards and alerting. Logs tell me what happened in detail: discrete events with context. Traces tell me where time is spent, following a single request across services. In Kubernetes, for metrics I run Prometheus scraping each service's metrics endpoint, Grafana for dashboards, and Alertmanager for notifications, usually through the kube-prometheus-stack. For logs, applications write to stdout, and a DaemonSet collector such as Fluent Bit or Promtail tails the container log files and ships them to Loki, Elasticsearch, or a cloud logging service. For traces, applications are instrumented with OpenTelemetry, which sends spans to a collector and then to Tempo or Jaeger. The real value comes from correlation: consistent labels, a trace ID in every log line, and exemplars linking metrics to traces, so I can go from an alert to the slow trace to the exact log lines.

Likely follow-ups

- How do you correlate a metric spike with a trace and its logs?
- What is OpenTelemetry's role versus a backend like Jaeger?

Metrics, logs, and traces together let you understand and debug a distributed system. Each answers a different question.

THE THREE PILLARS

```
Metrics : what & how much (aggregates, alerting)  -> Prometheus
Logs    : what happened (discrete events, detail)  -> Loki / ELK / Cloud Logging
Traces  : where & why slow (request path)          -> Jaeger / Tempo / OTel
```

IMPLEMENTATION IN K8S

```
Metrics : Prometheus scrapes /metrics; Grafana dashboards; Alertmanager
Logs    : DaemonSet collector (Fluent Bit/Promtail) tails container stdout
          -> ships to Loki/Elasticsearch (don't write logs to pod disk)
Traces  : OpenTelemetry SDK/collector; propagate trace context across services
```

THE GLUE: CORRELATION

Propagate a trace/correlation ID through all services and include it in logs + spans, so you can pivot metric alert -> trace (which hop) -> logs (root cause).

Interview tip: Stress the workflow: a metric alert tells you something's wrong, a trace localizes the slow service, logs give the why — and correlation IDs tie them together. Name **OpenTelemetry** as the vendor-neutral instrumentation standard and that container logs should go to stdout for a DaemonSet collector, never to files on the pod.

📚 Official docs

- OpenTelemetry: Signals (traces, metrics, logs)

certs.eknathalabs.com

Q90 **How do you do centralized logging in Kubernetes? Why not write logs to files in the pod?** Observability

🎙 Spoken answer · ~75 s

Containers are ephemeral: when a pod is deleted, rescheduled, or evicted, anything written to its local filesystem disappears. So logs must leave the pod immediately. The standard pattern is: the application writes to stdout and stderr; the container runtime writes that to log files on the node under var log containers; a DaemonSet collector such as Fluent Bit, Fluentd, or Promtail tails those files, enriches them with Kubernetes metadata like pod, namespace, and labels, and ships them to a central store such as Loki, Elasticsearch, or a cloud logging service, where I query them in Grafana or Kibana. Writing to files inside the pod is a bad idea for several reasons: the logs vanish with the pod, files consume the container's writable layer and can fill the node's disk, you need extra tooling to rotate them, and collectors can't find them in a uniform way. If an application insists on files, a sidecar can tail the file to stdout. I also use structured JSON logs, set retention and log levels deliberately, and watch the volume, since logging cost grows quickly.

Likely follow-ups

- What happens to logs when a node dies?
- How do you control logging cost at scale?

🗺 Central logging



Containers are ephemeral, so logs must leave the pod immediately. The standard pattern is stdout -> node collector -> central store.

THE PATTERN

```
App -> stdout/stderr
  -> container runtime writes to /var/log/containers/*.log on the node
  -> DaemonSet collector (Fluent Bit / Fluentd / Promtail) tails those
  -> ships to Loki / Elasticsearch / Cloud Logging
  -> Grafana / Kibana to query
```

WHY NOT LOG TO A FILE IN THE POD

```
- Pod deleted -> logs gone (ephemeral filesystem)
- Files count against the container's storage/memory (emptyDir)
- No central search; can't correlate across pods
- Log rotation inside the container is fragile
```

ENRICHMENT

The collector adds K8s metadata (namespace, pod, labels, node) so you can filter logs by workload. Use structured (JSON) logs for queryability.

Interview tip: "Log to stdout, let a DaemonSet ship it" is the canonical answer. Mention Loki (label-indexed, cheaper, pairs with Grafana) vs the EFK/ELK stack (full-text, heavier). The key principle: treat logs as a stream that leaves the pod instantly, never as

certs.eknathalabs.com

files on ephemeral storage.

📚 **Official docs**

- Kubernetes: Logging architecture

Q91 **How do you define SLOs and alert on them for a Kubernetes service? Explain burn-rate alerting.** Observability

🎙 **Spoken answer · ~78 s**

An SLO turns reliability into a measurable target. I pick an SLI, the signal that reflects user experience, for example the proportion of requests that succeed and complete under three hundred milliseconds. The SLO is the target, say ninety-nine point nine percent over thirty days, and the error budget is what's left, so zero point one percent of requests may fail. Alerting on a static threshold like error rate above one percent is either noisy or slow. Burn-rate alerting measures how fast the budget is being consumed. A burn rate of one means exactly using the budget over the window. I use multiple windows: a fast alert, for example a burn rate around fourteen over one hour with a short confirmation window, pages immediately for severe outages; a slower alert, like burn rate six over six hours, catches smaller sustained problems; and a slow burn becomes a ticket. Combining a long and a short window prevents paging after the problem is already over. In Kubernetes I compute the SLI from ingress or service mesh metrics in Prometheus, using recording rules, and tools like Sloth generate the rules.

Likely follow-ups

- Why combine a long and a short window in burn-rate alerts?
- How do you choose an SLI for a service?

SLOs turn reliability into a measurable target; burn-rate alerting fires on the *rate* of error-budget consumption, not raw thresholds.

SLI / SLO / ERROR BUDGET

```
SLI          : the measured signal (e.g. % of requests < 300ms and 2xx/3xx)
SLO          : the target (e.g. 99.9% success over 30 days)
Error budget : 100% - SLO = 0.1% of requests may fail in the window
```

WHY BURN-RATE BEATS STATIC THRESHOLDS

Alerting "error rate > 1%" is noisy (fires on harmless blips) or slow. Burn-rate measures how fast you're spending the error budget: a fast burn (budget gone in hours) pages immediately; a slow burn (budget gone in days) is a ticket.

```
# Multi-window, multi-burn-rate (Google SRE):
# fast: 14.4x burn over 1h  -> page (budget exhausts in ~2 days at this rate)
# slow: 3x burn over 6h     -> ticket
# Requires BOTH a short and long window to fire (avoids flapping)
```

Interview tip: Burn-rate, multi-window alerting (from the Google SRE book) is the senior answer — it ties alert urgency to budget impact, cutting noise while catching real fast-burn incidents quickly. Implement SLIs as Prometheus recording rules and alerts in Alertmanager.

certs.eknathalabs.com

📚 **Official docs**

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Alerting on SLOs
- Prometheus: Alerting rules

Q92 **Walk through what happens when a node goes NotReady. What's the timeline to pod rescheduling?** Cluster ops

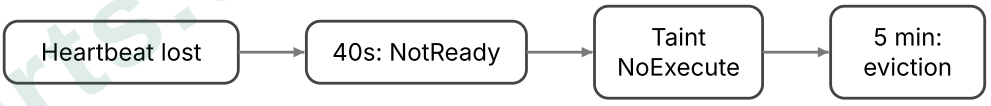
🎙 **Spoken answer · ~72 s**

The timeline is deliberately conservative to avoid flapping. The kubelet posts heartbeats through node leases. When they stop, after about forty seconds, the node controller marks the node NotReady or Unknown. The node also gets taints, not-ready and unreachable, with NoExecute. Pods tolerate those taints for three hundred seconds by default, the tolerationSeconds, so about five minutes later the pods are evicted. Then controllers such as the ReplicaSet notice the missing pods and create replacements on other nodes, which still need to be scheduled, pull images, and pass readiness. So realistically it takes five to six minutes before traffic recovers fully, though a Service stops routing to those pods earlier because their endpoints go not-ready. StatefulSets and ReadWriteOnce volumes add more delay, since the disk must be detached from the dead node first. It's slow on purpose: a brief network blip shouldn't trigger a mass reschedule. To speed it up, I can lower tolerationSeconds for latency-critical workloads, run enough replicas across zones to absorb the loss meanwhile, and use health checks and autoscaling.

Likely follow-ups

- How would you shorten the failover time for a critical service?
- Why does a Service stop routing before the pod is evicted?

🗺 **NotReady timeline**



Node failure recovery has a deliberately conservative timeline to avoid flapping — knowing it explains why recovery "takes minutes."

THE TIMELINE

```
1. kubelet stops posting heartbeats (node lease)
2. After ~40s (node-monitor-grace-period) node -> NotReady
3. Node controller waits ~5min (pod eviction timeout) before evicting
4. Pods marked for deletion; controllers (Deployment/RS) create replacements elsewhere
5. For StatefulSet + RWO volume: must detach from dead node first (more delay)
```

WHY SO SLOW ON PURPOSE

Aggressive eviction on a brief network blip would cause needless churn and double-scheduling. The grace periods trade recovery speed for stability.

SPEEDING IT UP WHERE NEEDED

certs.eknathalabs.com

```
- Tune node-monitor / eviction timeouts (carefully)
- Run enough replicas across nodes/zones so one node loss = no outage
- For stateful: app-level replication/failover beats waiting for volume reattach
```

Interview tip: The honest takeaway: K8s self-heals node failure in *minutes*, not seconds, because of intentional grace periods — so availability comes from running redundant replicas across failure domains, not from fast rescheduling. Stateful workloads are slower still due to volume detach; that's why operators handle failover at the app layer.

📚 **Official docs**

- Kubernetes: Node-pressure eviction

Q93

How do you upgrade a Kubernetes cluster safely with zero downtime? Control plane and nodes.

Cluster ops

🎙 **Spoken answer · ~73 s**

I upgrade in a fixed order and by rules. Kubernetes supports upgrading one minor version at a time, never skipping, so one point twenty-eight to twenty-nine to thirty. The control plane goes first, and the API server must be at least as new as the kubelets; kubelets can lag by a few minor versions, which allows nodes to follow afterwards. Before starting, I read the release notes for deprecated and removed APIs, run a tool such as pluto or kubent to find manifests using them, check addon and CRD compatibility, and take an etcd backup. Then the nodes: I do a rolling upgrade, cordoning and draining each node or using surge nodes, which honours PodDisruptionBudgets so replicas stay available. Zero downtime depends on the workloads having multiple replicas spread across nodes, accurate readiness probes, graceful shutdown, and sensible PDBs. On managed clusters I use node pool surge upgrades with maxSurge and maxUnavailable. I upgrade a non-production cluster first, watch metrics throughout, and have a rollback plan, remembering that control plane downgrades are generally not supported.

Likely follow-ups

- How do you find deprecated API usage before an upgrade?
- What do surge upgrades on a managed node pool do?

Cluster upgrades follow version-skew rules and a control-plane-first, then-nodes order, draining nodes gracefully.

ORDER + SKEW RULES

```
1. Upgrade control plane first (api-server, then controller-manager/scheduler)
2. Then nodes (kubelet) -- kubelet may be up to 3 minor versions behind apiserver
3. One minor version at a time (no skipping, e.g. 1.28 -> 1.29 -> 1.30)
4. Upgrade addons/CRDs as required by the new version
```

NODE UPGRADE (ROLLING, DRAINED)

```
kubectl cordon <node>       # stop new pods
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data  # evict (respects PDBs)
# upgrade kubelet / replace node, then:
kubectl uncordon <node>
```

certs.eknathalabs.com

Managed clusters (GKE/EKS) automate this with surge upgrades — spin up new-version nodes, drain old ones, respecting PDBs.

Pre-flight: Check deprecated/removed APIs in the target version (e.g. `kubectl deprecations` / pluto) — a removed API version breaks manifests/controllers. Verify PDBs allow draining (no single-replica deadlocks). Back up etcd first.

Interview tip: The headline rules: control plane before nodes, one minor at a time, PDBs make drains safe, and check for removed APIs before upgrading. Zero downtime depends on the same trio as deploys — multiple replicas, readiness probes, PDBs.

📚 **Official docs**

- Kubernetes: Upgrading kubeadm clusters

Q94

What is etcd's role, and how do you back up and restore it? What happens if etcd is unhealthy?

Cluster ops

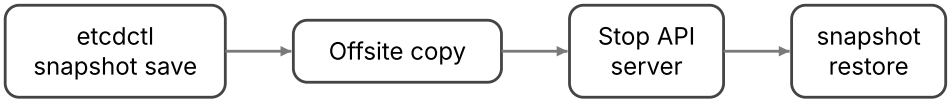
🎙 **Spoken answer · ~74 s**

etcd is a distributed key-value store using Raft consensus, and it holds all cluster state: every object, secret, and configuration. The API server is basically a typed front end over it, so etcd's health gates the whole control plane. For backup, I take a snapshot with etcdctl snapshot save, supplying the endpoint and client certificates, on a schedule, and store it encrypted and off-cluster. To restore, I stop the API servers and etcd, restore the snapshot into a new data directory with etcdctl snapshot restore, point etcd at it, restart, and verify. In a multi-member cluster, every member is restored from the same snapshot with new cluster settings. If etcd is unhealthy, the effects depend on quorum. With a majority of members, the cluster still works. Without quorum, the control plane goes read-only or unavailable: you can't create or change anything, but workloads already running continue. Slow disk is the commonest performance problem, so etcd needs fast SSDs, and I monitor fsync latency and database size, and defragment when needed. Managed Kubernetes handles all this for you.

Likely follow-ups

- What happens to running pods when etcd loses quorum?
- Why is disk latency so important for etcd?

🗺 **etcd backup and restore**



etcd is the cluster's single source of truth — every object's desired state. Its health gates the entire control plane.

ROLE

A distributed key-value store (Raft consensus) holding all API objects, secrets, and config. The API server is essentially a typed front-end over etcd.

BACKUP

certs.eknathalabs.com

```
ETCDCTL_API=3 etcdctl snapshot save snapshot.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=... --cert=... --key=...
# store the snapshot encrypted, off-cluster; schedule regularly
```

RESTORE

```
etcdctl snapshot restore snapshot.db --data-dir /var/lib/etcd-restore
# point etcd at the restored data dir, restart control plane
```

IF ETCD IS UNHEALTHY

Lose quorum (need majority of an odd-numbered cluster, e.g. 2 of 3) and the API becomes read-only or unavailable — no scheduling, no changes. Running pods keep running (kubelet is autonomous) but nothing can change. Disk-full or high latency on etcd degrades the whole control plane.

Interview tip: Key facts: odd-sized etcd (3 or 5) for quorum, regular tested snapshot restores, and that a healthy data plane survives a brief control-plane/etcd outage (existing pods run) but no changes are possible. On managed clusters the provider runs etcd; on self-managed it's your most critical backup.

📚 **Official docs**

- Kubernetes: Operating etcd clusters
- etcd: Disaster recovery
- etcd documentation

Q95 **How do you do capacity planning and bin-packing for a cluster? How do requests drive cost?** Cluster ops

🎙 **Spoken answer · ~73 s**

Cluster cost is driven by node count, and node count is driven by pod requests, not actual usage, because the scheduler reserves capacity based on requests. If teams over-request, nodes look full while the CPU sits idle, and I pay for waste. If they under-request, nodes get overcommitted, causing contention, throttling, and OOM kills. So the first step is right-sizing: measure real usage with Prometheus and VPA recommendations, and set requests to roughly the ninety-fifth percentile plus headroom for CPU, and close to peak for memory, with limits chosen deliberately. Then bin-packing: aim for fewer, fuller nodes using a most-allocated scoring strategy or an autoscaler such as Karpenter with consolidation, choose instance sizes that fit the pod shapes well, and avoid fragmentation from very large pods. I keep a buffer for failures and spikes, since a cluster at ninety-five percent can't tolerate losing a node. Cost visibility with tools like OpenCost or Kubecost, labels for attribution, spot instances for fault-tolerant work, and regular reviews close the loop, with showback to teams so they own their requests.

Likely follow-ups

- How do you choose requests for a spiky workload?
- Why keep spare capacity rather than run nodes as full as possible?

Cluster cost is driven by node count, which is driven by pod **requests** (not actual usage) and how well pods bin-pack onto nodes.

certs.eknathalabs.com

REQUESTS DRIVE SCHEDULING AND COST

```
# The scheduler reserves based on REQUESTS, not real usage.
# Over-requesting -> nodes "full" of reservations while actually idle -> wasted $$
# Under-requesting -> overcommit -> contention, OOMKills, noisy neighbors
```

RIGHT-SIZING

```
- Measure actual usage (Prometheus / VPA recommendations) -> set requests to ~p90+headroom
- Bin-pack: fewer, fuller nodes (MostAllocated scoring / Karpenter consolidation)
- Separate node pools by workload shape (CPU-heavy vs mem-heavy vs GPU)
- Use Cluster Autoscaler/Karpenter to match node count to demand
```

THE TRADE-OFF

Tight bin-packing cuts cost but raises blast radius (more pods per node) and reduces burst headroom. Spreading improves resilience but costs more. Balance per workload criticality.

Interview tip: The crisp insight interviewers want: **requests, not usage, determine cost** — the most common waste is over-provisioned requests reserving capacity that sits idle. VPA in recommendation mode + dashboards of request-vs-usage is how you find it. Karpenter's consolidation actively bin-packs to cut node count.

📚 **Official docs**

- Google SRE Book: Software Engineering in SRE (capacity planning)

Q96 **Explain GitOps with Argo CD or Flux. How does it differ from push-based CI/CD for clusters?** Cluster ops

🎙 **Spoken answer · ~73 s**

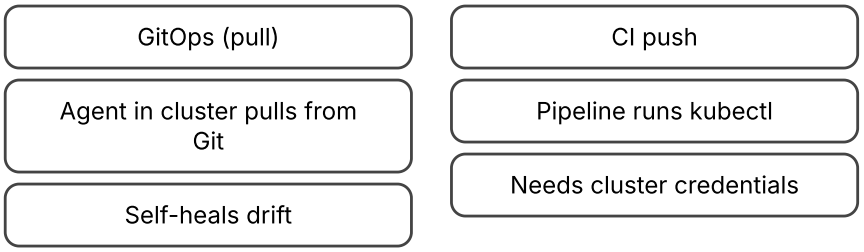
GitOps makes Git the source of truth for desired cluster state, and an agent inside the cluster, Argo CD or Flux, continuously reconciles the live state toward the repository. That's a pull model. In traditional push-based CI/CD, the pipeline holds cluster credentials and runs kubectl apply or helm upgrade from outside. That creates risks: an external system holds production credentials, and there's no continuous drift detection, so manual changes go unnoticed. With pull-based GitOps, the cluster credentials never leave the cluster, CI only builds and pushes an image and updates a manifest or version in Git, and the agent detects drift and can automatically revert it with self-heal. Other benefits: Git history is the audit trail, and rollback is a git revert; every change goes through pull request review; and rebuilding a cluster is just pointing the agent at the repo. Costs to be aware of: managing secrets needs a pattern like External Secrets or Sealed Secrets, the repo structure needs design, and there's a feedback delay since you can't see deployment results directly from CI.

Likely follow-ups

- How do you give CI feedback on whether a GitOps deploy succeeded?
- How do you organise repositories for multiple environments?

certs.eknathalabs.com

🗺 **Pull vs push deploys**



GitOps makes Git the single source of truth for cluster state; an in-cluster agent continuously reconciles the live cluster toward the repo.

PULL-BASED RECONCILIATION

```
Git repo (desired manifests) <- developers PR changes
        ^                              |
        | reconcile loop               v
  Argo CD / Flux agent IN the cluster pulls + applies, detects drift
```

VS PUSH-BASED CI/CD

```
Push (traditional): CI has cluster credentials, runs `kubectl apply` from outside
  - external system holds prod creds (bigger attack surface)
  - no continuous drift detection; manual changes go unnoticed
Pull (GitOps): cluster agent pulls; no external creds; drift auto-corrected
```

BENEFITS

Git is the audit log + rollback (revert a commit), drift detection/self-heal, no cluster creds in CI, declarative and reviewable. Argo CD has a rich UI + sync waves; Flux is lightweight and CRD-driven.

Interview tip: The security angle is the strongest differentiator: pull-based GitOps means CI never holds cluster credentials and the agent continuously corrects drift. Mention app-of-apps (Argo) for managing many apps, sync waves for ordering, and that Git revert = instant rollback. This ties to your CI/CD and platform work.

📚 **Official docs**

- OpenGitOps principles
- Argo CD documentation
- Flux documentation

certs.eknathalabs.com

Q97 **How do you manage configuration across many environments? Helm vs Kustomize vs both.**

Cluster ops

🎙 Spoken answer · ~73 s

Helm and Kustomize solve the multi-environment problem differently. Helm is templating plus packaging: a chart has templated YAML and a values file per environment, and it adds versioning, dependencies, releases, and rollback. It's good for distributing applications, like third-party software, and for heavy parameterisation. The downside is that Go templating inside YAML gets messy and hard to read. Kustomize is overlays with no templating: a base directory contains plain manifests, and per-environment overlays patch it declaratively with changes like replica counts, images, and resource limits. The result is always valid YAML and easy to review, and it's built into kubectl, but it's less suited to deep parameterisation or distribution as a package. In practice many teams use both: Helm for third-party charts, with values per environment, and Kustomize for their own applications, or Kustomize layered over rendered Helm output for last-mile patches. With GitOps tools, either works natively. My rule: start with Kustomize for internal apps for clarity, use Helm when I need to ship a configurable package or consume one, and avoid over-templating.

Likely follow-ups

- When would you use Kustomize on top of a Helm chart?
- What are the downsides of heavy Go templating?

Multi-environment config (dev/stage/prod) needs templating or overlays. The two dominant tools take different approaches.

HELM — TEMPLATING + PACKAGING

```
- Charts: templated YAML + values.yaml per environment
- Package manager: versioning, dependencies, releases, rollback
- Good for: distributing apps, complex parameterization
- Downside: Go templating in YAML can get gnarly
```

KUSTOMIZE — OVERLAYS, NO TEMPLATING

```
- base/ + overlays/{dev,prod}: patch the base declaratively
- No templating language: pure YAML merge/patch (kubectl -k built in)
- Good for: env-specific tweaks to the same base manifests
```

BOTH TOGETHER (COMMON)

Helm to package/version the app + Kustomize to apply env-specific patches on the rendered output. Argo CD and Flux support both natively.

Interview tip: The trade-off: Helm is powerful (packaging, dependencies, releases) but templating-heavy; Kustomize is simpler and template-free but lacks packaging/versioning. Many teams use Helm for third-party charts + Kustomize for their own overlays. Mention that secrets still need separate handling (External Secrets/Sealed Secrets) regardless of which you pick.

📚 Official docs

- Kusto Query Language (KQL)
- Helm documentation
- Kustomize reference

certs.eknathalabs.com

Q98 A service's p99 latency spiked but p50 is fine. How do you investigate in Kubernetes?

Observability

🎙 Spoken answer · ~74 s

A p99-only spike means a subset of requests are slow, so I'm hunting tail latency, not a general slowdown. I work from the application down. First, is it one pod or all? If one instance is slow, compare per-pod latency; an unhealthy pod or a noisy node would stand out, and load may be uneven. Second, CPU throttling: a pod with a CPU limit can be throttled by CFS even when average usage is low, hitting tail latency, visible in the throttled periods metric. Third, application causes: garbage collection pauses, lock contention, cold caches, or a slow code path for certain inputs. Fourth, dependencies: a tail of slow queries from the database or a slow downstream service, and retries that amplify it. Fifth, infrastructure: a node with disk or network trouble, noisy neighbours, and DNS timeouts affecting a fraction of lookups. I use distributed traces to compare slow and fast requests, and break down latency by endpoint, pod, and node. Fixes might be removing or raising CPU limits, adding timeouts and hedging, and fixing the slow dependency.

Likely follow-ups

- Why does CPU throttling hurt p99 more than p50?
- How do traces help isolate tail latency?

A p99-only spike means a subset of requests are slow — tail latency. The investigation walks the layers from app to infra.

LIKELY CAUSES (TAIL LATENCY)

```
- CPU throttling: CFS limits throttling some pods (check container_cpu_cfs_throttled)
- One slow/unhealthy pod behind the Service (uneven load)
- GC pauses, cold caches, lock contention on some requests
- Noisy neighbor on a node; or a node with degraded disk/network
- Downstream dependency tail (DB slow queries) on some calls
- DNS 5s timeouts on a fraction of lookups
```

INVESTIGATION PATH

```
1. Traces: find the slow requests, see WHICH hop adds latency (Jaeger/Tempo)
2. Per-pod metrics: is one replica slow? (uneven -> bad pod / placement)
3. CFS throttling metric: container_cpu_cfs_throttled_seconds_total
4. Node-level: is one node degraded? kubectl top nodes, node metrics
5. Check GC/lock/app metrics; DB slow-query log for downstream
```

Interview tip: The tell is p50-fine/p99-bad = tail, so look for what affects *some* requests: CPU throttling (very common — a CPU limit throttling bursty pods), one bad replica, or a downstream tail. Distributed tracing is the fastest way to localize the slow hop. Mentioning `cfs_throttled` shows you've debugged this for real.

📚 Official docs

- Prometheus: Histograms and summaries

certs.eknathalabs.com

Q99 **How do you implement multi-cluster Kubernetes? When do you need it and what are the patterns?** Cluster ops

🎙 Spoken answer · ~70 s

Multi-cluster means running workloads across several clusters, and I only do it when there's a clear reason. Reasons include surviving a region or cluster failure, serving users from the nearest region for latency, hard isolation between environments or teams for blast radius, scaling beyond a single cluster's limits, and data residency or compliance requirements. The cost is real: more to operate, upgrade, secure, and observe. For patterns, management is usually through GitOps: Argo CD or Flux managing a fleet from one repo, with ApplicationSets or app-of-apps generating per-cluster configuration. For traffic, global load balancing or DNS routes users to clusters, with health-based failover; a service mesh or Cilium Cluster Mesh can provide cross-cluster service discovery and mTLS; and there are multi-cluster Services APIs. For data, the hard part: stateful services need replication across regions, so I often keep state in a managed global database and treat clusters as stateless and replaceable. I also standardise cluster creation with Terraform or Cluster API so clusters are cattle, not pets.

Likely follow-ups

- What makes state the hard part of multi-cluster?
- When is a single larger cluster the better answer?

Multi-cluster runs workloads across several clusters for HA, geo-distribution, isolation, or scale beyond a single cluster's limits.

WHY MULTI-CLUSTER

```
- Region/DR: survive a region or cluster failure
- Geo: serve users from the nearest cluster (latency)
- Isolation: hard tenant/blast-radius separation (prod vs nonprod, per-team)
- Scale: beyond a single cluster's node/object limits
- Compliance/data residency
```

PATTERNS + TOOLING

```
GitOps fleet : Argo CD / Flux manage many clusters from one repo (app-of-apps)
Multi-cluster services : route/failover across clusters (mesh: Istio multi-cluster)
Global LB / GeoDNS : direct users to the right cluster
Fleet management : Rancher, Anthos, Cluster API for provisioning at scale
```

THE COST

Operational complexity multiplies: config consistency, cross-cluster networking/identity, observability aggregation, and data replication across clusters.

Don't over-reach: A single well-run regional cluster with multi-zone node pools already survives zone failures. Reach for multi-cluster only when you need cross-region DR, hard isolation, or scale a single cluster can't meet — the complexity is real.

Interview tip: Lead with "why" — most HA needs are met by one multi-zone cluster; multi-cluster is for region-level DR, hard isolation, or scale limits. Then name GitOps (Argo/Flux) for fleet config and Cluster API/Rancher/fleet tools for provisioning. Be honest about the complexity tax.

certs.eknathalabs.com

📚 **Official docs**

- Argo CD: ApplicationSet

Q100 **How do you debug high control-plane (API server) latency or throttling in a large cluster?**

Observability

🎙 **Spoken answer · ~75 s**

At scale, the API server and etcd become bottlenecks, and the symptoms are slow kubectl, lagging controllers, delayed scheduling, and clients getting throttled with 429 responses. Common causes include huge numbers or sizes of objects, such as giant ConfigMaps or Secrets and thousands of custom resources; expensive LIST calls without pagination, like listing all pods cluster-wide, that hammer etcd; a misbehaving controller or operator with a tight reconcile loop or a retry storm; slow etcd disks; and heavy watch churn or event volume. To investigate, I look at the API server metrics: request duration broken down by verb and resource, in-flight requests, and the API Priority and Fairness queues and rejections. The audit log and the user agent break down who is making the heavy calls. I check etcd metrics for fsync latency, database size, and leader changes. Fixes: stop or fix the noisy client, add pagination and watch caching, reduce object sizes and counts, clean up old Jobs and events, tune APF to protect critical traffic, speed up etcd storage, and shard workloads across clusters if the scale demands it.

Likely follow-ups

- How do you find which client is overloading the API server?
- What does API Priority and Fairness do?

At scale the API server and etcd become bottlenecks. Symptoms: slow kubectl, controller lag, scheduling delays, client throttling (429s).

COMMON CAUSES

```
- Too many/large objects (huge Secrets/ConfigMaps, thousands of CRs)
- Expensive LIST calls without pagination (list all pods cluster-wide) hammering etcd
- A hot controller / operator with a tight reconcile loop
- etcd slow (disk latency) -> everything slows
- High watch churn; large events volume
```

INVESTIGATION

```
- apiserver_request_duration_seconds  (latency by verb/resource)
- apiserver_flowcontrol_*             (APF / Priority and Fairness throttling)
- etcd_disk_wal_fsync_duration        (etcd disk health)
- audit log: who's making expensive LISTs?  rate-limit/fix that client
```

MITIGATIONS

API Priority and Fairness (APF) to isolate misbehaving clients, paginated/field-selector LISTs, fix chatty controllers, fast etcd disks, prune large objects/events, scale control plane (or rely on managed autoscaling).

Interview tip: Name **API Priority and Fairness** (APF) — it prevents one client's flood from starving the control plane and is where you see throttling. The classic culprit is an operator or script doing unpaginated full LISTs against etcd. etcd disk latency underlies many control-plane slowdowns. This is a strong large-cluster operations signal.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: API Priority and Fairness
- Microsoft: Cloud design pattern – Throttling

Q101 **What are finalizers and why does a namespace or resource get stuck Terminating?** Cluster ops

🎙 **Spoken answer · ~76 s**

Finalizers are pre-deletion hooks stored in metadata. When you delete an object that has finalizers, Kubernetes only sets a deletion timestamp; the object stays until each controller that owns a finalizer finishes its cleanup and removes its entry. Then the object is actually deleted. This lets controllers clean up external resources first, like a cloud load balancer or a disk. A resource gets stuck in Terminating when the controller responsible for the finalizer is gone or broken, so nobody removes it. A namespace sticks for similar reasons: it can't finish deleting until everything inside is gone, so a resource with a stuck finalizer, or an unavailable aggregated API service or webhook, blocks it. To debug, I look at the object's finalizers and its deletion timestamp, find which controller should handle them and check it is running, and for namespaces list the remaining resources and the namespace conditions. The last resort is manually patching the finalizers away, but that risks leaking external resources, such as orphaned cloud load balancers or disks, so I do it only after understanding what the finalizer was protecting.

Likely follow-ups

- Why is removing a finalizer manually risky?
- What can make a namespace stuck Terminating even when it looks empty?

Finalizers are pre-deletion hooks: a resource with finalizers won't be removed until each finalizer is cleared, letting controllers do cleanup first.

HOW THEY WORK

```
# metadata.finalizers: [ "example.com/cleanup" ]
# Delete sets deletionTimestamp but the object STAYS until the controller
# finishes cleanup and removes its finalizer -> then it's actually deleted.
```

WHY STUCK TERMINATING

```
- The controller responsible for the finalizer is gone/broken -> never clears it
- A namespace stuck Terminating: a resource in it has a finalizer that can't complete
  (common with broken CRDs / orphaned custom resources / failed APIService)
- Dependent resources can't be cleaned up (external system unreachable)
```

RESOLUTION (CAREFULLY)

```
kubectl get ns <ns> -o json | jq '.spec.finalizers=[]' \
  | kubectl replace --raw /api/v1/namespaces/<ns>/finalize -f -
# Force-removing a finalizer SKIPS the cleanup it guarded -> may orphan resources
```

Caution: Force-clearing finalizers can leak external resources (cloud LBs, disks) the controller would have cleaned up. Fix the broken controller/CRD first if possible; force-clear only as last resort.

certs.eknathalabs.com

Interview tip: Explain that Terminating-but-stuck almost always = an uncleared finalizer waiting on a dead/broken controller. A frequent real case is a leftover APIService/CRD whose controller is gone, blocking namespace deletion. Knowing the finalize subresource trick (and its risk) shows real cleanup experience.

📚 **Official docs**

- Kubernetes: Custom Resources

Q102 **How do you handle node maintenance, draining, and surge upgrades without impacting workloads?** Cluster ops

🎙 **Spoken answer · ~74 s**

Safe node maintenance depends on cordon, drain, and enough redundancy that losing a node is invisible to users. I cordon the node so nothing new schedules there, then drain it, using the Eviction API, which respects PodDisruptionBudgets, ignoring DaemonSets and deleting emptyDir data if appropriate. After the work I uncordon it. What makes it non-disruptive is the workload design: multiple replicas spread across nodes and zones with topology spread constraints; PDBs ensuring a minimum number stay available; a rolling strategy with maxUnavailable zero so new pods are Ready before old ones go; accurate readiness probes; and graceful shutdown with SIGTERM handling and preStop hooks so in-flight requests complete. For fleet-wide upgrades I prefer surge upgrades: add a new node first, then drain the old, so capacity never drops, or replace nodes immutably using new node images instead of patching in place. I do it one node or a small batch at a time, watch error rates, and schedule it away from peak traffic. I also verify that stateful pods can reattach their volumes in the right zone.

Likely follow-ups

- What is a surge upgrade and why is it safer?
- How do you handle a PDB that blocks the drain?

Safe node maintenance hinges on cordon + drain respecting PDBs, plus enough redundancy that losing a node is invisible.

THE DRAIN WORKFLOW

```
kubectl cordon <node>                        # mark unschedulable (no new pods)
kubectl drain <node> --ignore-daemonsets \
  --delete-emptydir-data --grace-period=...  # evict via Eviction API (honors PDBs)
# ... do maintenance / replace node ...
kubectl uncordon <node>                      # back into rotation
```

WHAT MAKES IT NON-DISRUPTIVE

```
- PDBs ensure minAvailable replicas stay up during eviction
- maxUnavailable:0 + readiness ensures replacements are Ready before old ones leave
- Topology spread / anti-affinity so replicas weren't all on this node
- DaemonSets stay (--ignore-daemonsets); emptyDir data is acknowledged lost
```

SURGE UPGRADES (MANAGED)

GKE/EKS surge upgrades create extra new-version nodes first, then cordon+drain old ones, keeping capacity constant — tune surge/unavailable to balance speed vs cost.

certs.eknathalabs.com

Interview tip: The drain honoring PDBs is the safety mechanism — but a bad PDB (single replica, minAvailable=replicas) makes drain hang forever. So the real prerequisites are: >=2 replicas, sane PDBs, topology spread, and readiness probes. Surge upgrades keep capacity flat during the roll.

📚 **Official docs**

- Kubernetes: Safely drain a node

Q103 **How do you instrument and trace a request across multiple microservices on Kubernetes?**

Observability

🎙 **Spoken answer · ~76 s**

Distributed tracing follows one request through all the services it touches. The entry point creates a trace ID and a root span. Each service creates a child span for its own work and passes the trace context to the next service, usually in the W3C traceparent header. Each service sends its spans to a collector, and the backend assembles them into one trace, displayed as a waterfall showing where the time went. In Kubernetes I implement it with OpenTelemetry. I instrument applications with the OTel SDKs, using auto-instrumentation for many languages, or get some of it from a service mesh. An OpenTelemetry Collector runs as a DaemonSet or a gateway Deployment, receives spans, batches them, applies sampling, and exports them to Tempo, Jaeger, or a vendor. Context propagation is the important detail: a single service that drops the header breaks the trace, including through message queues, so I propagate it in message metadata too. I put the trace ID into log lines to link logs to traces, and use sampling, like tail-based sampling that keeps errors and slow traces, to control cost.

Likely follow-ups

- Why do traces break across message queues, and how do you fix it?
- What's the difference between head-based and tail-based sampling?

Distributed tracing follows a single request across service hops by propagating a trace context, revealing where latency and errors occur.

HOW TRACING WORKS

```
# Entry point creates a trace_id + root span
# Each service: create a child span, propagate context via headers
#   (W3C traceparent header) to the next service
# Spans shipped to a collector -> assembled into one trace (waterfall view)
```

IMPLEMENTATION WITH OPENTELEMETRY

```
- Instrument apps with OTel SDK (auto-instrumentation for many languages)
- OTel Collector (DaemonSet/Deployment) receives + exports spans
- Backend: Jaeger / Tempo / cloud trace
- Service mesh can add spans for service-to-service hops automatically
```

SAMPLING

100% tracing is expensive at scale. Head sampling (decide upfront) is cheap but may miss rare errors; tail sampling (decide after seeing the trace) keeps slow/error traces — better signal, more infra.

certs.eknathalabs.com

Interview tip: The crux is **context propagation** — the W3C `traceparent` header carried hop to hop is what stitches spans into one trace. Name OpenTelemetry as the standard, mention a service mesh can auto-instrument the network hops, and discuss tail vs head sampling (tail keeps the interesting traces). Correlating trace IDs into logs closes the loop with the three pillars.

📚 **Official docs**

- OpenTelemetry: Signals (traces, metrics, logs)
- OpenTelemetry: Sampling

Q104 **Design a CI/CD pipeline that builds and deploys to Kubernetes. What are the stages?**

CI/CD & GitOps

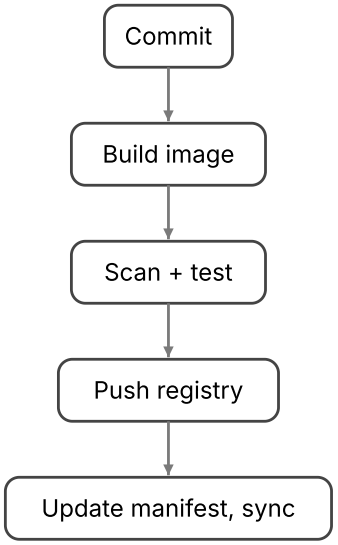
🎙 **Spoken answer · ~77 s**

I separate CI, which produces a trusted artifact, from CD, which delivers it, ideally with GitOps. The CI stages are: lint and unit tests; build the image reproducibly from a pinned base, tagged with the Git commit SHA, never latest; scan it with Trivy or Grype and fail on critical vulnerabilities; generate an SBOM and sign the image with cosign; push to the registry and record the digest; and update the deployment manifest or Helm values with the new digest, committing to the GitOps repo. The CD stages: Argo CD or Flux detects the change and syncs the cluster; the rollout is progressive, with canary or blue-green using Argo Rollouts, gated on metrics; there are smoke tests and health checks after deploy; and automatic rollback if SLOs degrade. Promotion between environments is by promoting the same immutable digest, not rebuilding, with approvals before production. Around this I add security: OIDC for short-lived credentials, no cluster credentials in CI, policy checks at admission for signatures, and secrets from a manager. And I track DORA metrics: deployment frequency, lead time, change failure rate, and recovery time.

Likely follow-ups

- Why promote the same image digest instead of rebuilding per environment?
- What gates would you put between staging and production?

🗺 **Build and deploy**



certs.eknathalabs.com

A production K8s pipeline separates CI (build/test/scan/push) from CD (deploy/verify), ideally with GitOps for the deploy half.

CI STAGES

```
1. Lint + unit test
2. Build image (reproducible, pinned base) -> tag with git SHA (not :latest)
3. Scan: Trivy/Grype for CVEs (fail on criticals)
4. SBOM (Syft) + sign image (cosign)
5. Push to registry by digest
6. Update the deployment manifest / Helm values with the new digest (commit to GitOps repo)
```

CD STAGES (GITOPS)

```
7. Argo CD / Flux detects the repo change -> syncs to cluster
8. Progressive rollout (canary / blue-green via Argo Rollouts)
9. Automated verification: smoke tests + SLO/metric checks
10. Auto-promote or auto-rollback on failure
```

Anti-patterns: deploying `:latest` (non-reproducible, can't roll back to a known image), giving CI direct cluster-admin creds (use GitOps pull instead), and skipping the scan/sign gate.

Interview tip: The senior structure is CI pushes an artifact + opens a commit to a config repo; CD is a pull-based GitOps agent. Tag by immutable git SHA/digest so every deploy is reproducible and rollback is just pointing at the prior digest. Mentions of scan/sign/SBOM gates show supply-chain awareness.

📚 **Official docs**

- OpenGitOps principles

Q105 **How do you build container images inside a Kubernetes cluster without the Docker daemon?** CI/CD & GitOps

🎙 **Spoken answer · ~73 s**

Mounting the Docker socket into a CI pod is a serious security risk, because anything that can talk to the Docker daemon is effectively root on the node and can start privileged containers. Docker-in-Docker avoids sharing the host socket but needs privileged mode, which is also insecure and slow. The alternative is daemonless builders. Kaniko builds from a Dockerfile inside an unprivileged container and pushes to the registry, with no daemon. Buildah builds OCI images without a daemon and can run rootless. BuildKit can run rootless and supports advanced caching and parallel builds, and is available as a standalone service. Cloud options like Cloud Build also isolate builds entirely. Practical considerations: use a layer cache stored in the registry to keep builds fast, run builds on dedicated, ephemeral node pools, isolated from production workloads, restrict network egress, and use short-lived credentials for pushing, via workload identity. I'd also scan and sign the resulting image. Kaniko has had limitations and maintenance concerns, so I check the current project status and consider BuildKit or Buildah as alternatives.

Likely follow-ups

- Why is mounting the Docker socket equivalent to root on the node?
- How do you speed up daemonless builds with caching?

certs.eknathalabs.com

Mounting the Docker socket into a CI pod is a major security risk (root on the node). Daemonless builders solve this.

WHY NOT DOCKER-IN-DOCKER / DOCKER.SOCK

```
# Mounting /var/run/docker.sock = container can control the host daemon
#   -> effectively root on the node (container escape). Avoid in CI.
# DinD (privileged) is also insecure + slow.
```

DAEMONLESS BUILDERS

```
Kaniko      : builds from a Dockerfile in an unprivileged container, pushes to registry
Buildah     : daemonless OCI builds (rootless capable)
BuildKit    : can run rootless; advanced caching, used by `docker buildx`
Cloud Native Buildpacks : no Dockerfile -> auto-detect + build (pack / kpack)
```

CACHING FOR SPEED

Layer caching (Kaniko cache, BuildKit registry cache) is essential or builds are slow. kpack/Buildpacks rebase only changed layers.

Interview tip: The headline: never mount the Docker socket in CI — use **Kaniko** or **BuildKit rootless** to build unprivileged inside the cluster. Buildpacks (kpack) remove the Dockerfile entirely and standardize images. This is a security + supply-chain answer as much as a build one.

📚 **Official docs**

- Docker documentation

Q106 **Explain progressive delivery with Argo Rollouts. How do canary and blue-green differ?** CI/CD & GitOps

🎙 **Spoken answer · ~74 s**

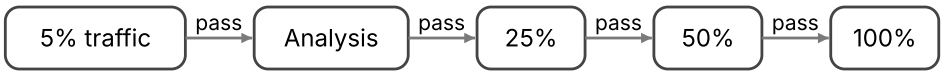
Argo Rollouts replaces the Deployment controller with a Rollout resource that supports advanced strategies and automated analysis. In a canary strategy, I define steps: set the weight to ten percent, pause, run an analysis, go to fifty percent, then to one hundred. Traffic is shifted using an ingress controller or service mesh for precise weights, or just by replica counts without one. The analysis step queries metrics, like success rate from Prometheus, and if the SLO degrades, the rollout aborts and rolls back automatically. In a blue-green strategy, the full new version, the green stack, is started alongside the current blue one, can be tested through a preview service, and then traffic switches all at once by changing the active Service, with the old version kept briefly for instant rollback. The trade-offs: canary limits blast radius and validates with real traffic, but takes longer and needs good metrics and traffic splitting. Blue-green gives instant cutover and rollback but needs double capacity temporarily and exposes all users at the same moment. Either way, database changes must be backward compatible.

Likely follow-ups

- What metrics would you use for automated canary analysis?
- How does traffic splitting work without a service mesh?

certs.eknathalabs.com

🗺 **Argo Rollouts canary**



Argo Rollouts replaces the Deployment controller with one that supports canary and blue-green strategies plus automated analysis.

CANARY — GRADUAL SHIFT

```
strategy:
  canary:
    steps:
    - setWeight: 10
    - pause: { duration: 5m }
    - analysis: { templates: [{ templateName: success-rate }] }
    - setWeight: 50
    - pause: {}
    - setWeight: 100
```

Shifts a small % of traffic to the new version, runs metric analysis, and auto-rolls-back if SLOs degrade — then ramps up.

BLUE-GREEN — INSTANT SWITCH

```
- Stand up the full new version (green) alongside old (blue)
- Test green via a preview Service
- Flip the Service selector blue -> green instantly (and back to roll back)
```

TRADE-OFFS

Canary: gradual risk exposure, less spare capacity, slower; great with metric analysis. Blue-green: instant cutover + instant rollback, but needs 2x capacity and exposes all users at once at the flip.

Interview tip: Tie the choice to risk + cost: canary for gradual, metric-gated rollout (best default); blue-green for fast cutover/rollback when you can afford double capacity and want an instant switch. The killer feature is automated analysis -> auto-rollback on SLO breach, removing humans from the rollback decision.

📚 **Official docs**

- Kubernetes: Deployments
- Google SRE Workbook: Canarying Releases
- Argo Rollouts

certs.eknathalabs.com

Q107 **How do you secure a CI/CD pipeline that deploys to Kubernetes end to end?** CI/CD & GitOps

🎙 Spoken answer · ~72 s

I secure the pipeline end to end because each stage is a supply-chain entry point. Source: protected branches, required reviews, signed commits, and secret scanning so no credentials are in Git. Build: ephemeral, isolated runners, no Docker socket, pinned base images and dependencies, and scanning of code and images. Artifact: generate an SBOM, sign the image with cosign, push by digest, and use immutable tags. Deploy: use GitOps pull so CI holds no cluster credentials, and admission control verifies signatures and registry origin. Runtime: least-privilege service accounts, network policies, and runtime detection with Falco. Credentials get special attention: use OIDC federation so the pipeline obtains short-lived cloud tokens instead of long-lived secrets, scope them narrowly per job, and keep secrets in a secret manager, masked in logs. I also treat the CI system itself as critical infrastructure: restrict who can edit pipeline definitions, since changing the pipeline can exfiltrate secrets, control third-party actions by pinning them to commit SHAs, and keep audit logs. Finally, separate duties between building and approving production deploys.

Likely follow-ups

- Why pin third-party actions to a commit SHA?
- How does OIDC federation remove long-lived CI secrets?

Pipeline security spans source, build, artifact, and deploy — each a potential supply-chain entry point.

LAYERED CONTROLS

```
Source : protected branches, signed commits, required reviews, no secrets in git
Build  : ephemeral runners, no docker.sock, pinned base images, scan (Trivy)
Artifact: SBOM (Syft), sign (cosign), push by digest, immutable tags
Deploy : GitOps pull (CI holds no cluster creds), admission verifies signatures
Runtime: least-privilege SA, NetworkPolicy, runtime detection (Falco)
```

CREDENTIALS

```
- OIDC federation from CI -> cloud (no long-lived keys stored in CI)
- Short-lived, scoped tokens; rotate; never echo secrets in logs
- Pull-based GitOps so prod cluster creds never live in the CI system
```

ADMISSION ENFORCEMENT

Cluster rejects unsigned images / disallowed registries (Kyverno/sigstore policy-controller), closing the loop so even a compromised CI can't deploy an unverified image.

Interview tip: Frame it as SLSA-style supply-chain integrity: every step adds provenance, and admission verification at the cluster is the final gate. The two biggest wins: OIDC (no static cloud keys in CI) and pull-based GitOps (no cluster creds in CI) — both remove the most-stolen credentials from the pipeline.

📚 Start here (official docs)

- Kubernetes documentation

certs.eknathalabs.com

Q108 How do you manage Helm releases in CI/CD, and how do rollbacks work?

CI/CD & GitOps

🎙 Spoken answer · ~73 s

In CI/CD I run helm upgrade with install, a values file per environment, and the image tag or digest set from the build, typically the Git commit SHA. The atomic flag is important: if the upgrade fails or times out, Helm automatically rolls back, and the wait flag with a timeout makes Helm wait for resources to become ready. Every upgrade stores a revision, kept in a Secret in the cluster, so helm history lists revisions and helm rollback goes back to a chosen one. Rollback re-applies the earlier manifests, but it can't undo non-Kubernetes side effects like database migrations, so schema changes need to be backward compatible. Also note that Helm doesn't upgrade CRDs in the crds directory, so those need separate handling. With GitOps through Argo CD or Flux, I don't usually run Helm from CI at all: the tool renders the chart from Git, and rollback means reverting the commit, which keeps Git the source of truth. I lint and template charts in CI, run helm diff before upgrades, and pin chart versions.

Likely follow-ups

- Why doesn't helm rollback undo a database migration?
- How are CRDs handled differently from other resources in Helm?

Helm packages and versions releases; in CI/CD you template values per environment and rely on release history for rollback.

RELEASE FLOW

```
helm upgrade --install myapp ./chart \
  -f values-prod.yaml \
  --set image.tag=$GIT_SHA \
  --atomic --timeout 5m
# --atomic: auto-rollback if the upgrade fails/times out
```

ROLLBACK

```
helm history myapp              # list revisions
helm rollback myapp 4           # revert to revision 4
# Each upgrade stores a revision (in a Secret) -> rollback is reverting to it
```

HELM IN GITOPS

With Argo CD/Flux you usually don't run `helm upgrade` directly — the agent renders the chart and applies it; rollback = git revert of the values/chart version. This keeps git as the source of truth and audit log.

Gotcha: `--atomic` + `--timeout` are important — without them a partially-applied failed upgrade leaves the release in a broken state. Also CRDs are not upgraded by Helm by default (Helm only installs CRDs on first install) — manage CRD upgrades explicitly.

Interview tip: Mention `--atomic` (auto-rollback on failure) as the safety flag, that Helm stores revision history for rollback, and that in GitOps the rollback is a git revert rather than a `helm rollback`. The CRD-upgrade caveat is a real-world gotcha that signals experience.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Deployments
- Helm: Chart hooks
- Helm documentation

Q109 **How do you run CI jobs themselves on Kubernetes (e.g. ephemeral runners)? Trade-offs?**

CI/CD & GitOps

🎙 **Spoken answer · ~74 s**

Running CI on Kubernetes means each job runs in a fresh pod instead of on a long-lived static runner. Tools include the GitHub Actions runner controller, GitLab's Kubernetes executor, Tekton, and Argo Workflows. The benefits are clean, isolated environments with no state bleeding between builds, elasticity, since pods scale to zero when idle and burst when the queue is long, using HPA or KEDA on queue depth, and efficient use of the existing cluster and spot nodes. The challenges: builds that need Docker require daemonless builders or sandboxing to avoid privileged containers; cold starts and image pulls slow jobs, so I use cached images and persistent or registry-backed build caches; resource contention can affect production if CI shares nodes, so I use dedicated node pools with taints; and security needs care, since CI pods run untrusted code, so I limit network access, service account permissions, and secrets. Debugging ephemeral pods is harder, so I keep logs and artifacts externally. For the right scale it's cost efficient, but small teams may be better off with a hosted runner service.

Likely follow-ups

- How do you isolate CI workloads from production workloads?
- How do you handle caching with ephemeral runners?

Running CI on K8s gives elastic, isolated, ephemeral build capacity — each job in a fresh pod — instead of static always-on runners.

THE PATTERN

```
- GitHub Actions runner controller / GitLab K8s executor / Tekton / Argo Workflows
- Each pipeline job -> a fresh pod (clean env, no state bleed between builds)
- Scale to zero when idle; burst to many pods under load (HPA/KEDA on queue depth)
```

TRADE-OFFS / CHALLENGES

```
+ Elastic + cost-efficient (scale to zero), isolated, reproducible
+ Reuse cluster autoscaling + spot nodes for cheap build capacity
- Build caching is harder (ephemeral pods lose local cache) -> use registry/remote cache
- Image builds need daemonless (Kaniko/BuildKit) since no docker.sock
- Cold-start latency for new pods/nodes; secrets must be injected securely
```

SCALING ON DEMAND

KEDA scales runner pods on queue depth (pending jobs), and Cluster Autoscaler/Karpenter add nodes — often on spot instances for cheap, interruptible build capacity.

Interview tip: The wins are isolation + scale-to-zero economics; the catches are caching (use remote/registry cache) and daemonless image builds. Name KEDA for queue-based runner autoscaling and spot nodes for cost — a nice tie-in to FinOps and autoscaling

certs.eknathalabs.com

topics.

📚 **Official docs**

- Kubernetes: Jobs
- Terraform: Sensitive data in state

Q110 **How do you handle database schema migrations safely during a Kubernetes rolling deploy?** CI/CD & GitOps

🎙 **Spoken answer · ~75 s**

During a rolling update, old and new versions run at the same time against one database, so migrations must be backward-compatible. A destructive change, like dropping or renaming a column, breaks whichever version doesn't expect it, and I can't atomically swap the schema and every pod. The pattern is expand and contract, also called parallel change. Step one, expand: add the new column or table in a way both versions tolerate, for example nullable or with a default. Step two, deploy code that writes to both old and new, and can read either, and backfill existing data in batches. Step three, switch reads to the new structure. Step four, contract: after all old versions are gone and it's verified, remove the old column in a later release. Operationally I run migrations as a separate, controlled step, such as a Job or pipeline stage before the rollout, not in every pod's startup where replicas would race. I avoid long locks on big tables by using online schema change tools, and I keep migrations idempotent and tested, with a rollback or forward-fix plan.

Likely follow-ups

- Why shouldn't every pod run migrations at startup?
- How do you backfill a huge table without locking it?

During a rolling update, old and new app versions run simultaneously against one database — so migrations must be backward-compatible. This is the expand/contract pattern.

THE PROBLEM

A rolling deploy means for a window both v1 and v2 pods are live. A destructive migration (drop/rename a column) breaks whichever version doesn't expect it. You cannot atomically swap schema + all pods.

EXPAND / CONTRACT (PARALLEL CHANGE)

```
1. EXPAND : add new column/table (additive, both versions still work)
2. MIGRATE: deploy code that writes BOTH old+new, reads new (backfill data)
3. CONTRACT: after all pods are v2 and data migrated, drop the old column
# each step is backward-compatible with the version before it
```

WHERE TO RUN MIGRATIONS

```
- Init container or a Job (gated before rollout), NOT in every pod's startup race
- Ensure idempotency + locking so concurrent pods don't double-run
```

Never do a destructive, non-backward-compatible migration in the same deploy as the code that needs it — mid-rollout the old pods will error. Split across releases (expand now, contract later).

certs.eknathalabs.com

Interview tip: Expand/contract (a.k.a. parallel change) is the expected answer. Stress that rolling deploys force backward-compatible schemas because two versions share the DB, and run migrations as a gated Job, not racing in pod startup. This connects to the zero-downtime rollout point from the pods section.

📚 **Official docs**

- gh-ost: GitHub's online schema migration
- Flyway documentation

Q111 **What's the difference between a Deployment rollback and a GitOps rollback? Which is safer?** Cluster ops

🎙 **Spoken answer · ~70 s**

Both revert to an earlier state, but they differ in how. kubectl rollout undo is imperative: it rolls the Deployment back to a previous ReplicaSet. It's fast, but it makes the live cluster diverge from Git, so the repository still contains the bad version. If a GitOps controller is running with self-heal, it will immediately re-apply the bad version from Git and undo your rollback, or the next deploy brings the bug back. A GitOps rollback is declarative: you git revert the bad commit and push. The controller syncs the cluster back, so the live state matches Git again, and the rollback is reviewed, recorded, and auditable. That's why it's safer and more consistent. The trade-off is speed: a GitOps rollback goes through the repository and sync interval, though a webhook and a manual sync make it quick. In an emergency I might pause auto-sync and use an imperative undo to stop the bleeding, but then immediately reconcile Git. Remember that neither undoes data changes such as migrations.

Likely follow-ups

- What happens if you run kubectl rollout undo on an Argo CD managed app?
- How do you make a GitOps rollback fast during an incident?

Both revert to a prior state, but one is imperative and ephemeral, the other declarative and auditable.

KUBECTL ROLLOUT UNDO

```
kubectl rollout undo deployment/web           # revert to previous ReplicaSet
kubectl rollout undo deployment/web --to-revision=3
# Fast, but: it diverges from git -> the live state no longer matches the repo
```

GITOPS ROLLBACK

```
git revert <bad-commit> && git push     # repo is source of truth
# Argo/Flux syncs the cluster back -> live state matches git again
```

WHY GITOPS IS SAFER

With GitOps, an imperative `rollout undo` creates **drift** — the agent will detect the live state differs from git and may re-sync forward to the "bad" version, undoing your manual rollback. The correct rollback is reverting the commit so git and cluster agree.

Gotcha: Doing `kubectl rollout undo` on a GitOps-managed app is a trap — Argo CD sees drift and re-applies the repo (the broken version) unless auto-sync is paused. Always roll back in git for GitOps-managed workloads.

certs.eknathalabs.com

Interview tip: The insight: in a GitOps world, the cluster is not the source of truth — git is. Manual `rollout undo` fights the reconciler. Safer rollback = git revert, which preserves auditability and keeps the reconcile loop consistent. Showing you understand drift vs source-of-truth is a strong platform signal.

📚 **Official docs**

- Kubernetes: Deployments
- OpenGitOps principles

Q112 **How do you schedule GPU or other specialized hardware workloads? What's the device plugin model?** Scheduling

🎙 **Spoken answer · ~73 s**

GPUs and similar devices aren't native Kubernetes resources, so the device plugin framework advertises them. A device plugin, such as NVIDIA's, runs as a DaemonSet on GPU nodes, discovers the devices, and registers them with the kubelet as an extended resource like nvidia.com slash gpu. Pods then request them in resource limits, for example one GPU, and the scheduler places them only on nodes with capacity. Extended resources are whole numbers and can't be overcommitted, so one GPU goes to one container, unless I use sharing features such as time slicing, MIG partitioning on supported NVIDIA cards, or MPS. For placement, I taint GPU nodes so ordinary pods don't land on expensive hardware, and give GPU pods a matching toleration and node affinity or selector for the GPU type. Operational points: install drivers and the container toolkit, often through the GPU Operator; autoscale GPU node pools carefully since they're expensive and slow to start; monitor GPU utilisation with DCGM exporter; and use spot capacity and queueing, with tools like Kueue, for batch training to improve utilisation.

Likely follow-ups

- How can multiple pods share one GPU?
- Why taint GPU nodes?

GPUs and other devices aren't native K8s resources — the device plugin framework advertises them as schedulable resources on nodes.

DEVICE PLUGIN MODEL

```
# A DaemonSet (e.g. NVIDIA device plugin) runs on GPU nodes:
# - discovers GPUs, registers them with kubelet as nvidia.com/gpu
# - pods request them like any resource:
resources:
  limits: { nvidia.com/gpu: 1 }
```

PLACEMENT

```
- Taint GPU nodes (nvidia.com/gpu=present:NoSchedule) so only GPU pods land there
- GPU pods add a matching toleration + node affinity for the GPU node label
- Node labels often auto-applied by NVIDIA GPU Operator / node feature discovery
```

certs.eknathalabs.com

SHARING + EFFICIENCY

GPUs are expensive and not natively shareable; options include time-slicing, MIG (multi-instance GPU partitioning), and MPS to pack multiple workloads per GPU. The NVIDIA GPU Operator manages drivers, plugin, and monitoring.

Interview tip: The device plugin framework (DaemonSet advertising `vendor.com/resource` to kubelet) is the general mechanism — works for GPUs, FPGAs, NICs. Pair with taints (keep non-GPU pods off pricey nodes) + affinity. Mention MIG/time-slicing for GPU cost efficiency — a nice FinOps tie-in given how expensive GPU nodes are.

📚 **Official docs**

- Kubernetes: Scheduling GPUs

Q113 **How do you alert effectively without alert fatigue? What makes a good vs bad Kubernetes alert?**

Observability

🎙 **Spoken answer · ~74 s**

Good alerting pages a human only for problems that are urgent, actionable, and user-impacting. Bad alerts cause fatigue: cause-based alerts on transient conditions like a pod restarted once or CPU above eighty percent for one minute, alerts where there's nothing for the on-call person to do, per-pod noise in a fleet that heals itself, and static thresholds that flap. Good alerts are symptom-based: a breach of a user-facing SLO, using error rate or latency burn-rate, or a clear condition such as a Deployment with zero available replicas for five minutes. Each alert should link to a runbook and a dashboard and state what to check first. Cause-level signals, such as restarts, node pressure, and disk usage trends, go to dashboards or tickets, not pages. I also use alert hygiene: group and deduplicate in Alertmanager, add inhibition so a root cause suppresses downstream alerts, use for-durations to avoid blips, review noisy alerts after each on-call rotation, and delete alerts nobody acts on. The test is simple: if it pages me and I do nothing, it should not page.

Likely follow-ups

- How does alert inhibition reduce noise?
- What belongs in an alert's runbook?

Good alerting pages humans only for actionable, user-impacting problems. Most cluster noise should be dashboards or tickets, not pages.

BAD ALERTS (CAUSE FATIGUE)

```
- Cause-based on transient states: "pod restarted once", "CPU > 80% for 1 min"
- Non-actionable: nothing for the on-call to do
- Per-pod noise in a fleet that self-heals
- Static thresholds that flap
```

GOOD ALERTS (SYMPTOM + ACTIONABLE)

```
- Symptom-based: user-facing SLO breach (error rate / latency burn-rate)
- "Deployment has 0 available replicas for 5m" (real outage)
- "PVC > 90% and filling" (will fail soon, actionable)
- Has a runbook link + clear owner
```

certs.eknathalabs.com

PRINCIPLES

Alert on symptoms (is the user affected?) not causes; use burn-rate/multi-window to cut flapping; route non-urgent signals to tickets/dashboards; every page must be actionable with a runbook.

Interview tip: The Google SRE principle — **alert on symptoms, not causes** — is the headline. A single node at 80% CPU isn't a page; users seeing errors is. Burn-rate alerting (from the SLO question) reduces noise. Demonstrating you'd ruthlessly cut non-actionable alerts shows on-call maturity.

📚 **Official docs**

- Google SRE Book: Practical Alerting

Q114 **How does Cilium / eBPF change Kubernetes networking, and why are teams adopting it?** Networking

🎙 **Spoken answer · ~72 s**

Cilium uses eBPF to run networking, security, and load-balancing logic inside the Linux kernel instead of relying on long chains of iptables rules. It can replace kube-proxy entirely, doing Service load balancing in eBPF, which avoids the rule explosion and slow updates that iptables suffers at scale. For policy, it enforces NetworkPolicy using identities based on labels rather than IP addresses, so it handles pod churn efficiently, and it adds layer seven aware rules, such as allowing only GET on a given path, and DNS-aware policies like allowing egress to a specific domain. Hubble gives deep observability, showing flows, drops, and which policy dropped a packet, without sidecars. Teams adopt it for performance at scale, richer security, and visibility, plus extras like transparent encryption with WireGuard, cluster mesh for multi-cluster connectivity, and a sidecar-less service mesh. The trade-offs: it needs a reasonably modern kernel, it's a significant component to learn and operate, and debugging eBPF behaviour requires different tools than iptables. I'd adopt it when scale, policy needs, or observability justify the change.

Likely follow-ups

- What advantage does identity-based policy have over IP-based policy?
- What kernel requirements does Cilium impose?

Cilium uses eBPF to run networking, security, and load-balancing logic in the kernel datapath, replacing iptables-based kube-proxy and unlocking richer features.

WHAT EBPF REPLACES

```
- kube-proxy iptables/IPVS -> eBPF Service load-balancing (no rule explosion)
- L3/L4 NetworkPolicy -> eBPF enforcement + identity-based (not IP-based) policy
- Adds L7-aware policy (allow only GET /api), DNS-aware policy
- Hubble: deep network observability (flow visibility) from eBPF
```

certs.eknathalabs.com

WHY TEAMS ADOPT IT

```
- Performance at scale: no linear iptables chains; lower latency
- Identity-based security: policy follows workload identity, not ephemeral pod IPs
- Observability: Hubble shows every flow without sidecars
- kube-proxy replacement + sidecar-less service mesh (ambient/eBPF mesh)
```

THE SHIFT

eBPF lets you run programmable logic in the kernel safely, so networking/security/observability move from userspace proxies and iptables into the datapath — faster and more flexible.

Interview tip: Connect to earlier points: iptables kube-proxy doesn't scale (linear rule eval), and IP-based NetworkPolicy is brittle as pods churn. Cilium's eBPF gives O(1) LB, identity-based policy, L7 awareness, and Hubble observability — and is moving toward sidecar-less service mesh. Naming Hubble and identity-based policy signals current knowledge.

📚 **Official docs**

- Linux kernel docs: BPF
- ebpf.io: What is eBPF?
- Cilium: Introduction

Q115 How do you handle stateful workload disaster recovery across regions in Kubernetes?

Storage

🎙 **Spoken answer · ~75 s**

Stateful disaster recovery needs two things: the data in another region, and the ability to rebuild the workload there. The data layer comes first, and the design is driven by RPO and RTO. For near-zero RPO I use application-level replication across regions, such as Postgres streaming or logical replication, or Kafka MirrorMaker, with a warm standby ready to be promoted. Volume snapshots replicated cross-region to object storage are cheaper but give a higher RPO. Velero backs up Kubernetes objects and volume snapshots and can restore them into another cluster. For the workload layer, the second region has a cluster built from the same Terraform and GitOps definitions, so it is reproducible, with the application manifests ready to apply, and DNS or a global load balancer to redirect traffic. The DR strategies range from backup and restore, which is cheapest and slowest, through pilot light and warm standby, to active-active, which is fastest and most complex and expensive. Whatever I choose, I document the failover runbook, automate what I can, and run regular DR drills, because untested DR is unreliable.

Likely follow-ups

- How do you choose between warm standby and active-active?
- What is the difference between RPO and RTO?

Stateful DR needs both the data (replicated/backed up off-region) and the ability to reconstitute the workload in another region/cluster.

DATA LAYER

```
- App-level replication across regions (DB native: Postgres streaming/logical,
  Kafka MirrorMaker) -> warm standby, low RPO
- Volume snapshots replicated cross-region (object storage) -> higher RPO
- Velero: backs up K8s objects + PV snapshots, restore to another cluster
```

certs.eknathalabs.com

RTO / RPO DRIVE THE DESIGN

```
RPO (data loss tolerance):  continuous replication = near-zero; snapshots = minutes/hrs
RTO (downtime tolerance):   warm standby = minutes; backup-restore = hours
# Tighter RPO/RTO = more cost (active-active / continuous replication)
```

FAILOVER ORCHESTRATION

Promote the standby (operator or runbook), repoint DNS/global LB to the DR region, and reconcile the workloads there (GitOps applies the same manifests to the DR cluster).

Untested DR is no DR: regularly run restore drills (Velero restore, standby promotion) and measure actual RTO/RPO — backups that have never been restored routinely fail when you need them.

Interview tip: Lead with RPO/RTO defining the architecture and cost, then map: continuous app-level replication (low RPO/RTO, costly) vs Velero snapshot/restore (cheaper, higher RPO/RTO). Stress tested restore drills and that GitOps makes re-deploying the workload to the DR cluster trivial — data is the hard part, not manifests.

📚 **Official docs**

- Google SRE Book: Data Integrity
- Kubernetes: Volume snapshots

Q116 **How do you debug 'my cluster is slow' when you don't know the layer? Give a systematic approach.**

Cluster ops

🎙 **Spoken answer · ~69 s**

When someone says the cluster is slow, I narrow the scope before diving in. Is it kubectl and deploys that are slow, which points at the control plane, or are application responses slow, which points at the app, network, or node resources? One service or the whole cluster, one node or all? When did it start, and does it correlate with a deploy, a scale event, or an upgrade? Then I sweep the layers. Control plane: API server request duration, API Priority and Fairness throttling, and etcd fsync latency. Nodes: top nodes, disk and network saturation, node conditions, and pressure. Scheduling: Pending pods and autoscaler activity. Networking: DNS latency, kube-proxy and conntrack issues, and CNI health. Workloads: restarts, OOMKills, CPU throttling, and HPA behaviour. Application: golden signals, traces, and dependency latency. I use the USE method for resources and RED for services. Finally, I check recent changes first because they are the most likely cause, mitigate by rolling back or scaling, and document the findings.

Likely follow-ups

- Which signals tell you the control plane is the bottleneck?
- Why check recent changes first?

"Cluster is slow" is ambiguous — a systematic top-down sweep localizes it to control plane, node, network, or app.

NARROW THE SCOPE FIRST

cers.eknathalabs.com

```
- Slow kubectl/deploys?  -> control plane (apiserver/etcd)
- Slow app responses?    -> app, network, or node resource
- Specific service or whole cluster? one node or all?
- When did it start? correlate with a deploy / scale event / upgrade
```

LAYER-BY-LAYER CHECKS

```
Control plane: apiserver_request_duration, APF throttling, etcd fsync latency
Nodes:         kubectl top nodes; disk/network saturation; node conditions
Workloads:     kubectl top pods; CFS throttling; OOMKills; restart counts
Network:       DNS latency (the 5s tell), NetworkPolicy drops, conntrack table full
App/deps:      traces for slow hop; downstream DB latency
```

COMMON CULPRITS

etcd on slow disk, CPU CFS throttling, DNS (ndots/5s timeouts), conntrack table exhaustion on busy nodes, a chatty controller hammering the API, a noisy neighbor.

Interview tip: Show method, not guessing: scope it (control plane vs data plane vs network), correlate with a change, then drill the suspect layer with specific metrics. Naming the usual suspects — etcd disk latency, CFS throttling, DNS 5s timeouts, conntrack exhaustion, APF throttling — proves you've actually chased these down.

📚 **Official docs**

- Kubernetes: Operating etcd clusters
- etcd documentation

Q117 **What is a Job and CronJob, and how do you make batch workloads reliable on Kubernetes?**

Pods & workloads

🎙 **Spoken answer · ~73 s**

A Job runs pods until a number of them complete successfully, and a CronJob creates Jobs on a schedule. Reliability comes from the knobs. Completions and parallelism control how many successful runs are needed and how many run at once. backoffLimit sets the number of retries before the Job is marked failed, with exponential backoff. activeDeadlineSeconds is a hard time cap so a hung job can't run forever. ttlSecondsAfterFinished cleans up finished Jobs so they don't pile up. For CronJobs, concurrencyPolicy matters: Forbid skips a new run if the previous is still going, Replace cancels the old one, and Allow permits overlap, so I choose based on whether overlap is safe. startingDeadlineSeconds decides how late a missed run may still start, and I watch for the case where too many missed runs stop the schedule. Beyond that, the job itself must be idempotent, because retries and rare duplicate runs happen, with checkpoints for long work, proper resource requests, and restartPolicy OnFailure or Never. I alert on failed or missing runs, since silent batch failures are common.

Likely follow-ups

- Why must batch jobs be idempotent?
- How do you detect a CronJob that silently stopped running?

Jobs run pods to completion; CronJobs schedule Jobs. Reliability comes from tuning retries, parallelism, deadlines, and concurrency.

certs.eknathalabs.com

JOB KNOBS

```
spec:
  completions: 5           # run to 5 successful pods
  parallelism: 2           # 2 at a time
  backoffLimit: 4          # retries before marking failed
  activeDeadlineSeconds: 600   # hard time cap
  ttlSecondsAfterFinished: 3600 # auto-clean finished jobs
```

CRONJOB CONCURRENCY

```
concurrencyPolicy: Forbid     # don't start a new run if the last is still going
                              # (Allow / Forbid / Replace)
startingDeadlineSeconds: 100  # skip missed runs older than this
successfulJobsHistoryLimit: 3
```

RELIABILITY CONCERNS

Make jobs idempotent (they can re-run on retry/restart), set deadlines so a hung job doesn't run forever, use `ttlSecondsAfterFinished` to avoid pile-up, and handle the at-least-once nature (a job pod can run more than once).

Gotcha: Default `concurrencyPolicy: Allow` can overlap runs if a job runs longer than its schedule interval — pile-ups and double-processing. Use `Forbid` or `Replace` for non-idempotent work, and design jobs to be idempotent regardless.

Interview tip: Emphasize idempotency (Kubernetes gives at-least-once, a pod may re-run), `concurrencyPolicy` to prevent overlap, and `activeDeadlineSeconds`/`ttlSecondsAfterFinished` for hygiene. For complex DAGs of jobs, mention Argo Workflows. These details separate "I wrote a CronJob" from "I run reliable batch."

📚 **Official docs**

- Kubernetes: Jobs
- Kubernetes: CronJob

certs.eknathalabs.com

Q118 **How do descheduler and rebalancing work? Why doesn't Kubernetes rebalance pods on its own?**

Scheduling

🎙 Spoken answer · ~70 s

The Kubernetes scheduler makes a one-time decision when a pod is created and never moves it afterwards. Over time, nodes are added and drained, pods come and go, requests change, taints and labels change, so the original placement becomes suboptimal, with some nodes overloaded and others nearly empty, but nothing rebalances automatically. The Descheduler fills that gap. It runs as a Job or CronJob and evicts pods that violate configured strategies so the scheduler can place them better. Strategies include removing duplicate replicas from the same node, spreading pods that violate topology spread constraints, moving pods from over-utilised nodes to under-utilised ones, evicting pods that violate affinity or taints that have changed, and removing pods that restart too often or are too old. It respects PodDisruptionBudgets and skips pods without owners, such as bare pods, or critical ones. The risk is churn, since evicting pods causes disruption, so I tune thresholds conservatively and use it with good PDBs. Karpenter's consolidation does similar work for cost at the node level.

Likely follow-ups

- Why does the descheduler not place pods itself?
- What risks come with aggressive descheduling?

The scheduler places pods once and never moves them — so over time the cluster drifts out of balance. The Descheduler evicts pods so they get rescheduled better.

WHY K8S DOESN'T REBALANCE

```
# Scheduling is a one-time decision at pod creation.
# After that, nodes get added/drained, pods come/go, requests change ->
# the original placement becomes suboptimal, but nothing moves pods automatically.
```

THE DESCHEDULER

```
# Runs as a Job/CronJob; evicts pods matching strategies, e.g.:
- RemoveDuplicates              : spread replicas off the same node
- LowNodeUtilization            : drain under-used nodes' candidates to rebalance
- RemovePodsViolatingTopologySpread / InterPodAntiAffinity (rule drift)
- RemovePodsViolatingNodeTaints
```

Evicted pods are recreated by their controllers and rescheduled per current state. It respects PDBs and won't evict un-replicated pods.

Caution: The descheduler causes churn (evictions = brief disruption). Tune it and rely on PDBs so it doesn't disrupt availability. Karpenter's consolidation does similar rebalancing for cost by repacking nodes.

Interview tip: The key concept: scheduling is one-shot, so the cluster needs an external rebalancer (Descheduler, or Karpenter consolidation) to correct drift after nodes/pods change. Mentioning that eviction churn must be bounded by PDBs shows you'd deploy it safely, not blindly.

📚 Official docs

- Kubernetes: Node-pressure eviction

certs.eknathalabs.com

Q119 **How do you manage CRDs and operators at scale? What are the upgrade and versioning pitfalls?**

Cluster ops

🎙 Spoken answer · ~75 s

CRDs extend the Kubernetes API, and operators act on those resources, so at scale their versioning and upgrades become a real source of outages. CRDs support multiple versions, such as v1alpha1, v1beta1, and v1, with a conversion webhook between them. There's one stored version and several served versions, so migrating stored objects needs care, and the OpenAPI schema gates what users can submit. The pitfalls: Helm installs CRDs from the crds directory on first install only and does not upgrade them, so CRD changes are silently skipped and a new operator expects a schema that isn't there, so I manage CRDs separately, for example in their own chart or GitOps application. Deleting a CRD deletes all of its custom resources, which can destroy data, so I protect CRDs from accidental removal. A broken conversion webhook can break reads of the resource. Operators themselves need version compatibility with the CRD and the Kubernetes version. My approach is to pin versions, upgrade CRDs before operators, test upgrades in staging, read the release notes, back up the custom resources, and monitor operator health.

Likely follow-ups

- Why is deleting a CRD dangerous?
- How do you upgrade CRDs when using Helm?

CRDs extend the API; operators act on them. At scale, CRD versioning and operator upgrades become a real source of outages if mishandled.

CRD VERSIONING

```
- CRDs support multiple versions (v1alpha1, v1beta1, v1) with a conversion webhook
- Stored version + served versions; migrating stored objects needs care
- Schema validation (OpenAPI) gates what users can submit
```

UPGRADE PITFALLS

```
- Helm does NOT upgrade CRDs by default (only installs on first install)
  -> CRD changes get silently skipped; operator expects new schema -> breaks
- A breaking CRD schema change can invalidate existing CRs
- Conversion webhook down -> CRs unreadable -> operator + deletes stuck (finalizers!)
- Operator RBAC drift after upgrade
```

SAFE PRACTICE

Version CRDs additively, test conversion webhooks, manage CRD upgrades explicitly (not via Helm's skip), and stage operator upgrades in nonprod. Watch for the finalizer-stuck-Terminating failure mode when a conversion webhook or operator is unhealthy.

Interview tip: The two pitfalls interviewers love: (1) Helm skips CRD upgrades, so schema changes get missed, and (2) a down conversion webhook makes CRs unreadable and blocks deletion (ties back to finalizers/stuck-Terminating). Treating CRDs as a versioned API with backward-compatible evolution is the senior framing.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Version skew policy
- Kubernetes: Custom Resources

Q120 **You inherit an undocumented production cluster. How do you assess its health, risk, and cost?**

Cluster ops

🎙 **Spoken answer · ~73 s**

I'd run a structured audit that produces a prioritised risk picture quickly, covering reliability, security, and cost. First, inventory and reliability: Kubernetes versions and whether they're end of life, upcoming forced upgrades, workloads without requests and limits, PDBs, or more than one replica, single points of failure like single-replica critical services or single-zone deployments, and whether etcd and stateful volumes are backed up and have ever been restored. Second, security posture: run kube-bench for the CIS benchmark, review RBAC for cluster-admin bindings and wildcard roles, check Pod Security settings, exposed services and dashboards, secrets handling, image sources and scanning, and network policies. Third, observability and operations: what monitoring, alerting, and logging exist, and is there a runbook or on-call process. Fourth, cost: compare requests with actual usage, find idle nodes and orphaned volumes and load balancers, and check autoscaling and instance choices. Then I rank findings by likelihood and impact, fix the quick wins and the dangerous ones first, and document the cluster as I go, using Infrastructure as Code to capture the state.

Likely follow-ups

- What would you fix in the first week?
- How do you capture an undocumented cluster's state into code?

A structured audit across reliability, security, and cost gives you a prioritized risk picture fast — a realistic senior/staff scenario.

1. INVENTORY + RELIABILITY

```
- Versions: kubelet/control-plane version, is it EOL? upcoming forced upgrade?
- Workloads without requests/limits, PDBs, multiple replicas, anti-affinity
- Single points of failure: single-replica critical services, one-zone deploys
- Backups: is etcd backed up? are stateful PVs backed up? ever restored?
```

2. SECURITY POSTURE

```
- kube-bench (CIS), kube-hunter; Trivy scan running images
- RBAC sprawl: who has cluster-admin? kubectl auth can-i audits
- privileged pods, hostPath/hostNetwork, no NetworkPolicies, secrets in env/git
- Pod Security Admission enforced? image signing/registry allowlist?
```

3. COST

```
- Request-vs-usage gap (over-provisioned requests = wasted nodes)
- Idle/under-utilized nodes; oversized node pools; no autoscaling
- Orphaned PVs/LBs; right-sizing via VPA recommendations / Kubecost
```

certs.eknathalabs.com

THEN PRIORITIZE

Rank by risk x effort: backups + EOL version + cluster-admin sprawl are usually top. Document findings, quick wins first, build a remediation roadmap.

Interview tip: Structure beats trivia here — assess across reliability, security, cost, then prioritize by risk. Name concrete tools (kube-bench, Trivy, Kubecost, VPA) and the highest-impact gaps (no backups, EOL version, over-broad RBAC, no NetworkPolicies, request bloat). This is exactly the platform-ownership framing senior interviews probe for.

📚 **Start here (official docs)**

- Kubernetes documentation

Q121 **What is the difference between a Pod, a ReplicaSet and a Deployment, and why do you almost never create bare Pods?** Pods & workloads

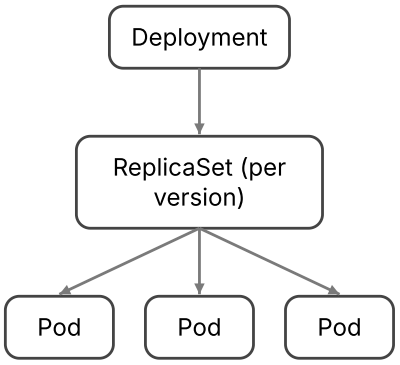
🎙 **Spoken answer · ~63 s**

A Pod is the smallest unit Kubernetes runs: one or more containers sharing network and storage. A ReplicaSet watches a label selector and makes sure a fixed number of identical Pods exist, recreating any that disappear. A Deployment sits above that. It owns ReplicaSets, and each change to the pod template creates a new ReplicaSet while the old one scales down, which is how rolling updates and rollbacks work. The chain is Deployment owns ReplicaSet owns Pod. I almost never create bare Pods in production because nothing recreates them if the node fails or someone drains it. The exceptions are throwaway debugging pods and static pods managed by the kubelet. For stateless apps I use a Deployment, for stable identity and storage a StatefulSet, for one per node a DaemonSet, and for batch work a Job. In an interview I would also mention revisionHistoryLimit, because old ReplicaSets are what rollback uses.

Likely follow-ups

- Where is rollback history stored?
- What does revisionHistoryLimit control?

🗺 **Deployment hierarchy**



A Pod is the smallest runnable unit, a ReplicaSet keeps N identical Pods alive, and a Deployment manages ReplicaSets so you get rolling updates and rollbacks. A bare Pod is not rescheduled if its node dies.

certs.eknathalabs.com

```
kubectl get deploy,rs,pods -l app=web
```

```
kubectl rollout history deploy/web
```

```
kubectl describe rs <name> | grep -i 'controlled by'
```

Interview tip: Say the ownership chain out loud: Deployment owns ReplicaSet owns Pod. It shows you understand where rollouts really happen.

📚 **Start here (official docs)**

- Kubernetes documentation

Q122 **What are the Kubernetes QoS classes and how do they decide who gets evicted first?** Scheduling

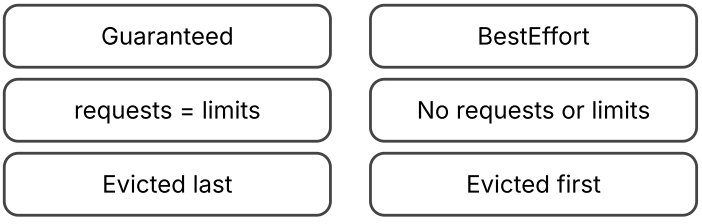
🎙 **Spoken answer · ~60 s**

QoS class is derived from the resource requests and limits on your containers, you do not set it directly. Guaranteed means every container sets CPU and memory requests equal to limits. Burstable means at least one request or limit is set but the pod does not meet the Guaranteed rule. BestEffort means nothing is set at all. It matters when a node runs short of memory or disk. The kubelet evicts BestEffort pods first, then Burstable pods that use more than they requested, and Guaranteed pods last. The same ordering feeds the Linux OOM score, so a Guaranteed pod is the last the kernel kills. For critical services I set requests equal to limits on memory at least, which gives predictable behaviour. For ordinary services I set realistic requests and a memory limit, and I am careful with CPU limits because they cause throttling.

Likely follow-ups

- Why can CPU limits hurt latency?
- How does the kubelet pick pods to evict?

🗺 **QoS classes**



Guaranteed means every container has requests equal to limits for CPU and memory. Burstable means at least one request or limit is set. BestEffort means none are set. Under node pressure the kubelet evicts BestEffort first, then Burstable pods exceeding requests, and Guaranteed last.

```
kubectl get pod <p> -o jsonpath='{.status.qosClass}'
```

```
kubectl describe node <n> | grep -A8 'Allocated resources'
```

certs.eknathalabs.com

Interview tip: Mention that QoS is derived, not configured, and that the OOM killer also uses it through oom_score_adj.

📚 Official docs

- Kubernetes: Pod Quality of Service classes
- Kubernetes: Node-pressure eviction

Q123 **How do Namespaces, ResourceQuota and LimitRange work together to share a cluster fairly?** Cluster ops

🎙 Spoken answer · ~59 s

A Namespace gives you a scope for names, RBAC and policies. ResourceQuota limits the aggregate: total CPU and memory requests, number of pods, services, PVCs, even object counts. LimitRange works at the individual object level. It sets default requests and limits, plus minimum and maximum values per container or pod, so a team cannot create a single huge pod or a pod with no limits. They work together. If a ResourceQuota covers CPU or memory, every new pod must specify them or be rejected, and a LimitRange default is what makes that painless. I use this for multi-team clusters: one namespace per team or environment, a quota sized to their budget, a LimitRange with sane defaults, and a NetworkPolicy plus RBAC on top. Namespaces alone are not a security boundary, so for hard isolation I would add policy engines or separate clusters.

Likely follow-ups

- Is a namespace a security boundary?
- What happens when a quota is exceeded?

A Namespace is the scope. ResourceQuota caps the total a namespace can consume or create, and LimitRange sets per-pod or per-container defaults and bounds, so a pod without requests still gets sensible ones.

```
kubectl describe quota -n team-a
```

```
kubectl describe limitrange -n team-a
```

```
kubectl create quota q --hard=cpu=20,memory=64Gi,pods=100 -n team-a
```

Interview tip: A quota on CPU or memory forces every pod to declare them, so pair it with a LimitRange default or deployments will be rejected.

📚 Official docs

- Kubernetes: Namespaces
- Kubernetes: Resource Quotas
- Kubernetes: LimitRange

certs.eknathalabs.com

Q124 **Compare ClusterIP, NodePort, LoadBalancer and ExternalName Services. When do you pick each?**

Networking

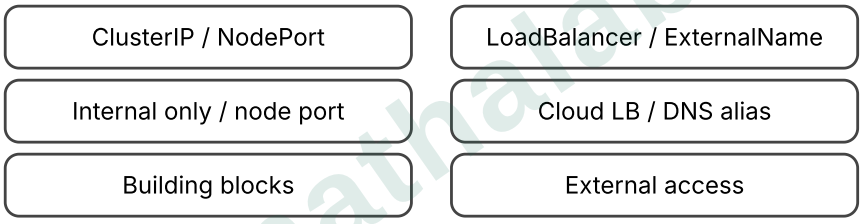
🎙 Spoken answer · ~56 s

ClusterIP is the default: a stable virtual IP reachable only inside the cluster, which is what service-to-service traffic uses. NodePort opens the same high port on every node and forwards to the Service, useful for simple tests or when an external load balancer is managed by hand. LoadBalancer builds on NodePort and asks the cloud provider to provision an external load balancer, so each Service gets its own cloud resource and cost. ExternalName is different, it creates a DNS CNAME to an outside hostname and does no proxying. In production I keep most Services as ClusterIP and expose a small number of entry points through an Ingress controller or Gateway API, so one load balancer serves many hosts and paths. I use a dedicated LoadBalancer Service for non-HTTP traffic such as TCP or UDP workloads.

Likely follow-ups

- How does externalTrafficPolicy change the source IP?
- When is a headless Service useful?

🗺 Service types



ClusterIP is internal only. NodePort opens a port on every node. LoadBalancer asks the cloud for an external load balancer in front of a NodePort. ExternalName is only a DNS CNAME to something outside the cluster.

```
kubectl get svc -o wide
```

```
kubectl expose deploy web --port=80 --target-port=8080 --type=ClusterIP
```

Interview tip: Add that most production traffic enters through one Ingress or Gateway rather than one cloud load balancer per Service, because of cost.

📚 Official docs

- Kubernetes: Service

certs.eknathalabs.com

Q125 **Name the control plane components and describe what each one does.** Cluster ops

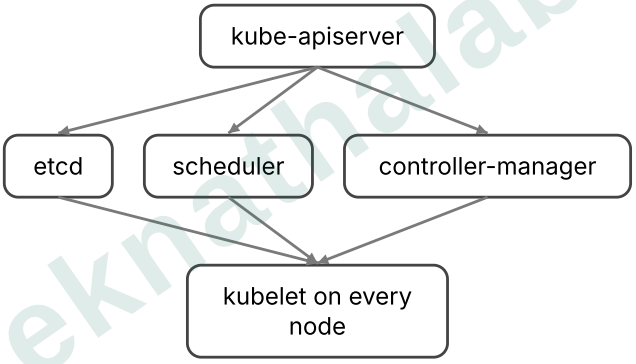
🎙 Spoken answer · ~55 s

The kube-apiserver is the single entry point. It authenticates and authorises requests, runs admission, validates objects and persists them to etcd. etcd is the consistent key-value store that holds all cluster state. The kube-scheduler watches for unscheduled pods and picks a node by filtering and scoring. The kube-controller-manager runs the reconciliation loops, such as the ReplicaSet, Node, Job and EndpointSlice controllers, each pushing actual state toward desired state. The cloud-controller-manager handles cloud-specific work such as load balancers and node lifecycle. On every node the kubelet runs pods through the container runtime and reports status, and kube-proxy or an eBPF replacement programs Service routing. The key idea is that components only talk through the API server, so if something looks stuck I start by checking the API server and the relevant controller logs.

Likely follow-ups

- What happens if the scheduler is down?
- Which component creates the pods of a Deployment?

🗺 Control plane



kube-apiserver is the only front door and talks to etcd. etcd stores state. kube-scheduler assigns pods to nodes. kube-controller-manager runs reconciliation loops. cloud-controller-manager talks to the cloud API. On nodes, kubelet and kube-proxy do the work.

```
kubectl get pods -n kube-system -o wide
```

```
kubectl get --raw='/readyz?verbose'
```

Interview tip: Stress that every component talks only to the API server, never to each other directly. That one fact explains most debugging paths.

📚 Start here (official docs)

- Kubernetes documentation

certs.eknathalabs.com

Q126 **ConfigMap as environment variables versus as a mounted volume: which one updates live, and what are the consequences?**

Pods & workloads

🎙 Spoken answer · ~59 s

When a ConfigMap is injected as environment variables, the values are copied into the container at startup, so changing the ConfigMap does nothing until the pod is restarted. When it is mounted as a volume, the kubelet eventually updates the files, usually within a minute, using an atomic symlink swap. The catch is that the application must reload the file itself, and a subPath mount never updates at all. Because of this I choose based on the app. If it supports hot reload, I mount the volume. Otherwise I prefer immutable config and trigger a rollout whenever it changes, for example with a checksum annotation in the pod template or a tool such as Reloader. Making ConfigMaps immutable also reduces API server load and avoids accidental live changes. I treat config changes like code changes: versioned, reviewed and rolled out gradually.

Likely follow-ups

- Why does subPath break updates?
- What does an immutable ConfigMap give you?

Environment variables are read at container start and never change. Mounted ConfigMap files are refreshed by the kubelet within a minute or so, except when mounted with subPath, which never updates.

```
kubectl set env deploy/web --from=configmap/app-config
```

```
kubectl rollout restart deploy/web
```

```
kubectl exec <pod> -- cat /etc/config/app.yaml
```

Interview tip: Mention a checksum annotation on the pod template, a common Helm trick, to force a rollout when the config changes.

📚 Official docs

- Kubernetes: ConfigMaps

certs.eknathalabs.com

Q127 **What are the postStart and preStop lifecycle hooks, and how do they interact with graceful shutdown?** Pods & workloads

🎙 Spoken answer · ~58 s

Lifecycle hooks let you run a command or HTTP call at two points. postStart fires immediately after the container starts, but it runs alongside the main process and there is no guarantee it finishes first, so I avoid depending on it. preStop runs when the pod is being terminated, before the container receives SIGTERM, and it is the right place to stop accepting new work and drain in-flight requests. The sequence is: the pod is marked Terminating, it is removed from Service endpoints, preStop runs, SIGTERM is sent, and after terminationGracePeriodSeconds the kubelet sends SIGKILL. The grace period includes the preStop time. The classic production issue is that endpoint removal and shutdown race, so clients still hit a pod that is closing. A short sleep in preStop, plus an app that handles SIGTERM properly, usually fixes dropped requests during deployments.

Likely follow-ups

- Why do users see 502s during a rollout?
- What happens if preStop exceeds the grace period?

postStart runs right after the container is created, without any ordering guarantee against the entrypoint. preStop runs before SIGTERM is sent and is used to drain connections. Both count toward terminationGracePeriodSeconds.

```
kubectl explain pod.spec.containers.lifecycle
```

```
kubectl get pod <p> -o jsonpath='{.spec.terminationGracePeriodSeconds}'
```

Interview tip: A short sleep in preStop is a common fix for 502s during rollouts, because endpoint removal and SIGTERM race each other.

📚 Official docs

- Kubernetes: Pod lifecycle & termination

certs.eknathalabs.com

Q128 **How does kubectl rollout undo work, and where are revisions stored?** CI/CD & GitOps

🎙 Spoken answer · ~58 s

A Deployment stores its history as ReplicaSets. Every change to the pod template, such as a new image or env var, creates a new ReplicaSet with a revision annotation. Old ReplicaSets are kept at zero replicas, up to revisionHistoryLimit, which defaults to ten. kubectl rollout undo simply scales an earlier ReplicaSet back up and the current one down using the same rolling update strategy, so it is safe and gradual. Changing replica count alone does not create a revision, because it does not touch the template. Adding a change-cause annotation makes the history readable. One thing I always call out is GitOps: if Argo CD or Flux manages the Deployment, a manual undo drifts from Git and will be reverted, so the correct rollback is to revert the commit. Also remember that rollback does not undo database migrations.

Likely follow-ups

- Does rollback restore ConfigMaps?
- What if revisionHistoryLimit is zero?

Each pod-template change creates a new ReplicaSet, and the Deployment keeps old ones up to revisionHistoryLimit. Undo scales an old ReplicaSet back up and the current one down. Scaling or changing replicas alone does not create a revision.

```
kubectl rollout history deploy/web
```

```
kubectl rollout undo deploy/web --to-revision=3
```

```
kubectl annotate deploy/web kubernetes.io/change-cause='bump to 1.4.2'
```

Interview tip: In a GitOps setup, prefer reverting the commit. A manual undo will be overwritten at the next sync.

📚 Official docs

- Kubernetes: Deployments

certs.eknathalabs.com

Q129 **Explain how a Helm chart is structured and what helm template, helm upgrade --install and helm diff are for.**

CI/CD & GitOps

🎙 Spoken answer · ~60 s

A Helm chart packages Kubernetes manifests as Go templates. Chart.yaml holds metadata and dependencies, values.yaml holds defaults, and the templates folder holds the manifests plus helpers. A release is an installed instance of a chart, with its history stored as Secrets in the namespace. helm template renders the manifests locally without touching a cluster, which is ideal for linting, policy checks and reviewing what will be applied. helm upgrade with the install flag is the idempotent command pipelines use: it installs if the release does not exist and upgrades otherwise. The diff plugin compares the proposed release to what is live, which I want in every pull request. In CI I use atomic and a timeout so a failed upgrade rolls back automatically. For environments, I keep one chart with a values file per environment, and pin chart versions rather than using latest.

Likely follow-ups

- Where does Helm store release state?
- When would you choose Kustomize instead?

A chart has Chart.yaml, values.yaml and a templates folder. helm template renders manifests locally with no cluster. helm upgrade --install is idempotent deploy. helm diff, a plugin, shows what would change before you apply.

```
helm template web ./chart -f values-prod.yaml
```

```
helm upgrade --install web ./chart -n prod --atomic --timeout 5m
```

```
helm diff upgrade web ./chart -n prod
```

Interview tip: Use --atomic or --wait in CI so a failed release rolls itself back instead of leaving a half-deployed state.

📚 Official docs

- Helm: Chart best practices
- Helm: Chart hooks
- Helm documentation

certs.eknathalabs.com

Q130 **What is an EndpointSlice, and why did it replace the Endpoints object?** Networking

🎙 **Spoken answer · ~55 s**

The original Endpoints object held every backend address for a Service in a single resource. In a Service with thousands of pods, each pod change rewrote that large object and pushed it to every node watching it, which created API server and network load. EndpointSlices fix this by splitting the addresses into multiple small objects, by default up to a hundred endpoints each. Only the affected slice is updated and sent. They also carry richer data such as ready, serving and terminating conditions, node name and topology zone, which supports features like topology-aware routing and graceful termination. kube-proxy, Cilium and most ingress controllers now consume EndpointSlices. For day-to-day debugging I still check that a Service has endpoints, and if the list is empty I look at the selector, the pod labels and readiness.

Likely follow-ups

- What does it mean if a Service has no endpoints?
- How does topology-aware routing use slices?

Endpoints stored every pod IP of a Service in one object, so a change to one pod rewrote and resent the whole thing to every node. EndpointSlices shard that into small chunks, default 100 endpoints each, which scales far better.

```
kubectl get endpointslices -l kubernetes.io/service-name=web
```

```
kubectl get endpoints web -o yaml
```

Interview tip: Mention that kube-proxy and the ingress controllers now watch EndpointSlices, so large Services update much faster.

📚 **Official docs**

- Kubernetes: EndpointSlices

certs.eknathalabs.com

Q131 **How do you manage access to multiple clusters with kubeconfig contexts without making mistakes?**

Cluster ops

🎙 Spoken answer · ~57 s

A kubeconfig file contains three lists: clusters with their API endpoints and CA, users with their credentials, and contexts that tie a cluster, a user and a default namespace together. The risk with several clusters is running a command against the wrong one. My habits are to show the current context and namespace in the shell prompt, colour production differently, pass the context explicitly in scripts with the context flag, and keep separate kubeconfig files per environment selected with the KUBECONFIG variable. For authentication I avoid long-lived static tokens and use the cloud provider's exec plugin or OIDC login so credentials are short-lived and tied to a person. I also use RBAC so day-to-day access to production is read-only, with elevated access granted just in time. Tools like kubectx help, but the safeguard is process, not tooling.

Likely follow-ups

- How do you give temporary admin access?
- Why avoid static tokens in kubeconfig?

A kubeconfig holds clusters, users and contexts. A context binds one cluster, one user and a default namespace. Switch with kubectl config use-context, or better, use the KUBECONFIG variable per shell and a prompt that shows the context.

```
kubectl config get-contexts
```

```
kubectl config use-context prod-eu -n payments
```

```
kubectl --context=staging get pods
```

```
kubectl auth whoami
```

Interview tip: Name a real habit: a coloured prompt for prod, and short-lived credentials from SSO or an exec plugin rather than static tokens.

📚 Start here (official docs)

- Kubernetes documentation

certs.eknathalabs.com

Q132 **When do you use kubectl debug node, and how is it different from SSH to the node?** Cluster ops

🎙 Spoken answer · ~58 s

kubectl debug node creates a pod scheduled on a specific node, with the host's root filesystem mounted under slash host and host namespaces available. From there I can read kubelet and container runtime logs, inspect disk usage, check iptables or conntrack, and chroot into the host if I need host tools. Compared with SSH, there is no need to open port 22 or manage keys, access is authorised through Kubernetes RBAC, and every action starts with an auditable API call. It is also often the only option on managed or immutable node images, such as Bottlerocket or Talos, where SSH is disabled by design. Because the pod is privileged, I restrict who can create it, clean it up when done, and avoid making changes by hand on nodes. Anything I learn goes back into automation or node images.

Likely follow-ups

- What RBAC permission does this need?
- How would you debug a node on Talos?

kubectl debug node/ starts a privileged pod on that node with the host filesystem mounted at /host. It gives you host-level access without opening SSH, and the access is audited through the API server.

```
kubectl debug node/ip-10-0-1-5 -it --image=busybox
```

```
chroot /host
```

```
journalctl -u kubelet --since '10 min ago'
```

Interview tip: Delete the debug pod afterwards, and mention that on managed or immutable OS nodes this is often the only way in.

📚 Official docs

- Kubernetes: Debug running Pods (kubectl debug)

certs.eknathalabs.com

Q133 **What are static pods, and which cluster components commonly run as static pods?** Cluster ops

🎙 Spoken answer · ~58 s

Static pods are pods the kubelet manages straight from manifest files in a configured directory, typically slash etc slash kubernetes slash manifests. The kubelet watches the folder, starts the pod, restarts it if it dies, and creates a read-only mirror pod in the API server so it is visible with kubectl. Because they do not depend on the control plane, they solve the bootstrap problem. In a kubeadm cluster the API server, controller manager, scheduler and etcd all run as static pods. To change an API server flag, I edit the manifest on that node and the kubelet recreates the pod, and kubectl edit on the mirror pod will not work. I also use static pods rarely for node-critical agents. The trade-off is that they are not managed by a controller, so there is no rolling update or central management.

Likely follow-ups

- Why can't you delete a static pod with kubectl?
- How would you upgrade control plane flags safely?

Static pods are defined as manifest files in a directory on a node, and the kubelet runs them directly with no API server involved. A mirror pod is created in the API so you can see them. kubeadm control plane components use this.

```
ls /etc/kubernetes/manifests
```

```
grep staticPodPath /var/lib/kubelet/config.yaml
```

```
kubectl get pods -n kube-system | grep -E 'apiserver|etcd'
```

Interview tip: This is why you edit a file on the control plane node, not use kubectl edit, to change the API server flags in a kubeadm cluster.

📚 Start here (official docs)

- Kubernetes documentation

certs.eknathalabs.com

Q134 **What do the kubelet and the container runtime each do, and what is CRI?** Cluster ops

🎙 Spoken answer · ~56 s

The kubelet runs on every node. It watches for pods assigned to its node, asks the runtime to pull images and start containers, runs probes, mounts volumes, reports status and enforces resource limits. It does not run containers itself. It speaks the Container Runtime Interface, a gRPC API, to a runtime such as containerd or CRI-O, and those in turn use a low-level runtime such as runc to create the actual Linux containers. Kubernetes removed the built-in Docker integration, dockershim, in version 1.24, but images built with Docker are standard OCI images and still run. For debugging on a node I use crictl, since the docker command may not exist. The usual failure points are a stopped kubelet, a full disk, runtime socket problems, or certificate expiry, so I check journalctl for the kubelet first.

Likely follow-ups

- What is the difference between containerd and runc?
- How do you list containers on a node without Docker?

The kubelet is the node agent that makes pods match the spec. It talks to a container runtime such as containerd or CRI-O through the Container Runtime Interface, a gRPC API. Docker Engine support through dockershim was removed in 1.24.

```
crictl ps
```

```
crictl pods
```

```
systemctl status kubelet containerd
```

```
journalctl -u kubelet -f
```

Interview tip: If asked about dockershim, say images built with Docker still run fine, because they are OCI images.

📚 Official docs

- Kubernetes: Container runtimes
- Kubernetes: kubelet
- containerd documentation

certs.eknathalabs.com

Q135 **What does volumeBindingMode WaitForFirstConsumer solve, and why does it matter on multi-zone clusters?** Storage

🎙 Spoken answer · ~58 s

Block storage such as EBS or a persistent disk lives in a single availability zone. With Immediate binding, Kubernetes provisions the volume as soon as the PVC is created, before any pod exists, so the scheduler may later be forced to place the pod in a different zone than the disk, leaving the pod stuck Pending with a volume node affinity conflict. WaitForFirstConsumer delays provisioning until a pod that uses the claim is scheduled. The scheduler then considers all the pod's constraints, picks a node, and the volume is created in that node's zone. It matters for multi-zone clusters and StatefulSets. A side effect is that the PVC stays Pending until a pod consumes it, which is expected. I set this mode on every zonal StorageClass, and combine it with topology spread so replicas land in different zones.

Likely follow-ups

- What is a volume node affinity conflict?
- How does this interact with pod anti-affinity?

With Immediate binding, a zonal disk is created before the pod is scheduled, possibly in a zone where the pod cannot run. WaitForFirstConsumer delays provisioning until the pod is scheduled, so the disk is created in the same zone as the node.

```
kubectl get sc -o custom-columns=NAME:.metadata.name,MODE:.volumeBindingMode
```

```
kubectl describe pvc data-0 | grep -i 'waiting for first consumer'
```

Interview tip: A PVC showing Pending with this mode is normal until a pod uses it. Do not misread it as a failure.

📚 Official docs

- Kubernetes: StorageClasses

certs.eknathalabs.com

Q136 **Explain the PersistentVolume access modes: ReadWriteOnce, ReadOnlyMany, ReadWriteMany and ReadWriteOncePod.**

Storage

🎙 Spoken answer · ~59 s

Access modes describe how many nodes can use a volume at once, and what they can do. ReadWriteOnce means a single node can mount it read-write. Note that it is per node, not per pod, so multiple pods on the same node can share it. ReadOnlyMany lets many nodes mount it read-only. ReadWriteMany lets many nodes mount it read-write, but block devices cannot do this, so you need a shared file system such as NFS, EFS, Azure Files or CephFS. ReadWriteOncePod restricts it to exactly one pod in the whole cluster and is the right choice when you need strict single-writer safety. The storage backend must support the mode you request or the claim stays Pending. In design reviews I ask whether the app really needs shared writes, because RWX file systems are slower and often the wrong tool for databases.

Likely follow-ups

- Why does a Deployment with RWO and rolling updates sometimes hang?
- Which mode suits a database?

RWO allows one node to mount read-write, not one pod. ROX allows many nodes read-only. RWX allows many nodes read-write and needs a shared filesystem such as NFS or EFS. RWOP restricts the volume to a single pod.

```
kubectl get pv -o custom-columns=NAME:.metadata.name,MODES:.spec.accessModes
```

```
kubectl explain pvc.spec.accessModes
```

Interview tip: Point out the RWO trap: several pods on the same node can still share an RWO volume.

📚 Official docs

- Kubernetes: Persistent Volumes

certs.eknathalabs.com

Q137 **What is the PersistentVolume reclaim policy, and when would you choose Retain over Delete?**

Storage

🎙 Spoken answer · ~58 s

The reclaim policy decides what happens to the storage when the claim bound to a PersistentVolume is deleted. With Delete, which is the default for dynamically provisioned volumes, the cloud disk is destroyed along with the PVC. That is convenient for dev and test but dangerous in production, because deleting a namespace or a StatefulSet's claims can destroy data. With Retain, the PV moves to a Released state and the disk stays, so an administrator can recover the data, but the volume is not automatically reused and needs manual cleanup or a new PV that references it. For production databases I set Retain on the StorageClass, or patch the PV, and add snapshots and tested restores, since Retain protects against accidental deletion, not corruption or region failure. I also enable protection finalizers so an in-use PVC cannot be removed.

Likely follow-ups

- How do you reuse a Released volume?
- What does the storage-object-in-use protection do?

Delete removes the underlying disk when the PVC is deleted, which is the default for dynamic provisioning. Retain keeps the disk and the PV in Released state, so data survives and an admin can recover it.

```
kubectl get pv -o custom-
columns=NAME:.metadata.name,POLICY:.spec.persistentVolumeReclaimPolicy
```

```
kubectl patch pv <pv> -p '{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}'
```

Interview tip: Say that Retain is not a backup. It only prevents accidental deletion, so you still need snapshots.

📚 Official docs

- Kubernetes: Persistent Volumes

certs.eknathalabs.com

Q138 **What is the difference between an Ingress resource and an Ingress controller, and how do you terminate TLS?** Networking

🎙 Spoken answer · ~55 s

The Ingress resource is just configuration: hosts, paths and which Service each should go to. By itself it does nothing. An Ingress controller is the running component that watches Ingress objects and programs a real proxy or cloud load balancer, and the IngressClass says which controller owns which Ingress. For TLS, the Ingress references a Secret of type kubernetes.io/tls containing the certificate and key, and the controller terminates HTTPS and forwards to pods, often over plain HTTP inside the cluster or re-encrypted if required. I normally automate certificates with cert-manager and Let's Encrypt or a private CA. Ingress is limited to HTTP and relies on annotations for advanced features, so newer designs use the Gateway API, which has clearer roles and richer routing. For TCP or UDP I use a LoadBalancer Service.

Likely follow-ups

- Why is Gateway API replacing Ingress?
- How do you do mTLS at the edge?

An Ingress resource is only routing rules in the API. An Ingress controller, such as NGINX, Traefik or a cloud ALB controller, watches those rules and configures a real proxy. TLS is terminated at the controller using a certificate stored in a Secret.

```
kubectl get ingressclass
```

```
kubectl describe ingress web
```

```
kubectl get secret web-tls -o jsonpath='{.type}'
```

Interview tip: Mention that without an installed controller an Ingress object does nothing, which is a common first-day surprise.

📚 Official docs

- Kubernetes: Ingress

certs.eknathalabs.com

Q139 How does cert-manager issue and renew certificates inside a cluster?

Security

🎙 Spoken answer · ~57 s

cert-manager adds custom resources for certificates. An Issuer or ClusterIssuer describes who signs, for example Let's Encrypt through ACME, HashiCorp Vault, or an internal CA. A Certificate resource states the names you need and the Secret to store them in. You can also annotate an Ingress and cert-manager creates the Certificate for you. For ACME it creates an Order and Challenges. HTTP-01 proves control by serving a token over port 80, DNS-01 proves it by creating a DNS record, which also supports wildcard certificates. Once issued, the key and certificate go in a Secret, which Ingress or workloads mount. cert-manager renews automatically, by default when about a third of the lifetime remains. When it fails, I look at the Certificate, CertificateRequest, Order and Challenge events in that order, and alert on expiry as a safety net.

Likely follow-ups

- When would you choose DNS-01?
- How do pods pick up a renewed cert?

You define an Issuer or ClusterIssuer, such as Let's Encrypt or Vault. A Certificate resource, or an annotated Ingress, requests a cert. cert-manager completes the ACME challenge, stores the result in a Secret, and renews it before expiry.

```
kubectl get clusterissuer,certificate -A
```

```
kubectl describe certificate web-tls
```

```
cmctl status certificate web-tls
```

Interview tip: Name the usual failure: the HTTP-01 challenge is blocked by a redirect or a firewall. DNS-01 avoids it and supports wildcards.

📚 Official docs

- cert-manager documentation

certs.eknathalabs.com

Q140 **What are good practices for Kubernetes labels, selectors and annotations?** Pods & workloads

🎙 Spoken answer · ~55 s

Labels are key-value pairs used to group and select objects: Services pick pods, Deployments own ReplicaSets, and policies target workloads, all by label selectors. Annotations hold metadata that tools use but that is never selected on, for example ingress settings, checksums or build information. My practices are to adopt the standard app.kubernetes.io labels for name, instance, version, component and part-of, add ownership and environment labels for cost allocation, keep selector labels minimal and stable because a Deployment's selector is immutable, and avoid putting changing values like the version in the selector. I never store secrets in either, and I keep annotation size modest. A common production bug is overlapping selectors, where a Service or Deployment accidentally adopts pods it should not own, so I review selectors carefully and use unique names per workload.

Likely follow-ups

- Why is a Deployment selector immutable?
- How would you find all pods by team for cost reports?

Labels identify and select; annotations hold non-identifying metadata. Use the recommended app.kubernetes.io set, keep selectors stable because Deployment selectors are immutable, and never put secrets or large data in either.

```
kubectl get pods -l 'app.kubernetes.io/name=web,tier in (frontend,api)'
```

```
kubectl label pod <p> debug=true --overwrite
```

```
kubectl get pods --show-labels
```

Interview tip: A well-known trap: if a Service selector matches pods from two Deployments, traffic is split between them.

📚 Start here (official docs)

- Kubernetes documentation

certs.eknathalabs.com

Q141 **What are native sidecar containers, and how do they differ from the old sidecar pattern?**

Pods & workloads

🎙 Spoken answer · ~57 s

Traditionally a sidecar was just another container in the pod's container list. That had two problems: there was no start ordering, so the app could start before the proxy was ready, and in a Job the pod never finished because the sidecar kept running. Native sidecars, enabled by default in recent Kubernetes versions, are declared as init containers with restartPolicy Always. They start in order before the main containers, stay running for the life of the pod, support their own probes, and are terminated after the main containers exit. That makes service mesh proxies, log shippers and credential refreshers behave properly, and lets batch Jobs complete. I would use a sidecar when the helper must share the pod's network or volumes and have the same lifecycle, and choose a separate Deployment or DaemonSet when it is independent.

Likely follow-ups

- How is this different from a regular init container?
- What happens to the sidecar at pod shutdown?

Native sidecars are init containers with restartPolicy Always. They start before the app, keep running alongside it, are restarted independently, and are stopped after the app ends. This fixes ordering and Job completion problems of regular sidecars.

```
kubectl explain pod.spec.initContainers.restartPolicy
```

```
kubectl get pod <p> -o jsonpath='{.status.initContainerStatuses[*].name}'
```

Interview tip: The headline benefit is that a Job with a log or proxy sidecar can now actually complete.

📚 Official docs

- Kubernetes: Sidecar containers
- Microsoft: Sidecar pattern

certs.eknathalabs.com

Q142 **Why do many teams remove CPU limits, and what does CFS throttling have to do with it?** Scheduling

🎙 Spoken answer · ~59 s

CPU requests set the share a container gets under contention, and the scheduler uses them for placement. A CPU limit is a hard cap enforced by the Linux CFS quota: the container may use a certain amount of CPU time per hundred millisecond period, and once it is spent the threads are paused until the next period. For multi-threaded or bursty applications this means you can be throttled even though the node has idle CPU, which shows up as latency spikes. You can see it in the container CPU throttled periods metric. Because of this, many teams set accurate CPU requests and no CPU limit, letting pods burst into spare capacity, while keeping memory limits because memory cannot be reclaimed safely. The trade-off is noisy neighbours, so I keep requests honest, use Guaranteed class for critical pods, and monitor node saturation.

Likely follow-ups

- How do you measure throttling?
- What is the downside of having no CPU limit?

CPU limits are enforced by the CFS quota in 100 ms periods. A multi-threaded app can use its whole quota in a few milliseconds and then be paused for the rest of the period, which hurts tail latency even when the node is idle.

```
rate(container_cpu_cfs_throttled_periods_total[5m]) /
rate(container_cpu_cfs_periods_total[5m])
```

```
kubectl top pod --containers
```

Interview tip: Keep CPU requests accurate, since requests drive scheduling and fair sharing. Memory limits you should keep.

📚 Official docs

- Kubernetes: Resource management for Pods & containers
- Linux kernel docs: CFS scheduler

certs.eknathalabs.com

Q143 **How do you write a default-deny NetworkPolicy that still lets pods resolve DNS?** Security

🎙 Spoken answer · ~58 s

By default pods accept traffic from anywhere. To get zero trust I first apply a policy per namespace with an empty pod selector and policy types set to both Ingress and Egress, with no rules, which denies everything. The first thing that breaks is DNS, because egress to CoreDNS is blocked, so I immediately add an egress rule that allows UDP and TCP port 53 to the kube-dns pods in kube-system, selected with a namespace selector and a pod label. Then I add explicit allow policies per application: the frontend can reach the API, the API can reach the database, and the ingress controller can reach the frontend. I test each step with a debug pod. It is vital to check that the CNI actually enforces policies, since some do not, and to remember that policies are additive.

Likely follow-ups

- How do you allow traffic to a cloud metadata service?
- How do you test policies before enforcing them?

Create an empty podSelector policy with both Ingress and Egress types, then add an egress allow to the kube-dns pods on UDP and TCP 53 in the kube-system namespace. Without that rule, all name lookups break.

```
kubectl apply -f default-deny.yaml
```

```
kubectl exec test -- nslookup kubernetes.default
```

```
kubectl describe netpol -n prod
```

Interview tip: Always confirm your CNI enforces NetworkPolicy. Some, like plain Flannel, silently ignore it.

📚 Official docs

- Kubernetes: Network Policies

certs.eknathalabs.com

Q144 What is Kubernetes audit logging, and how would you use it to investigate an incident?

Security

🎙 Spoken answer · ~60 s

Audit logging records requests handled by the API server, each as a structured event with the user, groups, source IP, verb, resource, namespace, response code and time. An audit policy decides what is recorded and at which level: None, Metadata, Request, or RequestResponse. I record Metadata for most things, log secrets only at Metadata so values never reach the logs, and capture more detail for RBAC changes and exec calls. The output goes to a file or a webhook, and from there to a log platform. In an incident I answer questions like who deleted this deployment, who created this cluster role binding, which service account listed secrets, and from which IP. On EKS, GKE and AKS I turn on the managed control plane audit logs, since they are not always enabled. Alerts on exec into pods, privileged pod creation and RBAC changes are valuable.

Likely follow-ups

- Why not log secrets at RequestResponse?
- Which events would you alert on?

The API server can record every request according to an audit policy, with levels None, Metadata, Request and RequestResponse. Logs go to a file or webhook, and you query who did what, to which resource, when, and from where.

```
kube-apiserver --audit-policy-file=... --audit-log-path=...
```

```
jq 'select(.verb=="delete" and .objectRef.resource=="secrets")' audit.log
```

Interview tip: On managed clusters, enable the provider's control plane logs, because audit is often off by default.

📚 Start here (official docs)

- Kubernetes documentation

certs.eknathalabs.com

Q145 **What is the difference between metrics-server and Prometheus, and where does kubectl top get its data?**

Observability

🎙 Spoken answer · ~60 s

metrics-server is a lightweight aggregator. It scrapes the kubelet's resource metrics endpoint, keeps the latest CPU and memory numbers in memory, and exposes them through the metrics API. That is what kubectl top and the Horizontal Pod Autoscaler read. It stores no history and has no application metrics, so it cannot support dashboards or alerting. Prometheus is a full monitoring system: it scrapes targets such as kubelet cAdvisor, kube-state-metrics, node exporters and application endpoints, stores time series, and lets you query with PromQL and fire alerts. For custom or external metrics the HPA needs an adapter such as the Prometheus adapter or KEDA. In production you need both: metrics-server for autoscaling and quick checks, Prometheus for history and alerts. If kubectl top returns an error I check that metrics-server is running and that its APIService is available, then look at kubelet TLS settings.

Likely follow-ups

- Why does the HPA show unknown for metrics?
- What does kube-state-metrics add?

metrics-server keeps only current CPU and memory in memory for kubectl top and the HPA. It stores no history. Prometheus scrapes many metrics and stores time series for dashboards, alerting and analysis.

```
kubectl top nodes
```

```
kubectl top pods -A --sort-by=memory
```

```
kubectl get apiservices | grep metrics
```

Interview tip: If kubectl top fails, check the metrics-server pod and the v1beta1.metrics.k8s.io APIService first.

📚 Official docs

- Kubernetes: kubectl quick reference
- Prometheus: Overview

certs.eknathalabs.com

Q146 **What is KEDA, and when would you use it instead of the standard HPA?** Scheduling

🎙 Spoken answer · ~58 s

The standard HPA scales a workload from resource metrics like CPU and memory, or custom metrics if you add an adapter, and it needs at least one replica. KEDA is an operator that adds event-driven autoscaling. You define a ScaledObject that points at a trigger such as SQS or RabbitMQ queue depth, Kafka consumer lag, a Prometheus query or a cron schedule. KEDA creates and manages an HPA behind the scenes and also handles the zero-to-one activation, so idle workers can scale to zero and cost nothing. I use it for queue consumers and batch workers where load is driven by backlog rather than CPU. I avoid scale to zero for latency-sensitive services because of cold start. Important settings are cooldown periods, polling interval and a sensible maximum replica count so a burst does not overwhelm downstream systems.

Likely follow-ups

- What happens to in-flight messages at scale-down?
- How do you cap downstream load?

KEDA scales workloads on event sources such as queue length, Kafka lag or cron schedules, and can scale to zero. It feeds the HPA with external metrics. The plain HPA mostly scales on CPU and memory and cannot go to zero.

```
kubectl get scaledobject -A
```

```
kubectl describe scaledobject worker
```

```
kubectl get hpa
```

Interview tip: Scale-to-zero suits workers, not user-facing APIs, because of cold start latency.

📚 Official docs

- Kubernetes: Horizontal Pod Autoscaling
- KEDA documentation

certs.eknathalabs.com

Q147 **Pods are being evicted because of DiskPressure on a node. How do you investigate and prevent a repeat?** Cluster ops

🎙 Spoken answer · ~62 s

I start with the node condition and events to confirm DiskPressure, then check which filesystem is affected: the node root, or the image and container storage. Using a debug pod I run du on the container runtime directory and var log to see what is large. Usual culprits are unrotated container logs, a pile of unused images, pods writing big files to emptyDir or their writable layer, and core dumps. Immediate relief is to prune unused images, remove old logs, or cordon and drain the node and replace it. Long term I set container log rotation in the kubelet, set ephemeral-storage requests and limits so a misbehaving pod is evicted rather than the node suffering, tune image garbage collection thresholds, and move heavy temporary data to a proper volume. I also alert on node disk usage before eviction thresholds, so we act at eighty percent, not at the edge.

Likely follow-ups

- What are the kubelet eviction thresholds?
- How do ephemeral-storage limits help?

The kubelet evicts pods when nodefs or imagefs falls under its threshold, and also garbage-collects images. Find what is filling the disk: container logs, images, emptyDir, or writable layers.

```
kubectl describe node <n> | grep -A5 Conditions
```

```
kubectl debug node/<n> -it --image=busybox -- du -xh /host/var/lib --max-depth=2 | sort -h | tail
```

```
crictl images; crictl rmi --prune
```

Interview tip: Name three preventers: log rotation limits, ephemeral-storage requests and limits, and image garbage collection thresholds.

📚 Official docs

- Kubernetes: Node-pressure eviction

certs.eknathalabs.com

Q148 **A Deployment rollout finished but some old pods are stuck in Terminating forever. What is going on?**

Pods & workloads

🎙 Spoken answer · ~58 s

First I look at the pod's deletion timestamp, finalizers, node and events. If the node is NotReady or unreachable, the kubelet cannot report the container as gone, so the pod stays Terminating until the node returns or is removed. If there are finalizers, some controller has not finished its cleanup, so I find that controller and fix it rather than removing the finalizer blindly. If the node is healthy, the application may be ignoring SIGTERM, with a long terminationGracePeriodSeconds, or a volume unmount is hanging, which I see in kubelet logs. Force deleting the pod object removes it from the API but does not guarantee the process is dead, which is dangerous for StatefulSets and volumes. So I only force delete after confirming the node or process is gone. Prevention is handling SIGTERM correctly and keeping sensible grace periods.

Likely follow-ups

- Why is force delete risky for StatefulSets?
- How do you find which controller owns a finalizer?

A pod stays Terminating when a finalizer blocks it, the node is unreachable so the kubelet cannot confirm deletion, the process ignores SIGTERM and the grace period is very long, or a volume cannot be unmounted.

```
kubectl get pod <p> -o jsonpath='{.metadata.finalizers} {.metadata.deletionTimestamp}'
```

```
kubectl describe pod <p> | tail -20
```

```
kubectl delete pod <p> --force --grace-period=0
```

Interview tip: Force delete is a last resort. For StatefulSets it can allow two pods with the same identity, which risks data corruption.

📚 Official docs

- Kubernetes: Deployments

certs.eknathalabs.com

Q149 **ImagePullBackOff happens only on some nodes, not all. How do you find the cause?**

Pods & workloads

🎙 Spoken answer · ~58 s

The key clue is that the same image works elsewhere, so I focus on what differs per node. I look at which nodes fail and read the pod events for the exact error. A rate limit message points to anonymous Docker Hub pulls shared by a NAT address, so I add imagePullSecrets or a registry mirror. An unauthorized error suggests node credentials, such as a missing ECR policy on one node group's IAM role. A timeout or no such host suggests DNS, proxy, security group or route problems on that subnet. I reproduce on the node using crictl pull, and check disk space, since a full image filesystem also fails pulls. The fix is usually to standardise node roles and networking, use a pull-through cache in our own registry, and pin images by digest so behaviour is consistent.

Likely follow-ups

- How do you avoid Docker Hub limits at scale?
- Why pin images by digest?

If the same image works on some nodes, the problem is node-specific: registry rate limits per IP, missing credentials or a node role without ECR access, DNS or proxy issues on that node, or a full disk.

```
kubectl get pod <p> -o wide
```

```
kubectl describe pod <p> | grep -A10 Events
```

```
crictl pull <image>  # run on the failing node
```

```
kubectl debug node/<n> -it --image=busybox
```

Interview tip: Docker Hub anonymous pull limits are a classic. Use authenticated pulls or a pull-through cache.

📚 Official docs

- Kubernetes: Debug Pods
- Kubernetes: Images (pull policy, registries)
- Kubernetes: Pull an image from a private registry

certs.eknathalabs.com

Q150 **A Service works from inside the cluster but is unreachable from outside through its NodePort. How do you debug?**

Networking

🎙 Spoken answer · ~61 s

I walk from the inside out. First confirm the Service has ready endpoints, because a selector or readiness problem would also break internal access. Since the ClusterIP works, the pods are probably fine, so the problem is the external path. I test the NodePort from another machine on the same network, then from the node itself. If it works locally but not remotely, it is a cloud security group, network ACL or firewall blocking the NodePort range. If one node works and another does not, I check externalTrafficPolicy: with Local, only nodes running a backend pod respond, and the load balancer health check should exclude the others. I also check kube-proxy is running and its rules exist. The fix is opening the right security group rule or switching traffic policy, and for the long term I prefer a managed load balancer or Ingress over raw NodePorts.

Likely follow-ups

- What does externalTrafficPolicy Cluster do to the client IP?
- How does a load balancer health check work with Local?

Check the layers in order: pod readiness and endpoints, kube-proxy rules on the node, cloud security groups or firewall for the port range 30000-32767, externalTrafficPolicy, and whether the node you hit has a local pod.

```
kubectl get svc web -o wide; kubectl get endpointslices -l kubernetes.io/service-name=web
```

```
curl -v http://<nodeIP>:<nodePort>
```

```
sudo iptables -t nat -L KUBE-NODEPORTS | grep <port>
```

Interview tip: With externalTrafficPolicy Local, nodes without a local pod drop the traffic. This is by design and preserves the source IP.

📚 Official docs

- Kubernetes: Debug Services
- Kubernetes: Service

certs.eknathalabs.com

Q151 **Every pod creation suddenly fails with a webhook timeout or certificate error. What happened and how do you recover?**

Security

🎙 Spoken answer · ~58 s

This is the classic self-inflicted outage. An admission webhook, for example from Gatekeeper, Kyverno or a service mesh injector, is called by the API server on matching requests. If the webhook pods are down, or the serving certificate expired, and the failure policy is Fail, every matching request is rejected, including the request that would recreate the webhook pods, which creates a deadlock. To recover I list the webhook configurations, find the one whose errors appear in the API server message, and temporarily delete it or set its failure policy to Ignore, then fix the underlying certificate or deployment and restore it. To prevent a repeat, I exclude system namespaces and the webhook's own namespace using namespace selectors, set short timeouts, run multiple replicas with disruption budgets, automate certificate rotation, and alert on webhook latency and error rates.

Likely follow-ups

- When is failurePolicy Fail appropriate?
- How do you roll out a new policy safely?

A validating or mutating admission webhook is unreachable or its serving certificate expired, and its failurePolicy is Fail, so the API server rejects all matching requests. Disable or fix the webhook configuration to unblock.

```
kubectl get validatingwebhookconfigurations,mutatingwebhookconfigurations
```

```
kubectl delete validatingwebhookconfiguration <name>
```

```
kubectl logs -n gatekeeper-system deploy/gatekeeper-controller-manager
```

Interview tip: Exempt kube-system and the webhook's own namespace, and set a timeout. The webhook that guards pods must not depend on pods it blocks.

📚 Official docs

- Cloudflare Learning: TLS handshake

certs.eknathalabs.com

Q152 **etcd reports 'mvcc: database space exceeded' and the cluster turns read-only. How do you recover?**

Cluster ops

🎙 Spoken answer · ~60 s

When the etcd database passes its quota, it raises a NOSPACE alarm and the cluster stops accepting writes, so the API server becomes effectively read-only. First I take a snapshot for safety. Then I compact the key history to the latest revision, which marks old versions as free, defragment each member one at a time to actually return the space, and finally disarm the alarm. Defrag blocks the member while it runs, so I do it sequentially, followers first. Then I investigate the root cause. Common causes are a huge number of Events, a controller or operator that updates objects in a loop, large Secrets or CRs, or leftover Jobs. I fix the source, enable automatic compaction, set an alert on database size, and, if needed, raise the quota modestly. On managed clusters I would open a support ticket because I cannot reach etcd directly.

Likely follow-ups

- Why isn't compaction enough on its own?
- How do you spot which resource type is largest?

etcd hits its quota, default 2 GB, raises a NOSPACE alarm and only accepts reads and deletes. Compact the history, defragment, then disarm the alarm. Find out what grew: usually events or a controller writing too much.

```
ETCDCTL_API=3 etcdctl endpoint status -w table
```

```
rev=$(etcdctl endpoint status -w json | jq '.[0].Status.header.revision'); etcdctl compact $rev
```

```
etcdctl defrag; etcdctl alarm disarm
```

Interview tip: Take a snapshot before you touch anything, and defrag one member at a time because it blocks that member.

📚 Official docs

- Kubernetes: Operating etcd clusters
- etcd: Disaster recovery
- PostgreSQL: Concurrency control (MVCC)

certs.eknathalabs.com

Q153 **A pod is stuck in ContainerCreating. What are the likely causes and how do you narrow them down?**

Pods & workloads

🎙 Spoken answer · ~60 s

ContainerCreating means the scheduler has done its job and the kubelet is preparing the pod. I go straight to the pod events because they usually give the reason. FailedMount or FailedAttachVolume points to storage: a missing PVC, a volume still attached to another node, a missing Secret or ConfigMap, or a CSI driver problem. FailedCreatePodSandBox often means the CNI plugin could not set up networking, for example because the node ran out of IP addresses or the CNI pod is unhealthy. Slow progress with pulling messages suggests registry or bandwidth problems with large images. If events are quiet, I read kubelet and container runtime logs on the node. Once I know the layer, I fix it, such as correcting the object name, freeing the volume, or replacing a node with exhausted IPs. I would also add alerts for pods stuck longer than a few minutes.

Likely follow-ups

- What does FailedCreatePodSandBox usually indicate?
- How do you check if a CSI driver is healthy?

The pod is scheduled but the kubelet cannot finish setup. Typical causes are a failing volume mount or attach, a missing ConfigMap or Secret, CNI problems assigning an IP, or the runtime failing to pull or create the sandbox.

```
kubectl describe pod <p> | sed -n '/Events/,$p'
```

```
kubectl get events --field-selector involvedObject.name=<p>
```

```
journalctl -u kubelet | grep <pod>
```

Interview tip: Events usually name the culprit in one line, such as FailedMount, FailedCreatePodSandBox or FailedAttachVolume.

📚 Official docs

- Kubernetes: Debug Pods

certs.eknathalabs.com

Q154 **External lookups from pods are slow and some calls take about five seconds. How do ndots and CoreDNS play a role?**

Networking

🎙 Spoken answer · ~57 s

Inside a pod, resolv.conf has several search domains and ndots set to five. Any name with fewer than five dots is first tried with each search suffix, so looking up an external host like api.example.com can trigger several failed queries before the right one, and each failure adds load and latency. If CoreDNS is overloaded, or there is a conntrack race on UDP, a lost packet causes a classic five second timeout. I check CoreDNS CPU and logs, its cache and forward settings, and the resolv.conf. Fixes include using fully qualified names with a trailing dot for external calls, lowering ndots with dnsConfig for chatty workloads, scaling CoreDNS with the cluster proportional autoscaler, and deploying NodeLocal DNSCache so lookups hit a local cache over TCP. I would also measure with dnsperf style tests to confirm the improvement.

Likely follow-ups

- What is NodeLocal DNSCache?
- Why does the UDP race cause 5 second delays?

The default ndots:5 makes the resolver try several search-domain suffixes before the real name, multiplying queries. Combined with the UDP conntrack race, timeouts of 5 seconds appear. Use fully qualified names, tune ndots, and add NodeLocal DNSCache.

```
kubectl exec <p> -- cat /etc/resolv.conf
```

```
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=50
```

```
kubectl top pod -n kube-system -l k8s-app=kube-dns
```

Interview tip: A trailing dot on external hostnames, api.example.com., skips the search path entirely.

📚 Official docs

- Kubernetes: Debugging DNS resolution
- Kubernetes: DNS for Services and Pods
- CoreDNS manual

certs.eknathalabs.com

Q155 **Pods cannot get IP addresses because the node ran out of IPs. How does that happen on EKS and what are the options?** Networking

🎙 Spoken answer · ~59 s

The AWS VPC CNI gives each pod a secondary private IP from the node's elastic network interfaces. The instance type limits how many interfaces and addresses it can hold, so it limits the pods per node, and subnets with small CIDR blocks can be exhausted as the cluster grows, leaving pods in ContainerCreating with IP allocation errors. I check the aws-node logs and subnet free address counts. Remedies include enabling prefix delegation, which assigns blocks of sixteen addresses per slot and greatly increases density, using custom networking so pods draw from a secondary CIDR such as the 100.64 range, adding larger subnets, or choosing an overlay or Cilium based networking. I would also right-size the number of nodes and consider IPv6. Long term, I plan pod CIDR capacity at design time and alert when subnets drop below a free address threshold.

Likely follow-ups

- What is prefix delegation?
- How does an overlay network avoid this?

With the AWS VPC CNI, each pod uses a real VPC IP drawn from the ENIs attached to the node, so the instance type's ENI and IP limits cap pods per node and small subnets run dry. Use prefix delegation, custom networking or larger subnets.

```
kubectl describe node <n> | grep -i 'pods:'
```

```
kubectl logs -n kube-system -l k8s-app=aws-node --tail=30
```

```
aws ec2 describe-subnets --query 'Subnets[].[SubnetId,AvailableIpAddressCount]'
```

Interview tip: Say the formula: max pods = ENIs x (IPs per ENI - 1) + 2. It shows you know why small instances fit few pods.

📚 Official docs

- Amazon EKS: User guide

certs.eknathalabs.com

Q156 **A pod fails with 'Multi-Attach error for volume' after a node failure. Why, and how do you resolve it safely?**

Storage

🎙 Spoken answer · ~59 s

A block volume with ReadWriteOnce can be attached to only one node. When a node dies without cleanly detaching, the VolumeAttachment object still says the disk belongs to it, so when the pod is rescheduled to a new node, the attach fails with a Multi-Attach error. Kubernetes eventually force-detaches after a timeout, around six minutes, once the node is considered unhealthy, and newer versions with non-graceful node shutdown handling can speed that up. Manually, I first confirm the old node is really down or terminated, preferably by stopping or deleting the instance, because if it is still running and writing, detaching risks data corruption. Then I delete the stale VolumeAttachment or the node object so the volume can attach to the new node. For better resilience I use replicated storage or databases with their own replication, instead of depending on one disk.

Likely follow-ups

- What is non-graceful node shutdown?
- How would a StatefulSet recover faster?

A ReadWriteOnce block volume is still attached to the failed node, so the CSI attacher will not attach it elsewhere until the old attachment is released. Wait for the force-detach timeout, or confirm the node is truly down and remove the VolumeAttachment.

```
kubectl get volumeattachment | grep <pv>
```

```
kubectl describe pod <p> | grep -i multi-attach
```

```
kubectl delete volumeattachment <name>   # only after node is confirmed gone
```

Interview tip: Never force the detach while the old node might still be writing. That can corrupt the filesystem.

📚 Start here (official docs)

- Kubernetes documentation

certs.eknathalabs.com

Q157 **The Horizontal Pod Autoscaler keeps flapping between scale up and scale down. How do you stabilise it?**

Scheduling

🎙 Spoken answer · ~59 s

First I watch the HPA events and the metric values to see what triggers each decision. Flapping typically has a few causes. The target utilisation may sit right at the normal operating point, so small changes cross the threshold. The metric may be spiky, like instantaneous CPU. New pods may take long to start, so the average drops, then the HPA scales down, and load rises again. To stabilise I use the behavior section: a longer scale-down stabilization window of five minutes, policies limiting how many pods can be removed per minute, and faster scale-up. I choose a steadier metric, such as requests per second or queue depth, set the CPU requests accurately since utilisation is relative to requests, make sure readiness probes reflect true readiness, and set minReplicas high enough to absorb normal load. Then I test with load to verify.

Likely follow-ups

- Why is CPU utilisation relative to requests?
- What does stabilizationWindowSeconds do?

Flapping comes from noisy metrics, short windows, a target near the natural load level, or slow pod startup. Use stabilizationWindowSeconds, scaling policies, a better metric, and readiness so new pods are counted correctly.

```
kubectl describe hpa web
```

```
kubectl get hpa web -w
```

```
kubectl get hpa web -o yaml | grep -A12 behavior
```

Interview tip: Scale up fast, scale down slowly is the usual rule. The behavior field lets you set each direction separately.

📚 Official docs

- Kubernetes: Horizontal Pod Autoscaling

certs.eknathalabs.com

Q158 **Users see 502 errors for a few seconds on every deployment. How do you make rollouts truly zero-downtime?**

Pods & workloads

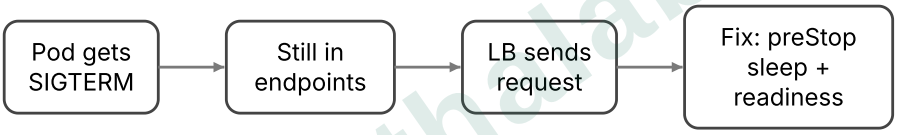
🎙 Spoken answer · ~59 s

The 502s mean requests reach a pod that cannot answer. There are two directions. On the way up, a pod may be marked Ready too early, so I make sure the readiness probe checks real dependencies and warm-up, and use a startup probe for slow apps. On the way down, endpoint removal and SIGTERM happen at the same time, so load balancers and the proxy keep sending requests for a few seconds. The fix is a short sleep in a preStop hook, an application that stops accepting new connections and finishes in-flight requests on SIGTERM, and a grace period longer than the longest request. I set maxUnavailable to zero with a surge, add a PodDisruptionBudget, and align load balancer deregistration delay and keep-alive timeouts with the application. Then I verify under load during a rollout, not just in a quiet test.

Likely follow-ups

- Why does SIGTERM race with endpoint removal?
- What should maxUnavailable be for a critical service?

🗺 502s on deploy



Traffic is still sent to terminating pods, or new pods receive traffic before they are ready. Add accurate readiness probes, a preStop sleep, SIGTERM handling, enough grace period, a PodDisruptionBudget and sensible maxUnavailable.

```
kubectl get deploy web -o jsonpath='{.spec.strategy}'
```

```
kubectl describe pod <p> | grep -A6 -E 'Readiness|Liveness'
```

```
kubectl rollout status deploy/web
```

Interview tip: Also check keep-alive and idle timeouts on the load balancer, because they can differ from the app's.

📚 Official docs

- Kubernetes: Deployments
- RFC 9110: HTTP Semantics (status codes, methods)

certs.eknathalabs.com

Q159 **The Cluster Autoscaler is not adding nodes even though pods are Pending. How do you investigate?** Scheduling

🎙 Spoken answer · ~61 s

I begin with a pending pod's events. If I see NotTriggerScaleUp, the message lists why each node group was rejected. Typical reasons are that the node group is already at its maximum size, that the pod requests more CPU or memory than the largest instance type, that node selectors, tolerations or zone constraints match no group, or that a PVC bound to one zone cannot use a group in another zone. Then I read the autoscaler logs and status ConfigMap. Next I check outside Kubernetes: cloud quotas, insufficient capacity for the instance type, launch template errors, and IAM permissions or tags that let the autoscaler discover the group. The fix depends on the cause, such as raising the maximum, adding more instance types, correcting requests, or moving to Karpenter which provisions right-sized nodes directly. I add an alert on pods pending longer than a few minutes.

Likely follow-ups

- How is Karpenter different in this situation?
- Why can PVC zones block scale up?

Check why the pods are Pending and whether a new node would help. Common reasons: node group at max size, pod requests larger than any node type, taints or affinity no node group satisfies, quotas, or the autoscaler not discovering the group.

```
kubectl -n kube-system logs deploy/cluster-autoscaler | grep -i -E 'scale-up|no.*node group'
```

```
kubectl get cm cluster-autoscaler-status -n kube-system -o yaml
```

```
kubectl describe pod <p> | grep -A5 Events
```

Interview tip: The status ConfigMap and the pod's TriggeredScaleUp or NotTriggerScaleUp event usually spell out the reason.

📚 Official docs

- Kubernetes: Debug Pods
- Kubernetes Autoscaler: Cluster Autoscaler

certs.eknathalabs.com

Q160 **A secret was committed to a Git repository and used in a Kubernetes manifest. What is your response?**

Security

🎙 Spoken answer · ~60 s

My first action is to revoke and rotate the credential, because once a secret has been pushed I assume it is exposed, even if the repository is private. Next I limit the blast radius: find what the secret grants access to and review the provider's access logs and the cluster audit logs for suspicious use since the commit time. Then I replace it in the cluster with the new value and roll out the workloads that use it. Cleaning history with git filter-repo is good hygiene, but it is not the fix. After the incident I address the root cause: add pre-commit and pipeline secret scanning, stop storing plain secrets in manifests by using External Secrets Operator, Sealed Secrets, or SOPS with a cloud key management service, prefer workload identity over static keys, and write a short post-incident review so the team learns without blame.

Likely follow-ups

- How does External Secrets Operator work?
- Why prefer workload identity over keys?

Treat it as compromised: rotate the credential first, then remove it from the cluster and history, check audit logs and cloud logs for use, and put a control in place such as secret scanning and an external secret manager.

```
git log -S'AKIA' --all --oneline
```

```
kubectl get secret app-creds -o yaml | grep -c data
```

```
trufflehog git file://. --only-verified
```

Interview tip: Rewriting Git history is not enough. Anyone who cloned already has it, so rotation is the only real fix.

📚 Start here (official docs)

- Kubernetes documentation

certs.eknathalabs.com

Q161 **A developer gets 'forbidden' from kubectl exec. How do you find the missing RBAC permission quickly?**

Security

🎙 Spoken answer · ~60 s

The forbidden message is precise: it states which user cannot perform which verb on which resource in which namespace. For exec the resource is the pods/exec subresource with the create verb, which is easy to miss when someone granted only get, list and watch on pods. I confirm with kubectl auth can-i using impersonation, and list everything the user can do with the list flag. Then I check which RoleBindings and ClusterRoleBindings reference the user or their groups. With single sign-on, I check that the group claim in the token matches the subject name in the binding, as a mismatch is a frequent cause. I then grant the least privilege needed, ideally a Role limited to the namespace and expiring access, and document it in code. For production, I might not allow exec at all, and instead provide ephemeral debug containers with audit.

Likely follow-ups

- Why is pods/exec a sensitive permission?
- How do group claims reach RBAC?

Read the error, which names the user, verb, resource and namespace. Use kubectl auth can-i with impersonation, check bindings that reference the user or groups, and remember exec needs create on pods/exec, not just get on pods.

```
kubectl auth can-i create pods/exec -n prod --as dev@corp.com
```

```
kubectl get rolebindings,clusterrolebindings -A -o wide | grep dev@corp.com
```

```
kubectl auth can-i --list -n prod --as dev@corp.com
```

Interview tip: For group-based access from OIDC, verify the group claim actually reaches the API server, since bindings to a group do nothing otherwise.

📚 Official docs

- Kubernetes: RBAC

certs.eknathalabs.com

Q162 **Packets are intermittently dropped and dmesg shows 'nf_conntrack: table full'. What do you do?**

Networking

🎙 **Spoken answer · ~55 s**

Linux tracks every connection through iptables in a fixed size table. When the table fills, new packets are dropped, which shows up as random timeouts, and the kernel logs table full. On busy Kubernetes nodes this is often caused by many short-lived connections, such as services without keep-alive, high-volume DNS over UDP, or NAT heavy traffic through Services. I check the current count against the maximum and look at which workloads open the most connections. A quick fix is raising nf_conntrack_max and the hash size with sysctl, making sure the node has the memory for it. Real solutions reduce churn: enable connection reuse, add NodeLocal DNSCache, lower unnecessary timeouts, and spread chatty pods across nodes. Cilium with kube-proxy replacement reduces dependence on conntrack. I also add node-level monitoring so we see it coming.

Likely follow-ups

- Why does DNS over UDP stress conntrack?
- How does eBPF reduce conntrack usage?

The node's connection tracking table is exhausted, usually by many short connections, high-traffic DNS over UDP, or leaked entries. Raise nf_conntrack_max and hashsize, reduce connection churn, and consider eBPF datapaths that avoid conntrack.

```
sysctl net.netfilter.nf_conntrack_count net.netfilter.nf_conntrack_max
```

```
conntrack -S
```

```
sudo sysctl -w net.netfilter.nf_conntrack_max=524288
```

Interview tip: Track the count over max ratio as a node metric and alert on it before drops start.

📚 **Official docs**

- Netfilter documentation

certs.eknathalabs.com

Q163 **You suspect a memory leak in a service. How do you confirm it from Kubernetes and Prometheus data?** Observability

🎙 Spoken answer · ~61 s

I plot container working set memory per pod over several days and compare it with the memory limit. A leak shows as a rising baseline after each garbage collection or restart, and eventually OOMKilled restarts, visible in the last terminated state. I make sure I'm using working set bytes, because that is the figure the kernel cgroup limit applies to, rather than cache-inclusive usage. I correlate the growth with traffic, deployments and specific endpoints to see what triggers it. Then I move to application level evidence: language runtime metrics like heap size and GC time, heap profiles or dumps taken from a canary, tools like pprof, async-profiler or memory profilers, and look for unbounded caches, goroutine or thread leaks, and unclosed connections. As a mitigation I set a right-sized limit, add a restart policy, and alert on the growth slope, while the code fix is worked on.

Likely follow-ups

- Why working set rather than usage?
- What else besides heap can exceed the limit?

Look for a sawtooth that never returns to baseline in container working set memory, restarts with OOMKilled, and a steady upward trend across days. Then use a profiler or heap dump in a staging copy to find the cause.

```
container_memory_working_set_bytes{pod=~"web-.*"}
```

```
kubectl get pod <p> -o
jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'
```

```
deriv(container_memory_working_set_bytes[1h]) > 0
```

Interview tip: Use working set, not RSS or usage, since the working set is what the OOM killer compares to the limit.

📚 Official docs

- Prometheus: Overview

certs.eknathalabs.com

Q164 **Prometheus keeps running out of memory. How do you find the high-cardinality metrics and fix them?**

Observability

🎙 Spoken answer · ~62 s

Prometheus holds the recent data and the index of all active series in memory, so memory grows with series count, which is the product of metrics, label combinations and targets. I start with the TSDB status endpoint or promtool analyze to list the metric names and labels with the most series. The usual cause is a label with unbounded values: user IDs, request paths with IDs inside, pod names in a high churn environment, or histogram buckets multiplied by many labels. Fixes are to remove or normalise those labels in the application, use metric relabeling or drop rules at scrape time, lower the number of scraped targets and histogram buckets, and shorten retention on the local store. For scale I shard by team with several Prometheus instances, use remote write to Thanos, Mimir or a managed service, and enforce series limits per scrape so one team cannot break monitoring.

Likely follow-ups

- What is cardinality, in one sentence?
- How do series limits protect Prometheus?

Memory scales with active time series. Find the biggest metrics and labels, drop or relabel unbounded labels like user ID or full URL, reduce scrape targets, and shard or move long-term storage to Thanos, Mimir or similar.

```
topk(10, count by (__name__)({__name__=~".+"}))
```

```
curl -s localhost:9090/api/v1/status/tsdb | jq '.data.seriesCountByMetricName[:10]'
```

```
promtool tsdb analyze /prometheus
```

Interview tip: The TSDB status page and promtool analyze show the worst offenders in seconds.

📚 Official docs

- Prometheus: Metric and label naming
- Prometheus: Overview

certs.eknathalabs.com

Q165 **An Argo CD application is stuck OutOfSync or Progressing. How do you diagnose it?**

CI/CD & GitOps

🎙 Spoken answer · ~61 s

I start with the app diff to see exactly which fields differ. A permanent OutOfSync is often caused by something else mutating the resource: an HPA changing replicas, a mutating webhook injecting sidecars, or API server defaulting that Git does not include. For those I configure ignoreDifferences or respect-ignore-differences rather than fighting it. If the app is Progressing, I look at the resource health tree to see which object is not healthy, such as a Deployment waiting for pods, a Job from a PreSync hook that never completes, or a custom resource without a health check defined. I read the operation state and controller logs for sync errors like invalid manifests, missing CRDs or permission problems. Fixes include correcting the manifest, ordering resources using sync waves, adding custom health checks, and setting retry limits. I also keep a sync window and alerts so problems surface early.

Likely follow-ups

- What are sync waves and hooks?
- Why would an HPA cause OutOfSync?

OutOfSync means live state differs from Git: check the diff, defaulted fields, and controllers mutating resources. Progressing means health checks have not passed: pods not ready, a hook stuck, or a CRD with no health definition.

```
argocd app diff web
```

```
argocd app get web --show-operation
```

```
kubectl describe application web -n argocd
```

```
argocd app sync web --dry-run
```

Interview tip: Use ignoreDifferences for fields that other controllers legitimately change, such as replicas managed by the HPA.

📚 Official docs

- Argo CD documentation

certs.eknathalabs.com

Q166 **helm upgrade failed and the release is stuck in 'pending-upgrade'. How do you recover?**

CI/CD & GitOps

🎙 Spoken answer · ~61 s

Helm records each release revision as a Secret, and if an upgrade is interrupted, for example when the pipeline is cancelled or the connection drops, the latest revision can remain in a pending state, and later upgrades report that another operation is in progress. I check the release history to see the revisions and statuses. The preferred recovery is helm rollback to the last good deployed revision, which creates a new revision and clears the problem. If rollback is blocked, I carefully delete the specific stuck revision Secret in the namespace, after backing it up, then rerun the upgrade. Before retrying I find out why the first upgrade failed, using helm template and dry run, and look at pod events. To prevent a repeat I use atomic with a timeout, avoid cancelling deployment jobs, use a concurrency lock in CI, and run helm diff in review.

Likely follow-ups

- Where does Helm store release data?
- What does --atomic do on failure?

A previous Helm operation was interrupted, leaving a lock-like release status. Check history, roll back to the last deployed revision, or if no rollback is possible remove the stuck release Secret, then fix the chart and upgrade again with --atomic.

```
helm history web -n prod
```

```
helm rollback web 7 -n prod
```

```
kubectl get secret -n prod -l owner=helm,name=web,status=pending-upgrade
```

```
helm upgrade --install web ./chart -n prod --atomic
```

Interview tip: Cancelling CI in the middle of a Helm run is the usual cause. Make pipeline jobs non-cancellable during deploy.

📚 Official docs

- Kubernetes: Debug Pods
- Helm: Chart hooks
- Helm documentation

certs.eknathalabs.com

Q167 **After a node scale-down or zone issue, pods are packed unevenly and one zone holds most replicas. How do you fix and prevent it?**

Scheduling

🎙 Spoken answer · ~58 s

Kubernetes schedules a pod only when it is created. After a scale-down, a node failure or a zone event, the replicas that survive stay where they are, so a service can end up concentrated in one zone, which defeats high availability. To fix the current state I can do a rollout restart so the scheduler places new pods with the right constraints, or run the descheduler to evict pods violating spread rules. To prevent it, I add topologySpreadConstraints on the zone and hostname keys with a maxSkew of one, using DoNotSchedule for strict workloads or ScheduleAnyway for flexible ones, together with enough capacity in every zone. I also use PodDisruptionBudgets so draining nodes does not remove too many replicas at once, and make sure the autoscaler has node groups in every zone. Then I test by simulating a zone loss.

Likely follow-ups

- What does maxSkew mean?
- When would you not use DoNotSchedule?

The scheduler only places pods at creation time and does not rebalance. Use topologySpreadConstraints with maxSkew, whenUnsatisfiable settings, PodDisruptionBudgets, and optionally the descheduler to rebalance existing pods.

```
kubectl get pods -l app=web -o wide --sort-by=.spec.nodeName
```

```
kubectl get nodes -L topology.kubernetes.io/zone
```

```
kubectl rollout restart deploy/web
```

Interview tip: Add minDomains or use nodeAffinityPolicy and nodeTaintsPolicy in newer versions to make spread behave predictably with autoscaling.

📚 Start here (official docs)

- Kubernetes documentation

certs.eknathalabs.com

Q168 **Users get 504 Gateway Timeout or 413 errors through the Ingress for certain requests. Where do you look?** Networking

🎙 Spoken answer · ~60 s

I look at the ingress controller logs to see which status, upstream and timing are involved. A 504 means the proxy waited for the upstream longer than its read timeout, 60 seconds by default in NGINX. I check whether the backend is genuinely slow, for example a heavy query or exhausted connection pool, whether pods are overloaded, and whether the cloud load balancer in front of the ingress has its own idle timeout that is shorter. Raising the proxy read timeout for a specific long-running endpoint is acceptable, but I prefer making long operations asynchronous. A 413 means the request body is larger than the proxy body size limit, one megabyte by default in NGINX, so uploads need a higher limit on that Ingress. I change these on specific Ingress objects through annotations, not globally, and I make sure the application and every hop agree.

Likely follow-ups

- Where else can a timeout be set in the path?
- How would you handle large uploads?

504 means the proxy gave up waiting for the backend, so check upstream timeouts and slow pods. 413 means the request body exceeds the proxy limit. Tune per-Ingress annotations, but first find out why the backend is slow.

```
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller | grep -E ' 504 | 413 '
```

```
kubectl annotate ingress web nginx.ingress.kubernetes.io/proxy-read-timeout='120'
nginx.ingress.kubernetes.io/proxy-body-size='50m' --overwrite
```

Interview tip: Raising the timeout hides a slow backend. Check the application latency first and the load balancer idle timeout in front too.

📚 Official docs

- Kubernetes: Ingress
- RFC 9110: HTTP Semantics (status codes, methods)
- Google SRE Book: Effective Troubleshooting

certs.eknathalabs.com

Q169 **A cluster upgrade is planned and some manifests use deprecated APIs. How do you avoid breaking workloads?**

Cluster ops

🎙 Spoken answer · ~58 s

Before the upgrade I inventory which API versions I use. The API server exposes a metric that counts requests to deprecated APIs, and tools like Pluto and kubent scan live clusters, Helm releases and manifests in Git for removed versions. I read the release notes and deprecation guide for each minor version in the path, since the upgrade must go one minor at a time. I convert the manifests to supported versions, update Helm charts and operators, and check the add-ons such as the CNI, CSI drivers, ingress controller, metrics-server and service mesh for compatibility with the target version. Then I upgrade a non-production cluster first and run smoke tests. For production I back up etcd, upgrade the control plane, then roll the nodes gradually with surge nodes and PodDisruptionBudgets. A written runbook with rollback criteria reduces risk.

Likely follow-ups

- Why only one minor version at a time?
- How do you test add-on compatibility?

Find the deprecated and removed API versions in use, in live objects and in Git or Helm charts, convert them before upgrading, and upgrade one minor version at a time, with add-ons checked for compatibility.

```
kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis
```

```
pluto detect-files -d ./manifests
```

```
kubent
```

```
kubectl convert -f old.yaml --output-version networking.k8s.io/v1
```

Interview tip: Check the Kubernetes deprecation guide for each target version. Removals such as PodSecurityPolicy or old Ingress versions have broken real clusters.

📚 Official docs

- Kubernetes: Upgrading kubeadm clusters
- Kubernetes: Deprecated API migration guide

certs.eknathalabs.com

Q170 **A Kubernetes Job never finishes, and a CronJob has missed several schedules. How do you troubleshoot?**

Pods & workloads

🎙 Spoken answer · ~60 s

For a Job I read its status, the pod events and logs. If pods repeatedly fail, the backoff limit eventually marks the Job failed, so I fix the cause. If the Job stays active, the main container may not exit, or a sidecar such as a proxy never terminates, which the native sidecar feature solves. I add activeDeadlineSeconds so a stuck run is stopped. For a CronJob that misses schedules, I check whether it is suspended, the concurrencyPolicy, since Forbid skips a run if the previous one is still active, and startingDeadlineSeconds. If more than a hundred schedules are missed, the controller stops scheduling and logs an error. Clock and timezone settings also matter. To make batch reliable I make the work idempotent, set history limits and ttlSecondsAfterFinished, send alerts on failed Jobs and on a missing successful run, and make sure resources are sufficient.

Likely follow-ups

- What does concurrencyPolicy Replace do?
- How do you alert on a CronJob that never ran?

A Job hangs if pods keep failing under backoffLimit, a sidecar keeps running, or the container never exits. A CronJob misses runs because of concurrencyPolicy, startingDeadlineSeconds, suspended state, or too many missed starts.

```
kubectl describe job nightly-report
```

```
kubectl get cronjob nightly -o yaml | grep -E 'suspend|concurrencyPolicy|startingDeadline'
```

```
kubectl logs job/nightly-report
```

Interview tip: Set activeDeadlineSeconds and ttlSecondsAfterFinished so stuck Jobs terminate and finished ones are cleaned up.

📚 Official docs

- Kubernetes: Jobs
- Kubernetes: CronJob

certs.eknathalabs.com